

TheKernel 操作系统设计与实现报告

基于 StarryOS 的 Rust 宏内核深度实现

陈天意

DOTADIW / T202610346999652

杭州师范大学(HZNU)

2026 年6 月30 日

摘要

TheKernel 是一个以 Rust 实现的 Linux 兼容宏内核，同时面向 RISC-V64 与 LoongArch64 两个架构，由 StarryOS/ArceOS 生态演进而来。报告从设计动机、技术方案、关键数据结构与执行推演四个层面说明内核各子系统的实现原理。

在对象模型层面，内核为 Linux 用户态可见的每类状态指派明确的所有者对象。ProcessData 承载进程级共享状态，包括地址空间、文件描述符表、凭证、命名空间、信号处置与作业控制；Thread 承载线程局部状态，包括 TID、信号掩码、robust futex、clear-child-tid、系统调用重启记录与调度实体。进程与线程的显式分离，使 fork 与 clone 的语义差异、信号递送的归属判定、以及多线程退出的资源回收都能落到稳定的数据所有者之上，而不必在各调用点临时拼装状态。

在系统调用与虚拟内存层面，系统调用层在 dispatcher 入口统一处理双架构 ABI 差异、restart 分类、fast path 与错误传播，将架构相关的寄存器布局与公共的语义处理解耦。地址空间以 Backend 抽象为核心，将 COW、文件映射、共享内存与线性映射统一到同一的生命周期框架中，使缺页处理、卸载回收与权限变更沿一致路径推进。

在文件系统与网络层面，文件系统采用 FileLike trait、FileDescription 与 fd table 三层模型，使普通文件、目录、管道、socket、timerfd、signalfd、pidfd、epoll、inotify、fanotify、userfaultfd 等文件类对象复用同一套 I/O 与 poll 路径。网络栈 axnet-ng 为本项目自维护模块，通过 SocketOps trait 与 Socket enum 抽象 TCP、UDP、Unix domain 与 vsock 传输层，再经 fd wrapper 接入内核的 I/O 多路复用模型。

相对 StarryOS 基线，TheKernel 的技术增量集中在三个方向：首先是对象模型深化，将进程、线程、文件描述符与虚拟内存后端拆成更清晰的所有权边界；其次是子系统补全，补入 in-tree 网络栈、mount ledger、扩展 pseudofs 与 BPF 支撑；最后是双架构工程化，在 syscall alias、restart bookkeeping 与 QEMU virt 适配上统一 RISC-V64 和 LoongArch64 的执行路径。

目录

摘要	ii
第一章 设计问题与项目背景	1
1.1 我们要做一个什么样的内核	1
1.2 为什么选择Rust 宏内核	1
1.3 源码概览	2
第二章 基于StarryOS 的技术继承路线	4
2.1 源代码继承关系	4
2.2 继承面与改动面	4
2.3 关键差异落点	4
2.3.1 系统调用层	4
2.3.2 任务模型	6
2.3.3 地址空间与内存管理	6
2.3.4 文件系统与fd 模型	7
2.3.5 网络栈	8
第三章 系统调用层：从寄存器到内核对象的翻译器	9
3.1 问题：两个架构的syscall 如何统一	9
3.2 Dispatcher 的结构：分发与横切状态	9
3.2.1 fast path：让不阻塞的syscall 直接返回	9
3.2.2 restart 分类：阻塞型syscall 被信号打断怎么办	10
3.2.3 RestartTracker：信号与syscall 的完整状态机	11
3.2.4 用户指针安全读写的集中化	12
3.3 双架构ABI 归一化	12
3.3.1 syscall number 差异	12
3.3.2 time64 alias	12
3.3.3 RISC-V 特有接口	13
3.4 实现状态分类	13
3.5 与StarryOS 的关键差异	13

第四章 进程与线程：Linux任务模型的对象化表达	15
4.1 问题：一个结构体装不下Linux的进程/线程语义	15
4.1.1 与StarryOS baseline 的比较	16
4.2 进程共享状态容器	16
4.2.1 身份与可执行映像	17
4.2.2 地址空间与资源范围	17
4.2.3 权限与安全	17
4.2.4 UID/GID 变更的capability 修正协议	18
4.2.5 信号与同步原语	19
4.2.6 进程生命周期事件（PollSet 体系）	19
4.2.7 控制面：job control、ptrace、exec、vfork	20
4.2.8 Namespace 体系	22
4.2.9 调度与资源上下文	23
4.2.10 POSIX Timers	24
4.2.11 CPU 时间统计	24
4.2.12 ProcessData::new：建立进程共享对象	25
4.3 Thread：线程局部状态的承载者	25
4.3.1 Thread 的线程局部边界	25
4.3.2 与调度器的集成	26
4.4 全局任务表与操作集合	26
4.4.1 四张全局弱引用表	27
4.4.2 孤儿进程回收协议	27
4.4.3 Robust Futex 退出处理	27
4.4.4 跨地址空间VM 写入	28
4.5 clone/fork/vfork：按flags 选择共享策略	29
4.5.1 CloneFlags：28 个flags 编码	29
4.5.2 CloneArgs：ABI 归一化层	29
4.5.3 Flags 验证：拒绝无意义的组合	30
4.5.4 NPROC 限制实施	30
4.5.5 do_clone：核心创建协议	31
4.5.6 clone flags 对资源的影响	32
4.5.7 调度状态继承	33
4.5.8 vfork：阻塞父线程的机制	33
4.6 execve：替换映像但保留进程	33

4.6.1	支持和约束	34
4.6.2	execve 处理顺序	34
4.6.3	execve 的参数加载	35
4.7	exit、zombie 与 wait: 正确的进程死亡	35
4.7.1	do_exit: 完全退出路径	35
4.7.2	sys_exit 与 sys_exit_group	36
4.7.3	wait: PID 选择器与事件消费	36
4.8	当前边界	37
第五章	地址空间与内存管理: 虚拟内存的后端抽象	41
5.1	问题: 不同映射需要不同填页方式	41
5.2	AddrSpace: 地址空间的状态边界	41
5.2.1	AddrSpace 的状态分层	42
5.2.2	区间集合的复用逻辑	42
5.2.3	栈向下增长状态	43
5.3	Page Fault 与 fork 的主路径	43
5.3.1	Page Fault 处理	43
5.3.2	fork 时的地址空间克隆	43
5.3.3	栈自动增长	44
5.4	Backend 抽象: 把填页方式编码为 trait	44
5.4.1	BackendOps 统一接口	44
5.4.2	四类 Backend 的语义边界	46
5.5	COW: 私有映射的核心后端	46
5.5.1	CowBackend 状态	46
5.5.2	全局物理页引用计数	47
5.5.3	fork 时的 COW 克隆	47
5.5.4	写 fault 的分裂决策	47
5.5.5	文件私有映射的首次填页	48
5.5.6	EOF、驻留与 fault-around	48
5.6	FileBackend: 共享文件映射与页缓存生命周期	48
5.6.1	文件映射状态	48
5.6.2	页缓存驱逐回调	49
5.6.3	populate: 页缓存与页表协调	49
5.6.4	文件 futex 与动态 EOF	50

5.7	SharedBackend: 共享匿名页池	50
5.7.1	SharedPages 页池	50
5.7.2	SharedBackend 状态与权限	50
5.7.3	SharedBackend 填页	50
5.8	LinearBackend: 物理地址直映射	50
5.9	mmap、munmap 与 mprotect	51
5.9.1	mmap 的参数校验与地址选择	51
5.9.2	mmap 的Backend 选择	51
5.9.3	munmap 与 mprotect	52
5.10	mremap: 旧地址段迁移	52
5.11	brk: 堆的扩展与收缩	52
5.12	madvise: 细粒度内存提示	53
5.13	mlock 与 mlockall: 内存锁定层级	53
5.14	ELF Loader: 从二进制到进程地址空间	54
5.14.1	ELF 缓存与段映射	54
5.14.2	动态链接器、栈与堆	54
5.14.3	Shebang 与脚本回退	54
5.15	用户内存访问: 安全的跨特权级读写	55
5.15.1	用户指针范围校验	55
5.15.2	用户拷贝中的page fault	55
5.15.3	用户拷贝分块	55
5.16	与StarryOS baseline 的设计比较	55
第六章	文件系统: 一切皆是文件对象	59
6.1	问题: 整数fd 无法承载完整文件模型	59
6.2	三层模型: 对象、打开描述、槽位	59
6.2.1	FileLike: 所有fd 对象的共同接口	59
6.2.2	FileDescription: 共享的打开文件状态	60
6.2.3	FileDescriptor: fd table 槽位	61
6.2.4	fd table 的分配、dup 与fcntl	61
6.2.5	三层模型与StarryOS baseline	62
6.3	统一接口下的特殊fd 对象	62
6.3.1	epoll: fd 整数和打开描述共同作为key	62
6.3.2	eventfd: 计数器就是文件状态	62

6.3.3	timerfd 与 signalfd: 内核事件转为可读字节流	62
6.3.4	pidfd 与 memfd: 对象身份和匿名文件	63
6.3.5	inotify、fanotify、userfaultfd 与异步环接口	63
6.4	普通文件I/O: 统一fd 入口后的对象分派	63
6.5	pseudofs: 内核状态到格式化文件	64
6.5.1	SimpleFs 与 SimpleFsNode	64
6.5.2	SimpleFile 的生成式读写	64
6.6	mount ledger: 挂载状态的真相源	64
6.7	工程边界	64
第七章	网络栈: axnet-ng的协议无关Socket 模型	67
7.1	多协议共享的socket ABI	67
7.2	统一socket 抽象	67
7.2.1	共同操作边界	67
7.2.2	接入fd 层与BPF filter	69
7.3	NetStack: 网络namespace 的状态容器	69
7.4	TCP: 状态事务、握手和被动打开	70
7.4.1	状态提交规则	70
7.4.2	主动连接	71
7.4.3	listen、accept 与SYN 队列	71
7.4.4	收发、poll 与关闭	71
7.5	UDP: 无连接数据报与双地址视图	72
7.6	Unix domain socket: 地址服务发现与本地传输	72
7.7	设备轮询、路由与通用阻塞语义	73
7.8	syscall 到数据包的路径	73
7.9	与StarryOS baseline 的设计差异	74
7.10	当前边界	74
第八章	信号、同步与时间: 阻塞路径上的共同控制面	78
8.1	阻塞线程的三个入口	78
8.2	信号: 两级pending 队列和投递点	78
8.2.1	队列所有权	78
8.2.2	发送与权限	79
8.2.3	返回用户态前的投递	79
8.2.4	等待类系统调用	79

8.3	futex: 地址key、等待队列与重启	80
8.3.1	地址到等待队列的映射	80
8.3.2	等待者生命周期	80
8.3.3	wake、requeue 和系统调用入口	81
8.4	时间: clockid、alarm 和文件描述符	82
8.4.1	clockid 和time namespace	82
8.4.2	CPU 时间、itimer 和alarm 队列	82
8.4.3	POSIX timer	82
8.4.4	timerfd	83
8.5	三类交叉场景	84
8.5.1	futex wait 被信号打断	84
8.5.2	POSIX timer 唤醒sigsuspend	84
8.5.3	timerfd 和epoll	84
8.6	与StarryOS baseline 的设计差异	84
8.7	当前边界	85
第九章	跨架构差异处理: RISC-V与LoongArch	88
9.1	同一套Linux ABI 落到两种trap 机器上	88
9.2	syscall 编号和重启边界	88
9.2.1	指令长度与restart	88
9.2.2	LoongArch 的time64 alias	89
9.3	用户态上下文和trap 返回	89
9.3.1	RISC-V 与LoongArch 的上下文差异	89
9.3.2	kernel 用户态主循环	90
9.4	页表patch 层	90
9.5	构建产物和运行入口	90
9.6	当前边界	91
第十章	AI工具使用与工程责任边界	92
10.1	使用的工具	92
10.2	任务范围	92
10.3	人机分工	93
10.4	错误发现与纠正	93
10.5	验证和追溯	94
10.6	团队掌握程度	94

第十一章 边界、局限与未来工作	96
11.1 系统调用覆盖边界	96
11.2 任务与调度边界	96
11.3 内存管理边界	97
11.4 文件与网络边界	98
11.5 并发与可靠性	98
11.6 未来工作	99
第十二章 QEMU单核环境下的热路径开销	100
12.1 评测压力	100
12.2 缺页路径	100
12.3 地址空间查找与TLB 维护	101
12.4 syscall 与时间读取	102
12.5 条件关闭的慢路径	103
12.6 文件和网络数据通道	103
12.7 当前边界	104
第十三章 LTP回归中的兼容性修复	105
13.1 LTP 回归范围	105
13.2 按子系统的LTP 问题分类	105
13.2.1 信号处理	105
13.2.2 futex 与同步	105
13.2.3 文件操作	106
13.2.4 进程生命周期	106
13.2.5 内存管理	107
13.2.6 网络	108
13.2.7 IPC	108
13.3 关键修复节点	108
13.4 LTP 回归流程	108
第十四章 核心代码路径	110
14.1 handle_syscall: 入口、快路径与restart 决策	110
14.2 do_clone: 子任务公开前的事务	110
14.3 do_execve: 新映像提交边界	112
14.4 COW fork 与写fault	112

14.5 Pipe: 有界字节流与端点语义	113
14.6 futex_wait: 注册等待与取消	114
14.7 RestartTracker: 嵌套 signal handler 场景	114
第十五章 总结与展望	115
15.1 工作总结	115
15.2 主要设计取舍	115
15.3 回归与性能结果	116
15.4 局限与未来工作	116
参考文献	117
附录A 系统调用覆盖矩阵	119
A.1 文件系统 family (fs)	119
A.2 任务 family (task)	120
A.3 内存管理 family (mm)	121
A.4 网络 family (net)	122
A.5 IO 多路复用 family (io_mpx)	123
A.6 信号 family (signal)	123
A.7 同步 family (sync)	123
A.8 时间 family (time)	124
A.9 IPC family (ipc)	124
A.10 资源控制	125
A.11 其他	125
附录B 关键源码索引	127
B.1 kernel crate	127
B.1.1 kernel/src/ —入口和顶层	127
B.1.2 kernel/src/syscall/ —系统调用层 (约26,000 行)	127
B.1.3 kernel/src/task/ —任务模型 (约6,600 行)	129
B.1.4 kernel/src/mm/ —内存管理 (约3,700 行)	130
B.1.5 kernel/src/file/ —文件系统 (约9,500 行)	130
B.1.6 kernel/src/pseudofs/ —伪文件系统 (约8,000 行)	131
B.1.7 kernel/src/bpf/ —BPF 子系统	131
B.2 crates/axnet-ng/ —网络栈 (约6,000 行)	132
B.3 third_party/rust-patches/ —第三方补丁	133

第一章 设计问题与项目背景

1.1 我们要做一个什么样的内核

TheKernel 的目标是为Linux 用户态程序提供一个语义兼容的运行环境。用户程序依赖的是一套可观察语义：进程创建与替换、地址空间映射、文件描述符、socket I/O、多路复用、用户态同步与信号处理。在Linux 上围绕这些接口编写的程序，在TheKernel 上也应能运行并产生一致的行为。

TheKernel 面向真实的libc、busybox、LTP 测试套件、iozone benchmark 与现代多线程程序。这些程序依赖clone flags 的组合语义、信号与阻塞系统调用的交叉、缺页时的COW 正确性、fcntl lock 的owner 语义、以及epoll 的边沿/水平触发行为；少量syscall stub 无法支撑这类用户态环境。

Linux 语义把多个子系统绑在同一条执行路径上。一次clone 同时决定地址空间、fd table、signal state、thread group、TLS 与调度状态是共享还是复制；一次execve 同时替换地址空间、处理close-on-exec、重置信号处置、释放vfork parent；一次mmap 则同时牵动缺页处理、fork COW、msync 与OOM 计费。这种跨子系统的耦合是Linux 语义的本质特征，也是TheKernel 架构必须正面处理的核心问题，如图 1.1 所示，内核的各子系统在同一地址空间内围绕这一耦合关系协同组织。

1.2 为什么选择Rust 宏内核

Rust的所有权模型帮助我们在编译期排除use-after-free、double-free 和数据竞争。但Rust 的类型系统本身并不解决内核设计问题，它只提供工具。真正驱动架构选择的是要解决的问题本身：

1. 宏内核意味着所有Linux 可见的对象（进程、线程、地址空间、fd、socket、IPC）都在同一内核地址空间内完成管理，跨子系统交互通过共享内核对象直接完成。这一选择简化了execve 这类需要同时触碰task、mm、fs、signal 四类状态的路径，但也将状态所有权的管理复杂度集中到内核自身。
2. 复用已有生态意味着不必从零实现一个OS：TheKernel 继承ArceOS 的底层runtime（axtask、axhal、axalloc、axdriver）与StarryOS 的进程及信号模型，改动集中在Linux 兼容层的对象组织。

1.3 源码概览

TheKernel的内核crate 约6 万行Rust 代码，按子系统组织如下。下表给出各子系统的路径、职责与关键入口，作为后续章节的源码索引；详细的路径与函数级引用在对应子系统章节中展开。

路径	职责	关键入口
kernel/src/entry.rs	内核启动主线：挂载pseudofs、加载init、创建第一个用户进程	init()
kernel/src/syscall/	系统调用兼容层：dispatcher、12 个 syscall family（如process、mm、fs、signal、net 等）及架构特化分支	handle_syscall()
kernel/src/task/	进程/线程状态模型、signal、futex、timer、restart、生命周期	ProcessData, Thread
kernel/src/mm/	地址空间、backend 抽象、ELF loader、user access	AddrSpace, Backend
kernel/src/file/	FileLike、FileDescription、fd table、special fd 对象族	FileLike, FileDescription
kernel/src/pseudofs/	/proc、/sys、/dev、/tmp、cgroup 的伪文件系统表面	SimpleFs
kernel/src/mounts.rs	mount ledger 与路径穿越	MountRecord
crates/axnet-ng/	candidate-local 网络栈	SocketOps, Socket
third_party/ rust-patches/	patched ArceOS/StarryOS 底层 crate	axtask, axsched, memory_set, page_table_multiarch

层级	TheKernel 组件	作用
用户与架构入口	User Space; RISC-V64 / LoongArch64 trap entry	用户程序通过架构syscall 指令进入内核; trap 层保存寄存器现场, 形成统一的UserContext。
系统调用分发	kernel/src/syscall/mod.rs	分发器处理syscall 编号、架构alias、getter 与时间读取fast path、restart 分类和family dispatch。
Linux 对象层	Task / Memory / File-VFS / Network	ProcessData/Thread、AddrSpace/Backend、FileLike/FileDescription 和axnet-ng socket 承载Linux 可见状态。
兼容补强层	Signal / Futex / ELF loader / Pseudofs / Poll	这些模块补齐进程信号、阻塞唤醒、ELF 装载、procfs-like 视图、mount ledger 与poll/epoll 等Linux 兼容行为。
基础运行层	Patched ArceOS / StarryOS substrate	axtask、axhal、axfs-ng、page_table_multiarch 和virtio 驱动提供调度、页表、文件系统与设备环境。
平台	QEMU virt	RISC-V64 使用virtio-mmio; LoongArch64 使用virtio-pci。两条架构路径在syscall 分发层汇合。

图 1.1 TheKernel 内核系统全景架构

第二章 基于StarryOS的技术继承路线

2.1 源代码继承关系

StarryOS baseline固定在upstream commit 2e075accf4fb。TheKernel 候选版本使用 commit d871e3a，它由该baseline 演进而来。两个commit 构成可追溯的源代码分叉关系，比较对象限定为这两个具体版本。

仓库内对比脚本按固定路径组扫描两边源码。在kernel/src/ 下，StarryOS baseline 有104 个文件；TheKernel 有161 个文件，其中57 个为TheKernel 新增文件，104 个同名文件中有93 个发生内容变化。把crates/、third_party/rust-patches/、scripts/ 等路径纳入口径后，TheKernel 的候选树还包含本地crates/axnet-ng、BPF 子系统、mount ledger、OSComp 支撑脚本，以及多组本地patch crate。数量本身只说明阅读范围；真正的技术差异集中在几个重塑Linux ABI 语义的子系统上。

上述路径数量来自仓库内的StarryOS 对比脚本，输入为固定在2e075accf4fb 的upstream checkout。

2.2 继承面与改动面

TheKernel相对StarryOS 的变化落在三类层面：

1. **继承层**。ArceOS runtime crate、基础task 扩展点、signal manager、页表与VFS 框架仍然是底座。TheKernel 沿着这些抽象已有的生命周期和锁边界继续扩展。
2. **强化层**。同名或同类路径继续存在，但状态容器、错误语义和跨子系统关系已经被改写。系统调用分发、任务模型、地址空间、fd模型和procfs 都属于这一类；这些改动把Linux 可观察状态提升为内核对象的一部分，范围已经超过syscall 数量补齐。
3. **新增层**。StarryOS baseline没有对应本地实现的模块由TheKernel 接管，例如本地crates/axnet-ng、kernel/src/bpf/、kernel/src/mounts.rs，以及围绕OSComp 评测环境准备的脚本和第三方patch。

关键差异落点给出各层面的源码入口和设计差异。

2.3 关键差异落点

2.3.1 系统调用层

StarryOS baseline的syscall dispatcher 在解析Sysno 后直接进入一个大match，每个分支负责自己的参数解释和返回值转换。TheKernel 保留了这种集中分发表，但把syscall restart

的分类提前到dispatcher入口；随后getter与部分时间读取可以直接返回，普通阻塞调用再进入完整的timer accounting、restart bookkeeping与分发表。

代码2.1 dispatcher前置的restart class判定

```

72 fn restart_class_for_syscall(sysno: Sysno, uctx: &UserContext) -> Option<RestartClass> {
73     match sysno {
74         Sysno::ioctl
75         | Sysno::openat
76         | Sysno::openat2
77         | Sysno::wait4
78         | Sysno::waitid
79         | Sysno::flock => Some(RestartClass::Sys),
80         #[cfg(target_arch = "x86_64")]
81         Sysno::open => Some(RestartClass::Sys),
82         Sysno::read | Sysno::readv => {
83             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Read)
84         }
85         Sysno::readahead => Some(RestartClass::Sys),
86         Sysno::write | Sysno::writev => {
87             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Write)
88         }
89         Sysno::accept | Sysno::accept4 | Sysno::recvfrom | Sysno::recvmsg | Sysno::recvmsg => {
90             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Read)
91         }
92         #[cfg(target_arch = "loongarch64")]
93         Sysno::recvmsg_time64 => {
94             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Read)
95         }
96         Sysno::connect | Sysno::sendto | Sysno::sendmsg | Sysno::sendmsg => {
97             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Write)
98         }
99         Sysno::futex => restart_class_for_futex(uctx),
100         #[cfg(target_arch = "loongarch64")]
101         Sysno::futex_time64 => restart_class_for_futex(uctx),
102         Sysno::futex_waitv => Some(RestartClass::Sys),
103         _ => None,
104     }

```

这段代码把三个原本容易散落在syscall分支里的问题集中到dispatcher层处理。

1. 信号中断后的重启策略由restart_class_for_syscall统一推导。普通fd读写、accept/recv/send、futex wait等路径在这里被标记为可重启；带socket timeout的I/O会退出默认重启路径，避免timeout语义被dispatcher重放覆盖。
2. **getter fast path**在handle_syscall入口处理getpid、gettid、getuid等只读当前线程状态的调用。这类调用不拷贝用户内存、不阻塞、也不需要syscall restart，因此可以跳过timer accounting和restart bookkeeping。
3. 架构别名在同一个dispatcher中映射到共享实现。LoongArch的clock_gettime64、futex_time64、recvmsg_time64、utimensat_time64等编号不再依赖各子模块重复处理。

对照代码时，StarryOS baseline 的直接分发表见/tmp/starryos-upstream/kernel/src/syscall/mod.rs:22-31；TheKernel 的getter fast path、time fast path 与主分发表从kernel/src/syscall/mod.rs:120-186 展开。

2.3.2 任务模型

StarryOS已经区分Thread与ProcessData：前者保存线程局部状态，后者保存进程共享状态。TheKernel的关键变化体现在ProcessData 职责扩展上：credential、capability、namespace、job control、exec/vfork 协调、membarrier 与prctl 可见状态都挂在同一个进程共享对象上。

```

1 pub struct ProcessData {
2     pub proc: Arc<Process>,
3     aspace_handle: RwLock<Arc<Mutex<AddrSpace>>>,
4     pub rlim: RwLock<Rlimits>,
5     pub exec_event: Arc<PollSet>,
6
7     creds: SpinNoIrq<Credentials>,
8     caps: SpinNoIrq<CapabilityState>,
9     supplementary_groups: SpinNoIrq<Vec<u32>>,
10    no_new_privs: AtomicU32,
11    seccomp_mode: AtomicU32,
12    membarrier_state: AtomicU32,
13
14    job_ctl: SpinNoIrq<JobControlState>,
15    ptrace_ctl: SpinNoIrq<PtraceControlState>,
16    exec_ctl: SpinNoIrq<ExecControlState>,
17    vfork_ctl: SpinNoIrq<VforkControlState>,
18
19    pub net_ns: Arc<NetStack>,
20    cgroup_ns: Arc<CgroupNamespace>,
21    pid_ns: Arc<PidNamespace>,
22    user_ns: Arc<UserNamespace>,
23    uts_ns: RwLock<Arc<UtsNamespace>>,
24    time_ns: RwLock<Arc<TimeNamespace>>,
25 }

```

这组字段给出了Linux 进程模型的边界划分：同一线程组共享的安全身份、namespace、wait/exec/vfork 协调和procfs 可见状态放在ProcessData；线程退出、robust futex、clear-child-tid、restart tracker 和proc state hint 仍由Thread 保存。这样，clone 只需要根据flag 决定哪些共享对象继承、哪些对象新建，后续syscall 和procfs 查询也都读取同一份状态。

StarryOS baseline 的Thread 与ProcessData 可在/tmp/starryos-upstream/kernel/src/task/mod.rs:47-221 对照；TheKernel 将对应定义拆到kernel/src/task/thread.rs:37-88 与kernel/src/task/process.rs:471-585。

2.3.3 地址空间与内存管理

TheKernel沿用StarryOS的AddrSpace+MemorySet<Backend>+PageTable 基本骨架，但把Linux VMA 行为显式拆成独立状态。这样一来，mmap、madvise、fork、page fault 处理

看到的不只是页表映射，还能看到growdown、dontfork、wipeonfork 与mlock 语义。

代码2.2 TheKernel AddrSpace 的Linux VMA 语义字段

```

31  /// The virtual memory address space.
32  pub struct AddrSpace {
33      va_range: VirtAddrRange,
34      areas: MemorySet<Backend>,
35      growdown_starts: BTreeSet<VirtAddr>,
36      wipe_on_fork_ranges: BTreeMap<VirtAddr, VirtAddr>,
37      dontfork_ranges: BTreeMap<VirtAddr, VirtAddr>,
38      locked_ranges: BTreeMap<VirtAddr, VirtAddr>,
39      lock_future_mappings: bool,
40      lock_future_on_fault: bool,
41      pt: PageTable,
42  }

```

这一层增量可以精确分成三部分：

1. **缺页填充后端。**TheKernel通过BackendOps::populate 让COW、shared anonymous、shared file、linear mapping 各自处理缺页填充，page fault 入口只面向backend 抽象。
2. **COW 帧引用计数。**COW backend 通过全局frame table 记录共享物理页引用数；写 fault 时，单引用页只提升权限，多引用页分配新frame 并复制内容。
3. **Linux VMA 附加语义。**growdown_starts、dontfork_ranges、wipe_on_fork_ranges、locked_ranges 等集合把fork、栈扩展、内存锁定和wipe-on-fork 行为从单个flags 位中拆出来，避免后续路径靠猜测解释VMA。

BackendOps 的populate/clone_map 接口见kernel/src/mm/aspace/backend/mod.rs:73-123；COW 写fault 分支见kernel/src/mm/aspace/backend/cow.rs:174-213。

2.3.4 文件系统与fd 模型

StarryOS baseline的fd table 槽位直接持有Arc<dynFileLike> 和cloexec。TheKernel 在FileLike 与整数fd 之间增加FileDescription，把”打开文件描述”从”fd 槽位”中分离出来。

```

1  pub struct FileDescription {
2      pub inner: Arc<dyn FileLike>,
3      open_credentials: OpenCredentials,
4      flock_owner: u64,
5      status_flags: AtomicU32,
6      write_open_key: Option<ExecutableKey>,
7      async_io: Mutex<AsyncIoState>,
8  }
9
10 pub struct FileDescriptor {
11     pub description: Arc<FileDescription>,
12     pub cloexec: bool,
13 }

```

这个分层解释了文件系统模型中的几个Linux 语义：dup 和fork 后的fd 槽位可以不同，但共享同一个open file description；O_APPEND、O_NONBLOCK、OFD lock owner、lease、异步I/O owner 与打开时凭证都绑定在description 上；而FD_CLOEXEC 仍然属于单个fd 槽位。TheKernel 的special fd 也因此能以统一方式接入poll/epoll/select，减少各syscall 入口的临时分支。

StarryOS baseline 的FileDescriptor 见/tmp/starryos-upstream/kernel/src/file/mod.rs:182-210；TheKernel 的FileDescription 与FileDescriptor 见kernel/src/file/desc.rs:92-99 与kernel/src/file/desc.rs:266-270。

2.3.5 网络栈

StarryOS baseline通过workspace 依赖使用发布版axnet-ng，并在kernel/src/file/net.rs 中提供基础socket wrapper。TheKernel 的增量在于把axnet-ng 本地化到仓库内，并把网络状态变成可挂到进程namespace 的内核对象。

代码2.3 per-namespace NetStack 状态容器

```

20  /// A self-contained network stack instance.
21  ///
22  /// Each `NetStack` holds its own socket set, listen table, service (interface +
23  /// router), and ephemeral port counters. Multiple `NetStack` instances can
24  /// coexist for network namespace isolation.
25  ///
26  /// **Drop ordering:** `listen_table` is declared before `socket_set` so that
27  /// listen-table entries (which hold `Weak<SocketSetWrapper>`) are cleaned up
28  /// while the socket set is still alive.
29  pub struct NetStack {
30      pub(crate) listen_table: Arc<ListenTable>,
31      pub(crate) socket_set: Arc<SocketSetWrapper<'static'>>,
32      pub(crate) service: Mutex<Service>,
33      pub(crate) tcp_ephemeral_port: Mutex<u16>,
34      pub(crate) udp_ephemeral_port: Mutex<u16>,
35      ipv4_conf_default_tag: AtomicI32,
36      ipv4_conf_lo_tag: AtomicI32,
37  }
```

NetStack 是网络namespace 的状态容器。它持有listen table、socket set、service/router 和临时端口分配器；ProcessData 再通过前文展示的net_ns 字段引用它。这样，CLONE_NEWNET 可以创建只含loopback 的新stack，普通clone 则共享父进程的stack。

内核fd 层仍把socket 包装为FileLike，但TheKernel 的wrapper 还承接BPF socket filter、TCP ULP 兼容状态和更完整的socket option 处理。

两个根Cargo.toml 给出的依赖边界也与这个结论一致：StarryOS baseline 第34 行使用crates.io 上的axnet-ng0.3.0-preview.2；TheKernel 第34 行把axnet 指向仓库内的crates/axnet-ng，并通过[patch.crates-io] 覆盖传递依赖。这里的差异集中在依赖和维护边界：socket wrapper 在baseline 已经存在，TheKernel 让网络栈实现随内核源码一起演进。

第三章 系统调用层：从寄存器到内核对象的翻译器

3.1 问题：两个架构的syscall 如何统一

Linux用户程序通过系统调用进入内核。对于宏内核而言，系统调用层是用户态与内核对象模型之间的第一层翻译器。其输入是架构寄存器中的一组整数（系统调用号与最多6个参数），输出则是对正确内核对象的操作。这一层既要完成ABI 意义上的参数搬运与合法性检查，又要在内核内部把抽象的整数请求落到具体的对象操作上。

TheKernel 同时面向RISC-V64 与 LoongArch64 两个架构。两者在系统调用号命名、time64 相关接口的编号、部分trap frame 布局以及参数寄存器约定上存在差异。如果任由每个系统调用文件自行处理这些架构差异，必然衍生大量重复代码与不一致的行为。TheKernel 的策略是：在dispatcher 中统一完成ABI 归一化，使具体的系统调用实现只面对统一的参数语义，从而把架构相关性收敛到一处。

3.2 Dispatcher 的结构：分发与横切状态

TheKernel 的系统调用dispatcher 以handle_syscall 为入口。它与StarryOS baseline 直接用match 分发的做法有本质区别：在进入具体系统调用实现之前与之后，dispatcher 注入了多层bookkeeping，使得分发过程同时承担了重启语义、计时统计与快路径优化等横切职责。整体分发流程如图3.1 所示。

3.2.1 fast path：让不阻塞的syscall 直接返回

Linux用户态会大量调用getter 型系统调用（如getpid、gettid、getuid、getegid）。这些调用不阻塞、不访问用户内存、通常不失败、不需要restart，因此完整走一遍入口与出口流程纯属浪费。TheKernel 的dispatcher 将这类调用识别为快路径，跳过enter_syscall、timer accounting 与restart bookkeeping，直接返回结果，从而把短周期高频调用的开销压缩到最低。

代码3.1 fast path getter

```
120 fn fast_path_getter(sysno: Sysno) -> Option<AxResult<isize>> {
121     Some(match sysno {
122         Sysno::getpid => sys_getpid(),
123         Sysno::getppid => sys_getppid(),
124         Sysno::gettid => sys_gettid(),
125         Sysno::getuid => sys_getuid(),
```

```

126     Sysno::geteuid => sys_geteuid(),
127     Sysno::getgid => sys_getgid(),
128     Sysno::getegid => sys_getegid(),
129     _ => return None,
130 }
131 }

```

`clock_gettime` 的快路径化需区分时钟类型：`CPUTIME` 类时钟需要精确的CPU 时间统计，不能走快路径；`REALTIME` 与 `MONOTONIC` 等时钟则可直接返回¹。这一区分避免了每次时钟读取都经过完整bookkeeping 路径，也正好落在 `cyclictest` 这类延迟测试敏感的调用路径上。

3.2.2 restart 分类：阻塞型syscall 被信号打断怎么办

这是系统调用层与信号层交叉处最复杂的部分。当一个线程阻塞在 `read`、`poll`、`futex`、`nanosleep` 时收到信号，内核不能简单地返回 `EINTR`。正确的行为取决于系统调用类型、信号 handler 的 `SA_RESTART` 标志，以及系统调用是否已产生副作用（例如 `read` 已经复制了部分数据）。

代码3.2 restart class 推导

```

72 fn restart_class_for_syscall(sysno: Sysno, uctx: &UserContext) -> Option<RestartClass> {
73     match sysno {
74         Sysno::ioctl
75         | Sysno::openat
76         | Sysno::openat2
77         | Sysno::wait4
78         | Sysno::waitid
79         | Sysno::flock => Some(RestartClass::Sys),
80         #[cfg(target_arch = "x86_64")]
81         Sysno::open => Some(RestartClass::Sys),
82         Sysno::read | Sysno::readv => {
83             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Read)
84         }
85         Sysno::readahead => Some(RestartClass::Sys),
86         Sysno::write | Sysno::writev => {
87             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Write)
88         }
89         Sysno::accept | Sysno::accept4 | Sysno::recvfrom | Sysno::recvmsg | Sysno::recvmmsg => {
90             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Read)
91         }
92         #[cfg(target_arch = "loongarch64")]
93         Sysno::recvmmsg_time64 => {
94             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Read)
95         }
96         Sysno::connect | Sysno::sendto | Sysno::sendmsg | Sysno::sendmmsg => {
97             restart_class_for_fd_io(uctx.arg0() as i32, SocketIoDirection::Write)
98         }
99         Sysno::futex => restart_class_for_futex(uctx),
100         #[cfg(target_arch = "loongarch64")]
101         Sysno::futex_time64 => restart_class_for_futex(uctx),
102         Sysno::futex_waitv => Some(RestartClass::Sys),

```

¹实现见 `kernel/src/syscall/mod.rs:153-177`。

```

103     _ => None,
104 }
105 }

```

为了把这一决策规则显式化，dispatcher 将每个系统调用归入三种restart class 之一²：

1. `RestartClass::Sys` 可被SA_RESTART handler 自动重启，默认行为是handler 返回后重放原始系统调用。
2. `RestartClass::NoIntr` 不会被EINTR 打断；无论是否被信号中断，都自动重试，用于某些特殊的poll 场景。
3. `RestartClass::NoHand` 即使在SA_RESTART handler 中也不重启，只返回EINTR。

socket I/O 的restart class 还取决于socket 是否配置了SO_RCVTIMEO 与SO_SNDTIMEO³：带有timeout 配置的socket I/O 交由timeout 机制处理，避免dispatcher 重启路径覆盖套接字自己的超时语义。

3.2.3 RestartTracker：信号与syscall 的完整状态机

单个restart class 只能描述一次中断的静态策略；真正棘手的是signal handler 嵌套时多套restart 状态的并存。为此TheKernel 在每个线程上挂载RestartTracker，把当前syscall、armed restart block、嵌套状态栈和handler 深度放在同一个状态机里。

代码3.3 RestartTracker 的核心状态

```

72 #[derive(Debug, Default)]
73 pub(crate) struct RestartTracker {
74     signal_handler_depth: usize,
75     current_restart: Option<PendingRestart>,
76     armed_restart_block: Option<RestartBlock>,
77     restart_states: Vec<RestartState>,
78     resume_restored_context: bool,
79 }

```

`current_restart` 记录当前可重启syscall 的编号、参数和返回地址；`armed_restart_block` 保存需要restart_syscall 接续的阻塞状态，例如futex wait 的剩余超时时间；`restart_states` 是按signal handler 嵌套深度维护的栈；`signal_handler_depth` 则决定内层handler 执行syscall 时是否保留外层restart 状态。

该设计的核心不变量是：内层signal handler 中执行的任何系统调用都不得破坏外层阻塞系统调用的restart 状态。围绕这一不变量，restart.rs 的单元测试覆盖了多层嵌套场景⁴。

完整的状态迁移路径如下：

²分类定义见kernel/src/task/restart.rs:12-16。

³推导逻辑见kernel/src/syscall/mod.rs:42-57。

⁴嵌套测试见kernel/src/task/restart.rs:362-521。

1. **syscall 进入**：`enter_syscall()` 捕获当前系统调用信息、设置restart class、清理过期的armed restart block。
2. **syscall 被信号打断**：返回 `AxError::Interrupted`。`dispatcher` 的 `maybe_request_syscall_restart()` 调用 `request_syscall_restart()`，将当前系统调用推入 `restart_states` 栈。
3. **信号handler 执行**：`finish_signal_delivery()` 根据信号disposition 与 handler 的 `SA_RESTART` 标志，决定每个pending restart state 的决策（Restart 或 EINTR）。
4. **handler 返回rt_sigreturn**：`complete_sigreturn()` 消费最顶层的restart state。若决策为Restart，则通过修改trap frame 的PC/SP/系统调用寄存器来重放被中断的系统调用。
5. **非handler 信号 (stop/continue)**：`finish_signal_resume()` 在task 被SIGCONT 恢复后重放被中断的系统调用。

3.2.4 用户指针安全读写的集中化

系统调用频繁读写用户态内存：路径字符串、结构体、iovec数组、signal set、socket address 等不一而足。这些操作面临两个共同风险：其一，用户传递的指针可能是NULL、未映射或不可读写；其二，即使指针起始有效，跨越页边界时也可能触发page fault。

TheKernel 将用户内存访问集中到 `kernel/src/mm/access.rs` 的 helper 中（涵盖user copy in/out、字符串读取与iovec 解析），而具体的结构体验证与字段解析则在各系统调用文件中完成。这种层次化的错误处理使每个系统调用不必裸写用户地址逻辑，也使非法指针到EFAULT 的映射保持一致。

3.3 双架构ABI 归一化

TheKernel 在以下三个层面处理RISC-V 与LoongArch 的差异。

3.3.1 syscall number 差异

在 `dispatcher` 的 `match` 语句中，若干x86-64 遗留接口只在对应架构下编译，例如 `open`、`link`、`unlink`、`chown` 与 `chmod`。RISC-V 与LoongArch 暴露的是at 版本入口：`openat` 以 `AT_FDCWD` 为目录，再依次传入路径、`flags` 与 `mode`。

3.3.2 time64 alias

LoongArch64 是纯64 位架构，但历史上Linux 的部分time 相关系统调用同时存在32 位 `time_t` 版本与64 位 `time64` 版本。TheKernel 在 `dispatcher` 中将LoongArch 的 `time64 alias`（如 `clock_gettime64`、`futex_time64`、`recvmsg_time64`、`utimensat_time64`）映射到与64 位版本相同的实现，使上层无需感知版本差异：

代码3.4 LoongArch clock_gettime64 与clock_gettime 共享实现

```
167     #[cfg(target_arch = "loongarch64")]
168     Sysno::clock_gettime64 => {
169         let clockid = uctx.arg0() as u32;
170         if !matches!(clockid, CLOCK_PROCESS_CPUTIME_ID | CLOCK_THREAD_CPUTIME_ID) {
171             let r = sys_clock_gettime(uctx.arg0() as _, uctx.arg1() as _);
172             uctx.set_retval(r.unwrap_or_else(|err| -LinuxError::from(err).code() as _ as _));
173             return;
174         }
175     }
176     _ => {}
```

3.3.3 RISC-V 特有接口

RISC-V平台新增sys_riscv_hwprobe⁵，为libc 与运行时提供硬件能力探测的兼容接口；同时保留riscv_flush_icache。这两个接口在LoongArch 上不存在，并通过cfg 在编译期排除。

3.4 实现状态分类

TheKernel 的系统调用实现按用户态可观察行为分为四档：

状态	判定标准	用户态含义
implemented	对象模型完整、参数校验、状态更新、错误路径、用户copy 均已实现	主路径按照Linux 风格语义执行
partial	主路径可用，但存在unsupported flag、固定返回或注释标记的边界	部分参数组合进入明确错误或兼容分支
stub	为了兼容用户态探测返回固定值或有限字段，不提供完整子系统语义	用户态feature probe 可继续，但应使用fallback
unsupported	返回ENOSYS、EINVAL、dummy fd 边界或显式unsupported helper	用户态应走fallback 或暴露明确错误

该分类比单纯统计match arm 数量更能反映真实情况：用户程序常先探测内核能力再选择路径，明确的partial/stub 行为比伪装的“实现”更有助于用户态做出正确决策。

3.5 与StarryOS 的关键差异

TheKernel 系统调用层相对StarryOS 的主要改进体现在三处具体机制上：

1. StarryOS baseline 的dispatcher 不维护restart 状态，信号打断阻塞系统调用后的返回值处理分散在各个系统调用文件中，行为不一致。TheKernel 将restart 集中到dispatcher 与RestartTracker。

⁵实现见kernel/src/syscall/sys.rs:430-465。

- 2. StarryOS baseline 中每个系统调用都走完整的入口与出口流程。TheKernel 通过对 getter 与 clock_gettime 的快路径化，降低了短周期高频系统调用的bookkeeping 开销。
- 3. StarryOS baseline 对LoongArch time64 的支持较窄，TheKernel 扩展了alias 映射的覆盖范围。

阶段	机制	返回结果
架构入口	用户态执行syscall 指令后，RISC-V64 与LoongArch64 trap 处理保存寄存器，并把现场交给UserContext。	内核获得syscall 编号、参数寄存器和返回寄存器位置。
编号检查	handle_syscall 先处理非法编号和架构alias，无法识别的编号直接映射到Linux errno。	非法编号返回-ENOSYS；合法编号进入fast path 或family dispatch。
快速返回	getpid/gettid/凭证getter 和常见时间读取不进入阻塞路径，也不建立restart 状态。	直接把返回值写回用户寄存器。
restart 登记	可能阻塞或可能被信号打断的syscall 会保存可重启现场，并记录Sys/NoIntr/NoHand 分类。	后续信号路径可以判断重放syscall 还是返回-EINTR。
family dispatch	主match 把编号分到fs、mm、net、task、ipc、sync、time、signal 等实现族。	实现层返回成功值或内核错误。
返回与信号	实现族结束后回到统一出口；错误被转换成Linux errno，pending signal 会先进入handler/trampoline 或restart 处理。	uctx.set_retval 写回最终结果，然后返回用户态。

图 3.1 Syscall Dispatcher 从入口到返回的执行路径

第四章 进程与线程：Linux 任务模型的对象化表达

4.1 问题：一个结构体装不下Linux 的进程/线程语义

Linux的clone(2) 系统调用接受约20 个flags，决定新创建的执行实体是“进程”还是“线程”；更准确地说，它决定哪些内核状态与父共享、哪些独立。pthread 创建线程时会同时请求共享线程组、地址空间、fd 表和信号处置，使新线程拥有独立的TID、内核栈和调度实体，却复用进程级资源。而fork 创建进程时，除少数共享标识外几乎所有资源都复制。

若将这些状态混入单一结构体（StarryOS baseline 即采用此类扁平布局），下列场景将无法被正确表达：

1. clone thread 的新线程共享父进程地址空间；fork 的子进程通过写时复制（COW）获得完整的地址空间副本。单一结构体中的aspace 字段无法同时表达“共享”和“独立”两种语义。
2. execve 替换地址空间时，线程组中除调用者外的所有线程必须被终止，因为它们运行在即将被替换的地址空间上。但PID、parent 关系、非CLOEXEC 的fd 仍需保留。这就要求存在一个“线程组级别”的数据容器，在execve 期间保持不变。
3. kill(2) 和tkill(2) 分别投递到进程和线程。进程级信号需根据每个线程的blocked mask 选择一个可行的目标线程执行handler；线程级信号直接投递到特定线程。扁平结构中，signal manager 的归属（进程级还是线程级）是模糊的。
4. exit_group(2) 终止线程组内所有线程，但单线程exit(2) 只终止调用者。必须存在一个线程组边界来区分“只退出自己”和“退出整个组”。
5. CPU 时间需要在thread 级别实时累积（线程调度时更新），又需要在process 级别汇总（procfs 的/proc/[pid]/stat 和wait4 的rusage 返回进程的总体使用量）。

TheKernel 的解决方案是将task state 显式拆分为两个层次的对象，如图 4.1 所示：ProcessData 承载线程组内共享的状态，Thread 承载单个线程独有的状态。一个线程组中的所有Thread 都通过Arc<ProcessData> 指向同一个共享容器；fork、exec、signal delivery 和wait 只需要沿着这个边界决定哪些状态共享、哪些状态复制。

```
pub struct Thread {
    pub proc_data: Arc<ProcessData>,
    clear_child_tid: AtomicUsize,
    robust_list_head: AtomicUsize,
    pub signal: Arc<ThreadSignalManager>,
}
```

```

    pub(in crate::task) restart: SpinNoIrq<RestartTracker>,
}

pub struct ProcessData {
    pub proc: Arc<Process>,
    aspace_handle: RwLock<Arc<Mutex<AddrSpace>>>,
    pub signal: Arc<ProcessSignalManager>,
    creds: SpinNoIrq<Credentials>,
    caps: SpinNoIrq<CapabilityState>,
    exec_ctl: SpinNoIrq<ExecControlState>,
    vfork_ctl: SpinNoIrq<VforkControlState>,
}

```

这个节选展示分层边界；完整字段定义见源码。`Thread` 保存单线程独有的退出同步、robust futex、线程级 signal 与 syscall restart。`ProcessData` 保存线程组共享的PID 层级、地址空间句柄、进程级 signal、安全身份和跨线程控制状态。这些状态组共同支撑线程创建、exec、exit、wait 与 procfs 等Linux 可见语义。

4.1.1 与StarryOS baseline 的比较

StarryOS baseline已经提供Process、ProcessGroup 与Session 等底层抽象，负责PID 分配、父子关系树、进程组、会话和zombie/exit 基础状态机。`TheKernel` 在这个基础上补齐Linux 任务语义层：

1. **进程共享容器。** `ProcessData` 接在StarryOS 的Process 之上，集中保存地址空间、凭证、namespace、futex 表和exec/vfork 控制状态。
2. **线程局部容器。** `Thread` 嵌入axtask 的TaskInner，保存clear-child-tid、robust list、线程级signal manager、restart tracker 和CPU 记账。
3. **namespace 与安全模型。** 轻量namespace 承载CLONE_NEW* 语义；credentials 与capabilities 负责权限检查、exec 后能力继承和uid/gid 变化后的修正。
4. **退出与等待协议。** `do_exit()` 与wait 路径维护robust list、clear_child_tid、SIGCHLD、zombie 快照、pidfd wait 和stop/continue 事件。

4.2 进程共享状态容器

进程共享状态由ProcessData 承载。执行execve 只替换用户态映像，不替换这个对象；PID、父子关系、文件描述符表和线程组级同步状态因此可以继续保留。

信号管理器、futex 表、CPU 时间汇总和wait 所需的退出快照也放在这里。线程创建、exec、exit 和wait 都以这个对象作为进程级协议的锚点。

4.2.1 身份与可执行映像

ProcessData 的身份段分成三类状态。第一类是 `proc`，它连接到底层进程树，提供 PID、父子关系、进程组、会话、线程成员和 `zombie` 状态；这些状态由 StarryOS 的 `Process` 对象维护，`ProcessData` 只保存稳定引用。第二类是用户态可观察的映像元数据，包括 `exe_path`、`cmdline` 和当前运行的可执行文件。`exec` 成功后会替换路径、任务名和参数向量，但不会替换 `ProcessData` 本身，因此 PID、父子关系、`fd table` 和 `wait` 关系继续有效。第三类是生命周期保护：`executable` 保存正在运行映像的 `inode key`，`retain_executable()` 与 `replace_executable()` 通过 `acquire/release` 配对，防止可执行文件在进程仍持有映像时被过早释放。相关字段与访问器见 `kernel/src/task/process.rs:471-484` 和 `kernel/src/task/process.rs:741-755`。

两个启动时间字段服务不同的 Linux 视图。墙上时钟秒数用于 `procfs` 中可见的进程启动时间；单调时钟纳秒数用于不受系统时钟调整影响的 `uptime` 和运行时长计算。把二者固定在进程共享对象上，可以让线程退出、`exec` 和 `wait` 都读取同一个创建时间基准。

4.2.2 地址空间与资源范围

`aspace_handle` 保存的是地址空间句柄，而不是把 `AddrSpace` 直接嵌入进程对象。这让 `clone` 可以根据 `CLONE_VM` 选择共享同一个 `Arc<Mutex<AddrSpace>>`，`fork` 可以创建 COW 后的新句柄，`execve` 可以在提交点一次性替换整套用户地址空间。外层 `RwLock` 保护句柄本身：普通路径读取当前句柄，`exec` 提交时写锁交换新旧句柄；内层 `Mutex` 保护页表和 `VMA` 元数据，使缺页、`mmap`、`munmap` 与权限修改都落到地址空间自己的锁边界内。`replace_aspace()` 只做句柄交换，旧地址空间在后续引用释放时回收页表页和文件映射，这使 `exec` 的提交点清楚地落在一次原子式的对象替换上。

`scope` 是进程级 `scope-local` 容器，`fd table`、文件系统上下文和写入凭证都通过它挂到当前线程执行环境上。线程进入 CPU 时绑定 `scope`，离开 CPU 时解绑；底层调度任务因此可以在 `syscall` 中访问 Linux 进程资源，同时保持 `axtask` 的调度对象不直接持有 `fd table`。`heap_top` 以原子值记录 `brk()` 管理的用户堆顶，`rlim` 用读写锁保存资源限制，`fork` 时复制，后续 `RLIMIT_NPROC`、文件大小、栈大小和 CPU 时间等检查都从这里取得进程级限制。相关字段见 `kernel/src/task/process.rs:485-494`，地址空间替换接口见 `kernel/src/task/process.rs:681-689`。

4.2.3 权限与安全

代码4.1 Credentials 与 CapabilityState

```

23  #[derive(Debug, Clone, Copy, Default)]
24  pub(crate) struct Credentials {
25      pub(in crate::task) ruid: u32,
26      pub(in crate::task) euid: u32,
27      pub(in crate::task) suid: u32,
28      pub(in crate::task) fsuid: u32,

```

```

29     pub(in crate::task) rgid: u32,
30     pub(in crate::task) egid: u32,
31     pub(in crate::task) sgid: u32,
32     pub(in crate::task) fsgid: u32,
33 }
34
35 #[derive(Debug, Clone, Copy)]
36 pub(crate) struct CapabilityState {
37     pub(crate) effective: [u32; CAPABILITY_WORDS],
38     pub(crate) permitted: [u32; CAPABILITY_WORDS],
39     pub(crate) inheritable: [u32; CAPABILITY_WORDS],
40     pub(crate) bounding: [u32; CAPABILITY_WORDS],
41     pub(crate) ambient: [u32; CAPABILITY_WORDS],
42     pub(crate) securebits: u32,
43 }

```

权限状态在 `ProcessData` 中由 `creds`、`caps` 与 `supplementary_groups` 持有；对应字段位于 `kernel/src/task/process.rs:511-542`。第一层是身份模型：`Credentials` 保存 `real/effective/saved/fs uid` 和 `gid`，补充组列表用于文件权限和组成员资格判断。`Credentials` 可按值复制，`setuid` 系列调用可以在持锁状态下取出 `old/new` 两个快照，再释放凭证锁后调整 `capability`，避免把凭证锁和 `capability` 锁长时间嵌套在一起。第二层是能力模型：`CapabilityState` 保存 `effective`、`permitted`、`inheritable`、`bounding`、`ambient` 五个 `capability` 集合，并用 `securebits` 控制 `setuid` 修正规则。位集宽度由 `CAP_LAST_CAP` 推导，非法 `capability` 编号会在 `cap_mask()` 处被拒绝。

`personality`、`no_new_privs`、`seccomp_mode`、`thp_disabled` 与 `membarrier_state` 则是进程级控制面。`no_new_privs` 只从 0 变为 1，`seccomp_mode` 保存 `prctl(PR_SET_SECCOMP)` 记录的模式，`thp_disabled` 对应透明大页禁用开关，`membarrier_state` 记录进程是否注册过对应 `membarrier` 能力。`prctl`、`personality`、`membarrier` 和 `procfs` 查询都读取同一个线程组共享快照；`clone` 生成新进程时，这些字段由父进程复制到新的 `ProcessData`。`exec` 成功后会清理 `membarrier` 注册和 `keep-caps` 状态，避免旧映像的进程级优化或保留能力直接泄漏到新程序。

4.2.4 UID/GID 变更的 `capability` 修正协议

UID/GID 变更涉及权限提升与降级，是 `TheKernel` 中安全敏感的核心路径。当 UID（或 GID）变更时，Linux 的 `capability` 模型要求自动调整 `effective/permitted sets`，使进程的能力与新的身份相符。

UID/GID 修改入口先做权限判定，再更新共享凭证。具备 `CAP_SETUID` 或 `CAP_SETGID` 的调用者可以直接写入 `real/effective/saved/fs ID`；普通调用者只能在 Linux 允许的现有 ID 集合内切换，例如把 `effective uid` 切回 `real uid` 或 `saved uid`。`setreuid/setresuid` 会同时维护 `saved uid` 和 `fsuid`，使后续文件权限检查看到的身份与 Linux 规则一致。GID 路径采用同样的授权模式，但 `capability` 自动修正只跟 UID/FSUID 相关。

UID 变化路径先在持锁状态下捕获 `old/new` 凭证快照，再在锁外执行 UID 修正规则；FSUID 变化则执行文件系统身份专用的 `capability` 修正规则。相关实现集中在 `kernel/src/`

task/process.rs:1101-1348。规则本身有四个决策：

1. 如果real/effective/saved uid 三者没有变化，capability 集合不需要调整。
2. 如果SECBIT_NO_SETUID_FIXUP 已设置，setuid 不再触发自动capability 修正。
3. 如果effective uid 从0 变为非0，先清空effective；如果real/effective/saved uid 全部离开0 且没有SECBIT_KEEP_CAPS，再清空permitted 和ambient。
4. 如果effective uid 从非0 变为0，则把permitted 复制到effective，使进程重新获得permitted 集中的能力。

FSUID 变化只影响文件系统相关capability。代码中的FS_CAPS 常量把这八个能力限定为文件所有权、DAC 绕过、FOWNER/FSETID、MAC 绕过和immutable 文件相关权限；旧FSUID 离开0 时清除这些effective 位，进入0 时只恢复permitted 中已有的位。这个规则让setfsuid() 能改变文件访问身份，同时避免把网络、进程控制或系统管理类capability 一并打开。

4.2.5 信号与同步原语

进程级事件、退出信号和同步字段位于kernel/src/task/process.rs:496-510，体现的是“线程组共享、线程局部消费”的边界。exit_signal 决定最后一个线程退出时父进程是否收到SIGCHLD 或clone 指定的其他信号；signal 保存进程级signal disposition、pending queue 和signal trampoline 信息；futex_table 则把私有futex 的等待队列放在进程地址空间边界内，供共享地址空间的线程共同访问。线程本身仍保留自己的blocked mask、线程pending queue 和备用信号栈；进程级信号只决定“这个信号属于哪个线程组”，实际执行handler 的线程还要经过线程局部状态筛选。

进程级信号投递入口位于kernel/src/task/signal.rs:212-253。它先拒绝zombie 或无线程的进程，使kill(2) 对已经不可投递的目标返回明确错误；随后处理SIGCONT，因为继续一个stopped 进程不依赖该信号的用户disposition。若ptrace stop 接管了该信号，投递路径会中断线程组内所有线程，使traced stop 尽快被观察到；普通信号则进入进程pending queue，再遍历线程组唤醒没有阻塞该signo 的线程。线程级信号投递另走TID 入口，因此kill(2) 与tkill(2) 的语义差异落在对象边界上，调用处无需临时判断。

4.2.6 进程生命周期事件（PollSet 体系）

TheKernel使用Arc<PollSet> 作为内核内部的异步等待者集合。等待路径注册waker，状态改变的一侧调用wake()；具体语义仍由ProcessData 的状态机和wait 路径负责判定。

生命周期通知按消费方式分为六类：

1. child_exit_event 面向父进程的wait 路径。子进程退出或继承来的zombie 需要新父进程回收时，退出路径唤醒父进程上的等待者。

2. `exit_event` 面向等待本进程退出的内核路径，例如 `pidfd poll`。它在线程组完成退出后触发；单个线程退出只唤醒线程级等待路径。
3. `exec_event` 服务于多线程 `exec` 去线程化。发起 `exec` 的线程等待兄弟线程退出；兄弟线程退出后唤醒该事件，使 `exec` 线程重新检查线程组是否只剩自己。
4. `stop_event` 连接 `job control` 和 `wait`。进程从 `stopped/stopping` 回到 `running` 时唤醒等待 `WCONTINUED` 或 `stop` 状态变化的 `wait` 路径。
5. `vfork_event` 只服务 `CLONE_VFORK`。子进程完成 `exec` 或整个线程组退出后释放父线程。
6. `wait_lock` 是 `destructive wait` 的串行化边界。`wait` 在扫描、声明 `stop/continue` 状态或回收 `zombie` 前持有它，避免两个 `wait` 调用消费同一个状态。

这些事件的唤醒点集中在退出和 `exec` 相关路径上。最后一个线程退出时，内核先完成文件描述符关闭、`ptrace` 断链、共享内存和文件锁清理，再发布 `zombie` 快照；父进程通知发生在 `zombie` 状态可见之后，`pidfd poll`、`vfork` 释放和 `exec` 等待者也在同一收尾阶段被唤醒。这个顺序保证 `wait` 观察到子进程退出时，子进程持有的 `fd` 和锁已经执行完关闭副作用。字段定义位于 `kernel/src/task/process.rs:553-569`；退出唤醒点位于 `kernel/src/task/ops.rs:660-685`。

4.2.7 控制面：job control、ptrace、exec、vfork

这些子状态机存储在 `ProcessData` 的专用 `SpinNoIrq` 字段中。它们共同解决一个问题：Linux 把 `stop/continue`、`ptrace`、`exec` 去线程化和 `vfork` 阻塞都定义在进程或线程组层面，状态必须集中在共享容器里，单个线程只负责在调度或信号边界响应该状态。

代码4.2 job、ptrace、exec 与 vfork 控制状态

```

42  #[derive(Debug, Clone, Copy)]
43  pub(in crate::task) struct JobControlState {
44      pub(in crate::task) state: StopState,
45      pub(in crate::task) stop_signal: u8,
46      pub(in crate::task) stop_kind: StopKind,
47      pub(in crate::task) continued: bool,
48      pub(in crate::task) stop_reported: bool,
49  }
50
51  impl Default for JobControlState {
52      fn default() -> Self {
53          Self {
54              state: StopState::Running,
55              stop_signal: 0,
56              stop_kind: StopKind::JobControl,
57              continued: false,
58              stop_reported: false,
59          }
60      }
61  }
62
63  #[derive(Debug, Clone, Copy, Default)]

```

```

64 pub(in crate::task) struct PtraceControlState {
65     pub(in crate::task) tracer: Option<Pid>,
66     pub(in crate::task) options: u32,
67     pub(in crate::task) event_message: usize,
68 }
69
70 #[derive(Debug, Clone, Copy, Default)]
71 pub(in crate::task) struct ExecControlState {
72     pub(in crate::task) owner: Option<Pid>,
73 }
74
75 #[derive(Debug, Clone, Copy, Default)]
76 pub(in crate::task) struct VforkControlState {
77     pub(in crate::task) parent_tid: Option<Pid>,
78 }

```

Job Control

job control 围绕图 4.3 中的三态转换展开。Running 表示线程可继续运行；Stopping 是停止信号到达后等待线程实际停下的窗口；Stopped 是可被wait 报告的稳定状态。stop_signal 和 stop_kind 记录wait 需要返回的停止原因，stop_reported 与 continued 分别让WUNTRACED 和WCONTINUED 事件只被消费一次。状态迁移入口集中在kernel/src/task/process.rs:1456-1584，wait 路径通过claim/restore 接口先声明事件，再完成用户态写回；写回失败时恢复状态，避免一次失败的copy 把stop/continue 事件吞掉。

状态迁移有四条路径：

1. begin_stop() 只接受Running，进入Stopping 并记录停止信号。已经处于停止相关状态时返回失败，避免重复开始同一次stop。
2. finish_stop() 只接受仍在Stopping 的进程，提交为Stopped，并重置wait 可见的报告位。
3. continue_job() 可以取消Stopping，也可以恢复Stopped。完整的Stopped 到Running 会设置continued，随后唤醒stop_event。
4. ptrace_stop() 直接进入Stopped，并把停止类型标记为Ptrace，让wait 路径把traced stop 与普通job-control stop 区分开。

Ptrace

ptrace 控制面有两个方向：被谁追踪，以及自己正在追踪谁。begin_ptrace() 要求当前没有tracer，成功后清空旧options 和event message；end_ptrace() 要求调用者与已记录tracer 一致，避免无关进程拆掉追踪关系；clear_ptrace() 用于退出清理，返回旧tracer 以便上层断链。ptrace_tracees 则维护反向列表，添加时去重，清理时一次性取出旧列表，供退出路径解除所有tracee 的追踪状态。这样attach/detach 的双向关系都由ProcessData 串行化，线程退出时只需要沿着进程共享对象清理，而不需要在每个线程上维护独立tracer 集合。实现入口见kernel/src/task/process.rs:1393-1439。

Exec 控制

exec去线程化用一个owner TID 表达“当前线程组正在被谁改写”。owner 存在期间,clone 不能向该线程组添加新线程,非owner 线程在信号检查边界会退出,owner 则等待线程组收缩到只剩自己后再替换地址空间。这个协议把“杀死兄弟线程”和“提交新映像”之间的窗口显式化,避免旧地址空间中的线程继续运行。实现入口见kernel/src/task/process.rs:1586-1637 和kernel/src/syscall/task/execve.rs:57-98。

协议分为四步:

1. `begin_exec()` 在空闲时登记owner; 同一个owner 重入视为成功,其他线程发起的exec 被拒绝。
2. `should_exit_for_exec()` 让非owner 线程在信号检查边界发现自己应退出,避免它们继续运行在即将被替换的地址空间上。
3. `exec_ready()` 同时检查owner 仍有效且线程组只剩owner,本质上是地址空间替换前的安全门。
4. `end_exec()` 只允许owner 清除状态,并唤醒`exec_event`,让等待中的exec 路径或退出路径重新检查条件。

Vfork 控制

vfork的状态比exec 更小,但语义更强:父线程必须停在clone 返回之前,直到子进程exec 成功或退出。子进程的ProcessData 记录被阻塞的父TID;父线程在`vfork_event` 上等待;释放路径取走`parent_tid` 后唤醒父线程。这个状态挂在子进程上,正好对应Linux 的CLONE_VFORK 语义:阻塞关系由新创建的执行实体负责解除。

等待循环采用“双检查”结构:父线程先查看子进程是否仍处于vfork 状态,再把waker 注册到`vfork_event`,注册后再次检查一次。这样可以覆盖子进程在第一次检查之后、waker 注册之前已经exec 或退出的竞争窗口。释放入口见kernel/src/task/process.rs:1639-1656,等待循环见kernel/src/syscall/task/clone.rs:153-165。

4.2.8 Namespace 体系

TheKernel 把namespace 做成挂在ProcessData 上的轻量对象。共同模式是:对象有稳定ID, fork 时按CLONE_NEW* 决定复制或共享,procfs 通过ID 暴露namespace fd 的inode 语义。

前三类namespace 只保存Linux ABI 当前需要的最小状态,定义见kernel/src/task/process.rs:53-153:

1. cgroup namespace 当前是纯身份对象。它的ID 用于procfs namespace fd 和cgroup 打开时的归属检查, fork 新namespace 时重新分配ID。

2. PID namespace 额外记录parent 与 init PID。它把该namespace 的init 进程映射为可见PID 1，其余PID 暂时保持全局值，为后续更完整的层级PID 映射留下边界。
3. user namespace 记录parent 与 owner uid。新用户 namespace 用当前进程的effective uid 作为owner，使权限判断可以区分“在父namespace 中谁创建了这个root 视图”。

UTS namespace 的状态是可变的，因此对象内部用SpinNoIrq<UtsState> 包住hostname 和domainname。fork 创建新UTS namespace 时深拷贝当前状态；共享UTS namespace 时多个进程读写同一个对象，从而对应Linux 中CLONE_NEWUTS 和默认共享的差异。nodename 与domainname 的读写接口只暴露字节向量，字符串长度和截断规则留在UtsState 内部处理，调用路径不直接操作固定数组。

Time namespace 围绕“用户可见时间”和“宿主deadline”两个方向做换算。它只保存monotonic 与boottime 两个偏移量；读取CLOCK_MONOTONIC 或CLOCK_BOOTTIME 时把偏移加到内核时间上，用户态提交绝对超时时再反向扣除偏移，转换回宿主alarm 使用的deadline。所有换算最后都经过饱和和加减和Duration 边界裁剪，避免负时间或超过u64 纳秒范围的值进入timer 子系统。UTS 与time namespace 的实现见kernel/src/task/process.rs:204-319。

clone 路径只在对应CLONE_NEW* flag 出现时fork 新namespace；否则把父进程现有对象的Arc 传入新的ProcessData。Time namespace 稍有不同：clone 读取的是time_ns_for_children()，因为unshare(CLONE_NEWTIME) 只改变未来子进程继承的namespace，调用者当前看到的时间仍来自原namespace。

ProcessData 同时保存当前time namespace 和future-children time namespace。replace_time_ns() 会同时替换两者，unshare_time_ns() 只fork 后者；clone 对NEW namespace flags 的选择见kernel/src/syscall/task/clone.rs:344-364，time namespace 访问器见kernel/src/task/process.rs:715-731。

4.2.9 调度与资源上下文

这组状态描述的是调度器外侧的Linux 资源上下文。ioprio 是用户可配置的I/O 优先级原始值；io_context 和sysvsem_undo 是clone 共享语义所需的身份对象；mempolicy 保存进程默认策略和地址区间覆盖，供set_mempolicy、mbind、procfs numa maps 等路径读取。这些字段位于kernel/src/task/process.rs:521-528，clone 的继承逻辑位于kernel/src/syscall/task/clone.rs:395-412。

clone 对这些资源有三种处理方式：

1. CLONE_IO 和 CLONE_SYSVSEM 出现时共享父进程的身份对象；未设置时创建新的Arc<()>，表达独立的Linux 共享上下文。
2. I/O 优先级按数值继承。子进程得到父进程当时的raw ioprio，后续修改互不影响。
3. mempolicy 复制完整状态。默认策略和per-range 覆盖一起复制到子进程，使fork 后的

内存策略视图与父进程一致。

`ioprio` 与 `mempolicy` 的进程级读写接口不直接暴露内部锁。读路径返回当前 `raw ioprio` 或策略快照；写路径在 `SpinNoIrq` 边界内替换默认策略和 `per-range` 覆盖，保证 `procfs`、`clone` 和 `syscall` 读取到的是同一个进程级视图。网络 `namespace` 的进程字段保存 `axnet-ng` 的 `NetStack`：默认 `clone` 共享父进程的栈，`CLONE_NEWNET` 创建一个只有 `loopback` 设备的新栈，避免新 `namespace` 一开始继承外部连接性。这样网络隔离的边界落在实际 `socket/router/listen table` 所属的栈对象上，而不只是一个 `namespace ID`。相关接口见 `kernel/src/task/process.rs:909-962` 和 `crates/axnet-ng/src/net_stack.rs:55-75`。

4.2.10 POSIX Timers

POSIX timer 是进程级资源，`ProcessData` 中的 `timer` 数组用 `slot` 下标作为 `timerid`。每个 `slot` 保存三个维度：计时基准、通知方式和运行状态。计时基准决定使用 `realtime` 还是 `monotonic alarm`；通知方式决定到期时是否发信号以及是否定向到某个 `TID`；运行状态记录 `interval`、`deadline`、`sequence`、`overrun` 和 `signal-pending` 位。

完整协议由创建、设置和到期三个阶段组成。第一，`timer_create()` 解析 `clock id` 和 `sigevent`，在进程 `timer` 数组中复用空 `slot` 或追加新 `slot`，然后把 `slot` 下标写回用户态。第二，`timer_settime()` 校验 `timerid` 与 `flags`，把相对或绝对时间转换为 `deadline`，重置 `overrun` 与 `pending` 状态，并递增 `sequence` 后注册 `alarm`；`sequence` 是这个协议的防陈旧机制，旧 `alarm` 到期时如果发现 `sequence` 不匹配，会直接退出。第三，`alarm` 回调在持锁状态下计算本次到期跨过了多少个 `interval`，单次 `timer` 会清空 `deadline`，周期 `timer` 会推进到下一个 `deadline` 并重新注册 `alarm`；如果上一次信号仍处于 `pending`，则累积 `overrun`，如果可以投递，则生成带 `timerid` 和 `overrun` 的 `siginfo`，按配置发往进程或特定线程。结构定义见 `kernel/src/task/timer.rs:58-108`，三个阶段的实现分别见 `kernel/src/syscall/time.rs:583-688` 与 `kernel/src/task/timer.rs:715-790`。

4.2.11 CPU 时间统计

Linux 的 `getrusage()`、`wait4()` 和 `times()` 都要求进程级视图，而 CPU 时间实际从线程级计账产生。因此 `TheKernel` 把用量分成三本账：仍存活线程的即时快照、已经退出兄弟线程的累积值、已经被 `wait` 回收的子进程树累积值。最大 `RSS` 单独用原子 `maximum` 保存，避免 `procfs` 或 `wait` 路径读取时重新计算历史峰值。

统计路径按生命周期推进：

1. 线程仍存活时，`live_process_usage()` 遍历线程组，读取每个线程的 `TimeManager` 或最新快照，再加上已退出线程的累积账。
2. 线程退出时，退出路径调用 `account_exited_thread()`，把该线程最终的 `user/system` CPU 时间转入进程级累积账。

3. 父进程 wait 回收 zombie 时，wait 路径把子进程的 `total_usage()` 加入 `waited_children_usage`，因此 `RUSAGE_CHILDREN` 不需要保留已回收进程对象。
4. `self_usage()` 合成当前进程自身用量，并用 `sample_maxrss_kb()` 更新最大 RSS；`children_usage()` 返回 wait 账本；`total_usage()` 用于进程退出快照，把自身和已回收子树一起带给父进程。

`AtomicTaskUsage` 用原子累加 user/system CPU 时间，并用 compare-exchange 维护最大 RSS；`live_process_usage()` 从线程组读取仍存活线程快照，再加上已退出线程账本。进程级接口见 `kernel/src/task/accounting.rs:127-190` 和 `kernel/src/task/process.rs:821-860`。

4.2.12 ProcessData::new：建立进程共享对象

`ProcessData::new()` 接收地址空间、文件系统上下文、信号处置、namespace 与调度相关参数，建立一个完整的 Linux 进程共享对象。初始化顺序分成四段：

1. 先记录墙上时钟和单调时钟创建时间，并建立进程树、地址空间句柄、资源限制、scope 与初始堆顶。
2. 再根据传入的 `signal_actions` 和 `trampoline` 地址创建进程级 signal manager，并初始化 `futex` 表、计时器集合和 `rusage` 账本。
3. 权限状态先使用 `root/default` 快照建立对象；`clone` 路径随后用父进程的 `credentials`、`capability` 和 `supplementary groups` 覆盖这些字段，使 `fork` 继承发生在对象创建之后。
4. 最后挂入 `job control`、`ptrace`、`exec`、`vfork`、`membarrier` 与 `namespace` 对象。当前 `time namespace` 与 `future children time namespace` 初始相同，后续 `unshare(CLONE_NEWTIME)` 只改变后者。

这段初始化先创建完整对象，再由 `clone` 路径覆盖父进程继承项。好处是失败边界集中在 `clone` 的资源分配和回写阶段：`ProcessData::new()` 产出的对象始终处于内部自洽状态，后续只是在明确字段上应用父进程快照。实现见 `kernel/src/task/process.rs:589-673`。`ProcessData` 字段在进程地址空间、凭证、命名空间等维度上的整体布局如图 4.2 所示。

4.3 Thread：线程局部状态的承载者

`Thread` 持有每个线程独立的状态。它作为 `TaskExt trait` 的具体实现被嵌入到 `axtask` 的 `TaskInner` 中，从而把 Linux pthread 语义注入到底层调度实体之上。字段定义见 `kernel/src/task/thread.rs:37-88`。

4.3.1 Thread 的线程局部边界

`Thread` 的状态按语义边界分成五组：

1. **线程身份**。`proc_data` 把线程接回线程组共享状态；`visible_tid` 是用户可见TID，允许非leader exec 后把调用线程重新绑定为进程PID。
2. **退出同步**。`clear_child_tid` 和 `robust_list_head` 服务 `pthread join` 与 `robust futex` 恢复。它们只属于单个线程，因为 `join` 地址和 `robust list` 都是线程局部ABI。
3. **信号与重启**。`signal` 是线程级 `signal manager`，保存 `blocked mask`、线程 `pending queue` 和备用信号栈；`restart` 保存第3章讨论的 `syscall restart` 状态。
4. **调度计账**。`time` 只在上下文切换边界被当前线程可变借用，`live_usage` 发布无锁快照，供 `procfs` 和进程级聚合读取。
5. **可观察状态与安全边界**。`proc_state_hint` 给 `procfs` 一个 `best-effort` 阻塞状态；`exit` 是调度与信号路径检查的退出标志；`oom_score_adj`、`accessing_user_memory` 和 `active_scope_read_held` 分别服务 OOM 评分、core dump 一致性和 `scope` 读锁生命周期。

退出路径说明了为什么这些状态必须在线程对象上：线程退出时先清理自己的 `clear_child_tid`，再按 `futex key` 唤醒一个等待者；随后读取自己的 `robust list head`，并遍历用户态链表修复持锁死亡状态。父进程 `wait` 只看到进程级 `zombie`，而 `pthread join` 需要的是这个线程局部清理点。对应清理入口位于 `kernel/src/task/ops.rs:582-607`。

4.3.2 与调度器的集成

`Thread` 为 `axtask` 的 `TaskExt` trait 提供两个钩子。进入 CPU 时，线程先把 `active scope` 绑定到所属进程的 `scope`，然后恢复 CPU 时间计账；离开 CPU 时，线程暂停计账、发布 `live_usage` 快照，处理到期的 `per-thread timer` 信号，最后把 `active scope` 恢复成全局 `scope` 并释放之前保留的读锁。

这个协议把 Linux 语义压到调度边界：`fd table`、`fs context` 等 `scope-local` 资源只在用户线程运行期间指向该进程；CPU 时间只在实际占用 CPU 的窗口累积；内核线程重新使用全局 `scope`，不会误读上一个用户线程的进程资源。上下文切换钩子见 `kernel/src/task/thread.rs:216-297`。

`AsThread` trait (`thread.rs:300-315`) 提供了一个从 `TaskInner` (`axtask` 的底层任务结构) 安全提取 `Thread` 引用的方法。所有 `syscall` 代码通过 `current().as_thread()` 访问当前线程状态。

4.4 全局任务表与操作集合

`kernel/src/task/ops.rs` 提供了全局任务、进程、进程组和 `session` 的弱引用索引，以及跨任务操作原语。这几张表共同解决 Linux 的多种 ID 视图如何映射到同一批内核对象。

4.4.1 四张全局弱引用表

这些索引以及同文件中的session索引都使用WeakMap。强引用由调度任务、进程层级和线程组成员关系持有；全局表只负责查找，不延长对象生命周期。这样任务退出后，强引用清零即可让表项自然失效，避免手写“从所有全局表删除”的长尾清理。表定义位于kernel/src/task/ops.rs:35-40，插入协议位于kernel/src/task/ops.rs:102-158。

add_task_to_table()的插入顺序体现了四层ID视图：

1. 调度器TID总是插入TASK_TABLE，这是最直接的内核任务索引。
2. 如果用户可见TID与调度器TID不同，再插入TASK_ALIAS_TABLE。这个别名支持非leader exec后的TID重绑定，以及pidfd等用户可见查找路径。
3. 首次看见某个PID时插入PROCESS_TABLE，把进程ID映射到ProcessData。
4. 首次看见某个PGID或SID时插入进程组与session表。后续同组进程复用已有表项。

这与Linux内核的PID哈希表加task_struct链表结构对应，但TheKernel以WeakMap替代手动的引用计数与回收逻辑，强引用清零时条目自动移除。

4.4.2 孤儿进程回收协议

父进程先退出时，子进程会被重新挂到新的reaper。继承来的zombie不能静默留在旧父进程的wait队列里，必须按新父进程的SIGCHLD策略重新处理：

1. 如果找不到新的parent或parent的ProcessData，通知路径直接返回；此时没有可唤醒的wait目标。
2. 如果子进程的退出信号是SIGCHLD，先检查新父进程是否忽略SIGCHLD或设置SA_NOCLDWAIT。满足自动回收条件时，直接reap()并从cgroup解绑。
3. 否则，根据子进程保存的exit_signal向新父进程投递信号；如果SIGCHLD被显式忽略，则抑制信号。
4. 最后唤醒新父进程的child_exit_event，让wait路径重新扫描可回收子进程。

这段逻辑位于kernel/src/task/ops.rs:42-75。它的关键点是把“继承zombie”视为新父进程上的新事件，而不是把旧父进程的通知状态原样搬过去；因此自动回收、信号投递和wait唤醒都会按新parent的disposition重新计算。

4.4.3 Robust Futex 退出处理

Robust futex是线程死亡时的用户态互斥锁恢复协议。线程退出时，内核读取用户态注册的robust list，并对仍由该线程持有的futex标记owner-died。流程分为四步：

1. 先读取RobustListHead，取得链表头、futex 偏移和list_op_pending。读取失败或用户地址非法时直接停止，不让退出路径被坏用户指针拖死。
2. 对每个非pending 条目，根据entry 地址和offset 算出futex 字地址。只有futex 中记录的TID 与当前退出线程匹配时，CAS 才会把owner 位改成FUTEX_OWNER_DIED 并保留waiters 位。
3. 标记成功后，按该地址构造FutexKey，如果等待队列存在就唤醒一个等待者，让用户态互斥锁恢复路径接管。
4. 链表遍历受ROBUST_LIST_LIMIT 限制，每步后让出CPU；普通链表走完后再处理pending 条目，覆盖线程死在list 操作中间的窗口。

实现位于kernel/src/task/ops.rs:481-580。用户指针读取失败只终止robust list 清理，不阻止线程退出；CAS 失败也只表示futex word 已被用户态或其他路径改变，退出路径继续处理后续节点。这个失败策略保证坏链表不能卡死进程退出。

4.4.4 跨地址空间VM 写入

代码4.3 临时切换用户页表根的跨地址空间写入

```

314 pub fn with_current_user_page_table_root<R>(root: PhysAddr, f: impl FnOnce() -> R) -> R {
315     let _guard = NoPreemptIrqSave::new();
316     let curr = current();
317     let curr_ptr = (&***curr) as *const TaskInner as *mut TaskInner;
318     let old_root = axhal::asm::read_user_page_table();
319     install_current_user_page_table_root(curr_ptr, root);
320     let result = f();
321     install_current_user_page_table_root(curr_ptr, old_root);
322     result
323 }
324
325 /// Writes `value` into `ptr` within `aspace_handle`.
326 ///
327 /// The caller may target a different process or a not-yet-running child, so we
328 /// must both pre-populate the destination pages in that address space and then
329 /// execute the copy under the matching user page table root.
330 pub fn vm_write_in_aspace<T: Copy>(
331     aspace_handle: &Arc<Mutex<AddrSpace>>,
332     ptr: *mut T,
333     value: T,
334 ) -> AxResult<()> {
335     let layout = Layout::new::<T>();
336     let start = VirtAddr::from_usize(ptr.addr());
337     let root = {
338         let mut aspace = aspace_handle.lock();
339         if !aspace.can_access_range(start, layout.size(), MappingFlags::WRITE) {
340             return Err(AxError::BadAddress);
341         }
342         let page_start = start.align_down_4k();
343         let page_end = (start + layout.size()).align_up_4k();
344         aspace.populate_area(page_start, page_end - page_start, MappingFlags::WRITE)?;
345         aspace.page_table_root()
346     };
347     with_current_user_page_table_root(root, || ptr.vm_write(value)).map_err(Into::into)

```

348 }

`vm_write_in_aspace()` 解决的是“当前线程替另一个地址空间写用户指针”的问题。`clone` 创建子任务时, 内核可能需要把 `child TID` 或 `pidfd` 写入子进程用户地址, 但当前 CPU 仍在父线程上下文中运行。该函数先在目标 `AddrSpace` 中验证写范围并填充目标页面, 再在禁止抢占的临界区内临时安装目标用户页表根, 执行 `vm_write()` 后恢复旧页表根。

这个设计把“要写什么”留在 `clone` 语义层, 把“如何安全切换页表并写入”封装在任务操作层。结果是 `CLONE_CHILD_SETTID`、`CLONE_CHILD_CLEAR_TID` 和 `pidfd` 返回值都复用同一个写入机制, 不需要在 `clone` 路径散落页表切换细节。`clone` 中的调用点见 `kernel/src/syscall/task/clone.rs:462-484`。

4.5 clone/fork/vfork: 按flags 选择共享策略

4.5.1 CloneFlags: 28 个 flags 编码

`CloneFlags` 位标志在单个 `u64` 中编码约 28 个独立的共享策略决定, 定义见 `kernel/src/syscall/task/clone.rs:41-101`。TheKernel 使用 `u64` 作为基础类型, 以容纳 `CLONE_CLEAR_SIGHAND` 和 `CLONE_INTO_CGROUP` 这类超出 32 位掩码范围的扩展标志。

这些 flags 分成四类资源边界。第一组决定进程/线程身份, 例如 `CLONE_THREAD`、`CLONE_PARENT` 和退出信号; 第二组决定共享资源, 例如 `CLONE_VM`、`CLONE_FILES`、`CLONE_FS`、`CLONE_SIGHAND`、`CLONE_IO` 和 `CLONE_SYSVSEM`; 第三组决定用户态回写和同步, 例如 `parent/child TID`、`clear-child-tid`、`pidfd`、`TLS` 与 `vfork`; 第四组决定 `namespace` 与 `cgroup` 放置, 例如 `CLONE_NEWNET`、`CLONE_NEWPID`、`CLONE_NEWUSER`、`CLONE_NEWUTS`、`CLONE_NEWCGROUP` 和 `CLONE_INTO_CGROUP`。下表按这个资源边界解释每个 flag 对子任务的影响。

4.5.2 CloneArgs: ABI 归一化层

代码4.4 CloneArgs 归一化后的参数结构

```
134 #[derive(Debug, Clone, Copy, Default, Eq, PartialEq)]
135 pub struct CloneArgs {
136     pub flags: CloneFlags,
137     pub exit_signal: u64,
138     pub stack: usize,
139     pub tls: usize,
140     pub parent_tid: usize,
141     pub child_tid: usize,
142     pub pidfd: usize,
143     pub cgroup_fd: Option<i32>,
144 }
```

`CloneArgs` 把传统 `clone`、`clone3`、`fork` 和 `vfork` 的入口差异压成同一个内部结构。字段含义分为三组: `flags` 与 `exit_signal` 决定共享策略和退出通知; `stack` 与 `tls` 改写子

线程的用户上下文；`parent_tid`、`child_tid`、`pidfd` 和 `cgroup_fd` 承载用户地址回写和 `clone3` 扩展能力。

传统 `clone` 把退出信号编码在 `flags` 低 8 位，因此入口先拆分 `clone_flags` 和 `exit_signal`；当设置 `CLONE_PIDFD` 时，Linux ABI 复用 `parent_tid` 参数作为 `pidfd` 回写地址，代码在构造 `CloneArgs` 时把它转入 `pidfd` 字段。`clone3` 则直接把用户态结构体中的字段映射到同一对象。进入 `do_clone()` 后，所有入口都走同一套验证和资源分配流程。传统 `clone` 的归一化入口见 `kernel/src/syscall/task/clone.rs:529-558`。

4.5.3 Flags 验证：拒绝无意义的组合

`validate_for()` (`clone.rs:167-212`) 在资源分配之前强制实施一致性约束：

1. 如果子进程与父进程处于同一线程组，它必须共享地址空间和信号处理（否则线程组的概念将无意义）。
2. 共享信号处理器但没有共享地址空间意味着一个线程写入 `handler` 地址时其他线程不可见，这破坏了信号 `delivery` 语义。
3. `vfork` 要求父进程阻塞直至子进程 `exec` 或 `exit`，其语义建立在亲子为独立进程上下文的前提下；与线程组共存会导致语义冲突，故二者互斥。
4. `pidfd` 是进程句柄，因此与 `CLONE_THREAD` 互斥。
5. 同一线程组中的线程必须共享网络 `namespace`。
6. 共享文件系统上下文与新的挂载 `namespace` 冲突。
7. 修改 `UTS namespace` 需要提升权限。
8. 该标志对应 Linux 2.5 时代已移除的分离语义，仅出于兼容性在旧式 `clone` 中接受。
9. 传统 `clone` 缺少 `cgroup_fd` 参数字段。

违反这些约束会被归类为无效参数或权限不足错误。

4.5.4 NPROC 限制实施

`check_rlimit_nproc()` 在 `fork` 资源分配前执行 Linux 的 `per-uid` 任务数量限制。`root` 或具备 `CAP_SYS_RESOURCE/CAP_SYS_ADMIN` 的调用者跳过限制；无限限制同样直接通过。普通用户路径会扫描当前任务表，只统计同一 `uid` 且尚未 `pending-exit` 的任务；计数达到 `soft limit` 时返回 `EAGAIN`，让 `fork` 失败发生在创建新任务之前。实现见 `kernel/src/syscall/task/clone.rs:104-131`。

4.5.5 do_clone：核心创建协议

`do_clone()` 是 `clone` 系统调用族的中心协议。它先把用户态参数归一化为内核状态，再在新任务尚未运行之前完成地址空间、进程树、线程对象、用户态 TID 回写、`pidfd` 和 `cgroup` 计费。这个顺序很重要：任何一步失败都应在子任务进入调度队列之前返回错误，避免留下半初始化的线程或进程。

1. 入口阶段先调用 `validate_for(api)`，拒绝前一节列出的无意义 `flag` 组合。随后将 `exit_signal` 转成 `Signo`，复制父线程的 `UserContext`，按参数覆盖子线程栈指针和 TLS，并把子线程的返回值寄存器设为 0。这样父线程看到的是新 TID，子线程从同一条系统调用返回时看到的是 0。
2. 分配资源之前，内核检查父进程是否处于 `exec de-thread` 阶段；若另一个线程正在接管 `exec`，`clone` 直接返回中断错误。随后清理已退出任务的延迟回收队列，并执行前一节的 `NPROC` 限制检查，让 `fork` 压力在创建任务前被拒绝。
3. 调度状态从父任务复制而来，但独立进程 `fork` 会遵守 `SCHED_RESET_ON_FORK` 语义。实时类、`deadline` 类以及负 `nice` 值都会在子进程侧回退到普通可继承状态；线程克隆不走这条重置路径，因为它仍属于同一个线程组。
4. 资源分叉以 `CLONE_THREAD` 为界。线程克隆复用父进程的 `ProcessData`，并把新任务的页表根设为父地址空间；新进程克隆则先通过 `proc.fork()` 接入进程树，再按 `CLONE_VM`、`CLONE_SIGHAND` 和各类 `CLONE_NEW*` 标志决定共享、复制或新建地址空间、信号处置表和 `namespace`。
5. 新的 `ProcessData` 建立后，父进程可见属性被复制到子进程快照中，包括 `umask`、凭证、`capability`、附加组、`heap top`、I/O 优先级、内存策略、`timer slack`、`no_new_privs`、`seccomp` 模式和 THP 开关。文件描述符表与文件系统上下文则按 `CLONE_FILES` 和 `CLONE_FS` 在共享 `scope` 与深拷贝之间选择。
6. 子任务入队之前完成所有对外可见的注册和回写。线程组插入受 `try_add_thread()` 保护，避免 `clone` 注册落入正在提交的 `exec` 窗口；`CHILD_SETTID`、`CHILD_CLEAR_TID` 和 `PIDFD` 分别设置用户态 TID、退出清零地址和 `pidfd`。若后续 `pidfd` 插入、用户态写入或 `cgroup` 计费失败，`rollback_clone_setup` 会撤销进程树或线程组中的半成品记录。最后，内核把 `Thread` 装入 `AxTaskExt`、按继承后的调度状态生成任务、登记全局任务表，并在 `vfork` 或同步型 `clone` 场景下主动让出 CPU 给予任务。

`do_clone()` 的实现集中在 `kernel/src/syscall/task/clone.rs:214-522`。这段代码的提交点在 `spawn_task_with_sched_from()` 和 `add_task_to_table()` 附近；在此之前的 `pidfd` 插入、用户态回写或 `cgroup` 计费失败都可以通过 `rollback` 撤销半成品。这个顺序比逐项初始化更重要，因为 Linux 用户态只应看到“创建成功的任务”或“完整失败”，不应看到已经进程树可见但无法调度或无法回收的对象。

4.5.6 clone flags 对资源的影响

flag	控制资源	TheKernel克隆时的处理
CLONE_VM	地址空间	未设置：通过COW 克隆完整的AddrSpace (clone.rs:319-326)。设置：子进程共享父进程的AddrSpace Arc (clone.rs:317)
CLONE_FILES	文件描述符表	未设置：深度复制fd table，所有槽位的description 引用递增 (clone.rs:425-429)。设置：共享FD_TABLE scope (clone.rs:423)
CLONE_FS	文件系统上下文	未设置：深度复制cwd/root/umask (clone.rs:432-438)。设置：共享FS_CONTEXT scope (clone.rs:431)
CLONE_SIGHAND	信号处置	未设置且无CLONE_CLEAR_SIGHAND：深拷贝 disposition 表 (clone.rs:336)。设置：共享父进程的SignalActions Arc (clone.rs:332)
CLONE_THREAD	线程组归属	设置：加入父线程组并共享ProcessData (clone.rs:303-307)。未设置：新建Process/ProcessData (clone.rs:308-442)
CLONE_VFORK	父进程阻塞	设置：记录父TID 并在子任务生成后等待vfork_event (clone.rs:504,517-519)。未设置：父子并发运行
CLONE_PARENT	父子关系	设置：parent 取父进程的parent，即兄弟节点 (clone.rs:309-310)。未设置：当前进程作为parent (clone.rs:312)
CLONE_CHILD_CLEAR_TID	退出同步	记录clear_child_tid 用户空间地址 (clone.rs:469)。退出时向该地址写0 并futex wake (ops.rs:591-603)
CLONE_PARENT_SETTID	TID 返回	通过VM 写入立即将子进程的调度器TID 写入父进程的parent_tid 地址 (clone.rs:299-301)
CLONE_CHILD_SETTID	TID 返回	通过VM 写入将子进程的调度器TID 写入子进程的child_tid 地址 (clone.rs:462-467)
CLONE_SETTLS	TLS 寄存器	将子进程用户上下文的TLS 指针设置为tls 参数 (clone.rs:243)
CLONE_PIDFD	进程文件描述符	创建一个PidFd 对象（线程/进程变体），插入fd table，将fd 编号回写到父进程地址空间 (clone.rs:471-489)
CLONE_NEWNET	网络namespace	fork 一个新的仅loopback NetStack (clone.rs:340) vs. clone 父进程的 (clone.rs:342)
CLONE_NEWCGROUP	Cgroup namespace	fork 一个新的CgroupNamespace (clone.rs:345) vs. clone (clone.rs:347)
CLONE_NEWPID	PID namespace	fork 一个新namespace，父进程的PidNamespace 作为parent，子进程的TID 作为init_pid (clone.rs:350)

flag	控制资源	TheKernel 克隆时的处理
CLONE_NEWUSER	用户namespace	fork 一个新namespace，父进程的UserNamespace 作为parent，父进程的euid 作为owner_uid (clone.rs:355)
CLONE_NEWUTS	UTS namespace	fork 当前UTS 状态的深拷贝 (clone.rs:360) vs. clone 父进程的 (clone.rs:362)
CLONE_IO	I/O 上下文	共享父进程的io_context Arc (clone.rs:396) vs. 新Arc (clone.rs:398)
CLONE_SYSVSEM	SysV 信号量撤销	共享父进程的sysvsem_undo Arc (clone.rs:401) vs. 新Arc (clone.rs:403)
CLONE_CLEAR_SIGHAND	清除信号句柄	使用空的SignalActions 表初始化子进程 (clone.rs:334)
CLONE_INTO_CGROUP	Cgroup 放置	clone3 特有；子进程加入指定cgroup，覆盖默认的父进程cgroup 继承 (clone.rs:491-493)

4.5.7 调度状态继承

SchedState 的reset_on_fork 标志 (clone.rs:261-263) 控制子进程的调度类是否重置。Linux sched(7) 规定：fork(2) 生成的子进程继承父进程的调度策略与参数。但存在一个例外：如果SCHD_RESET_ON_FORK 标志在父进程上处于活动状态，子进程的调度类型被重置为SCHD_NORMAL。

TheKernel通过child_sched_state.reset_on_fork 检查后跟显式状态清零来实现此行为：对于实时类 (FIFO/RR)，子进程回退到Normal，nice=0。对于Deadline，子进程同样回退（因为EDF 调度器尚未完全实现）。在重置之后，reset_on_fork 自身被清除，因为该标志不传后代。

4.5.8 vfork：阻塞父线程的机制

vfork的语义要求父线程在子进程完成exec 或退出前停止继续执行。等待循环先检查vfork_in_progress()，避免已经完成的子进程让父线程无意义地睡眠；若条件仍未满足，则把当前waker 注册到vfork_event，注册后再检查一次以覆盖完成事件与注册动作之间的竞争窗口。只有两次检查都确认vfork 仍在进行时，父线程才返回Poll::Pending。

唤醒路径有两个出口：exec 成功提交新映像时调用release_vfork()，完整线程组退出时也调用同一函数。这样父线程恢复执行的时刻被固定在子进程不再共享原执行映像之后。等待循环见kernel/src/syscall/task/clone.rs:153-165。

4.6 execve：替换映像但保留进程

execve是TheKernel中最复杂的单系统调用之一，主体位于kernel/src/syscall/task/execve.rs。它在用全新的可执行映像替换地址空间、信号句柄和元数据的同时，必

须保留PID、父子关系及非CLOEXEC的文件描述符。这种“替换映像但保留进程身份”的语义，正是第4.1节将线程组级容器（ProcessData）与地址空间分离的直接收益。

4.6.1 支持和约束

TheKernel的exec入口先把用户指针转换成内核字符串与参数向量，再把路径解析结果交给同一个do_execve()。普通execve只依赖当前文件系统上下文解析路径；execveat额外接受目录fd与两个受支持的Linux标志：空路径执行和不跟随符号链接。参数和环境的大小约束集中在EXEC_MAX_ARGSTRLEN、EXEC_ARG_MAX与exec_arg_limit()中，既限制单个字符串，也为最终用户栈保留安全余量。边界常量见kernel/src/syscall/task/execve.rs:100-108，两个入口见kernel/src/syscall/task/execve.rs:337-389。

4.6.2 execve 处理顺序

do_execve()的顺序围绕一个提交边界组织。权限检查和ELF加载可以在旧地址空间仍然有效时失败返回；进入去线程化和地址空间替换后，内核必须保证调用线程成为新映像的主线程，已经被中断的兄弟线程也不能再回到旧用户空间继续运行。

1. 首先拒绝正在被写打开的可执行文件，并通过fanotify发送执行权限检查。这个阶段还没有创建新地址空间，失败时不会改变进程状态。
2. 接着创建空用户地址空间，复制内核页表部分，并调用load_user_app()装载ELF或脚本解释器。加载器返回新入口地址和用户栈地址，同时生成argv、envp与auxiliary vector的初始栈布局。
3. 可执行文件通过executable::acquire()持有，防止运行期间被unlink后提前回收；fanotify的普通打开与执行打开通知也在这里发出。
4. 若线程组中还有其他线程，调用线程先取得exec所有权，再中断所有兄弟线程并等待线程组收缩到只剩自己。等待期间若收到致命信号，内核会继续检查并处理该信号；若exec所有权丢失或进程开始退出，则释放可执行文件引用并返回中断错误。
5. 去线程化完成后，新地址空间被包装为进程共享对象，页表根安装到当前CPU，旧地址空间句柄被丢弃。这个点之后，进程身份还在，但用户映像已经换成新程序。
6. 提交阶段会清理membarrier注册和exec后保留capability的开关，把调用线程的可见TID重新绑定为进程PID，并替换可执行文件、任务名、exe_path和cmdline。LoongArch64分支采用显式replace路径，其他架构直接写入锁保护字段。
7. 新映像继承进程身份，但清理线程态残留：heap top重置，per-range NUMA策略清空，用户handler恢复默认，备用信号栈、clear-child-tid和robust-list入口清零；CLOEXEC fd关闭，其他fd保留。
8. 最后释放vfork父线程的阻塞，设置新入口指针和用户栈指针。如果进程正被ptrace追踪，exec成功后进入SIGTRAP停止点并唤醒tracer。

这条路径的源代码分布在`kernel/src/syscall/task/execve.rs:207-334`。真正的提交点是`replace_aspace()`和`set_current_user_page_table_root()`：在此之前，失败可以保留旧映像；在此之后，进程身份仍然存在，但用户态执行上下文已经指向新程序。

4.6.3 execve 的参数加载

`load_exec_args_env()`把两层用户指针转换成内核向量：先读取以`null`结束的指针数组，再逐个读取以`null`结束的字符串。每读入一个字符串，`ExecArgSizer`都同时计入字符串字节、结尾`null`和指针槽位大小，因此限制对应最终压入用户栈的真实负载。空`argv`会被补成一个空字符串，保证新程序仍能得到存在的`argv[0]`。实现见`kernel/src/syscall/task/execve.rs:126-205`。

4.7 exit、zombie 与 wait：正确的进程死亡

4.7.1 do_exit：完全退出路径

`do_exit()`同时服务`sys_exit`和`sys_exit_group`。它先完成线程级清理，再由`starry_process`判断这次退出是否让整个进程结束；只有最后一个线程离开时，才进入文件、IPC、zombie 和父进程通知等进程级清理。

1. 当前线程先处理用户态退出同步。若设置了`clear-child-tid`，内核尝试在对应用户地址写入0，并唤醒该地址上的一个`futex`等待者；写入失败不会阻止线程退出。随后`robust futex`链表被扫描，仍由该线程持有的锁会进入`owner-died`状态。
2. 线程CPU时间被结算到进程账本中，可见TID别名被移除，未完成的`exec`所有权被释放。然后`exit_thread()`更新线程组状态，并返回这是否是进程的最后一个线程。
3. 若线程组仍有其他线程存活，进程级资源保持有效，只执行最后的事件唤醒和退出标记。若这是最后一个线程，内核会在可确认地址空间没有被其他活跃进程共享时丢弃私有匿名页，以降低zombie等待期间的驻留内存。
4. 进程级退出先收集自身和子树使用量，然后释放可执行文件持有引用，关闭整个进程`fd table`，并显式丢弃关闭结果。这样父进程的`wait`观察到zombie之前，文件描述关闭带来的`writeback`、`unlink`和`socket`引用释放已经完成。
5. `ptrace`、System V 共享内存和POSIX 文件锁随后断开。`ptrace`清理会解除双向追踪关系并恢复被追踪任务，IPC与锁清理则移除以进程为`owner`的内核状态。
6. zombie发布发生在父进程通知之前。内核根据父进程对`SIGCHLD`的处置决定是否自动回收；非自动回收路径会发布包含`wait status`、使用量和`uid`的快照，再切换进程状态并把孤儿zombie移交给新的`reaper`。父进程的`child-exit`事件、`pidfd poll`事件和`vfork`释放都在这一阶段唤醒。
7. 所有退出路径最后都会唤醒`exec`等待者和线程级退出等待者。若调用来自`exit_`

group, 进程会被标记为 group-exited, 并向剩余线程发送内核 SIGKILL; 最终 set_exit() 让调度器停止再次调度当前任务。

完整退出路径位于 kernel/src/task/ops.rs:582-695。这一段最关键的顺序是: 线程私有清理先于进程级 zombie 发布, fd 关闭和锁/IPC 释放先于父进程 wait 唤醒。父进程因此不会在 wait 成功后再观察到子进程仍持有旧 fd 或旧锁。

4.7.2 sys_exit 与 sys_exit_group

两个系统调用入口只负责把用户传入的退出码转换成 Linux wait 状态编码, 并选择退出范围。sys_exit() 只退出当前线程; sys_exit_group() 让 do_exit() 在收尾阶段杀死线程组内的其余线程。入口代码见 kernel/src/syscall/task/exit.rs:1-13。

4.7.3 wait: PID 选择器与事件消费

TheKernel 的 wait 逻辑由 sys_waitpid() 和 sys_waitid() 两条入口承载, 但内部共享同一套候选收集、事件选择和事务性写回协议。

1. PID 选择器先被归一化为任意子进程、精确 PID 或进程组三种形式。这个转换覆盖了 waitpid 的 -1/0/正数/负数参数, 也覆盖了 waitid 的 P_ALL、P_PID 和 P_PGID。
2. 候选集合先来自进程树中的直接子进程, 再按 __WALL 与 __WCLONE 决定是否等待 clone child。ptrace tracee 也会被并入候选集合, 但这类候选只允许报告 stopped 或 continued 状态, 不允许被当前进程当作子进程 reap。
3. 事件选择有固定优先级: 可报告的 stopped 事件优先, 其次是调用者请求的 continued 事件, 最后才是可回收的 zombie 退出事件。这个顺序让作业控制和 ptrace 可观察状态不会被退出回收抢先吞掉。
4. 写回用户缓冲区采用声明-恢复协议。Stopped 和 continued 事件在写回前会先从进程状态中 claim; 如果写 exit_code、info 或 rusage 失败, 内核把已 claim 的状态恢复。Zombie 退出事件则只在写回成功后执行 reap(), 随后分离 cgroup 并把子进程使用量计入父进程账本。
5. 阻塞循环每次睡眠前都会重新检查候选事件。若设置 WNOHANG, 没有事件时立即返回 0; 若有待处理系统调用信号或任务中断, 则返回中断错误。注册 child_exit_event waker 后再次检查事件, 避免事件在检查与睡眠之间发生而丢失唤醒。
6. waitid 的 pidfd 分支把 fd 解析为 PidFd, 验证目标进程仍是调用者的子进程, 并复用同一套类型过滤。若 pidfd 自身是非阻塞的, 内核隐式加入 WNOHANG; 在没有显式 WNOHANG 且首次检查无事件时返回 EAGAIN, 用来区分“暂时无事件”和“没有可等待子进程”。
7. waitid 写出的 siginfo_t 由 Linux wait status 解码而来: 正常退出映射为 CLD_EXITED,

带core dump的信号终止映射为CLD_DUMPED, 其他信号终止映射为CLD_KILLED。填充函数只写SIGCHLD、code、pid、uid和status这些waitid需要的字段。

wait的源码分成几个紧密相连的片段: 选择器与事件类型见kernel/src/syscall/task/wait.rs:53-140, 候选集合和pidfd过滤见kernel/src/syscall/task/wait.rs:156-247, 事件选择和写回恢复见kernel/src/syscall/task/wait.rs:249-340, sys_waitpid()与sys_waitid()的阻塞循环分别见kernel/src/syscall/task/wait.rs:342-452和kernel/src/syscall/task/wait.rs:479-637。这些实现共同维护一个事务边界: wait状态只在用户态写回成功后被真正消费。

4.8 当前边界

ProcessData和Thread结构体现了在TheKernel中对Linux任务模型的相当完整的捕捉, 但已知差距仍然存在:

1. CLONE_NEWPID与CLONE_THREAD的组合以及其他罕见的flag对缺少明确的拒绝或语义验证。所有可能的CLONE_*组合语义尚未经过详尽测试。
2. CFS和RT队列在功能上可用, 但deadline调度器仍未实现完整EDF行为; 创建deadline任务会回退到普通调度类 (clone.rs:281-289)。
3. 在CLONE_VFORK下, 子进程对父进程地址空间的写入 (因为CLONE_VM被隐式设置) 如果子进程在exec前对堆或栈进行了更改, 可能产生副作用。此交互的完整缓解措施尚未实施。
4. ptrace框架 (process.rs:1369-1568) 支持基本的attach/detach/stop/continue操作, 但syscall enter/exit stop、exec trap、clone/fork/vfork event和seccomp event的完整链路仍需系统验证。
5. 实现的五个namespace提供了procfs namespace fd ABI的基本标识操作, 但未完全实现预期的在各自域中的隔离语义。例如, PidNamespace对来自外部namespace的PID提供重映射, 但PID分配和可见性在所有namespace中保持全局唯一。
6. CapabilityState结构体提供完整的位集存储, 但capability检查覆盖仍不完整; 部分特权路径仍只检查uid == 0。

语义维度	ProcessData：线程组共享	Thread：线程局部
身份与资源	保存进程对象、可执行文件路径、命令行、地址空间句柄、fd/fs scope、堆顶和资源限制。	保存当前线程的visible tid，并通过Arc<ProcessData> 回到共享进程状态。
安全控制面	凭证、capability、supplementary groups、personality、no_new_privs 和seccomp mode 都属于进程级快照。	线程不复制这组安全状态；clone/exec/prctl 都通过共享对象看到同一份控制面。
信号与同步	进程pending signal、信号处置、进程私有futex table 和生命周期事件由共享对象承载。	线程pending signal、clear_child_tid、robust list 和restart tracker 只影响当前线程返回路径。
命名空间	cgroup、pid、user、uts、time 和network namespace 记录在线程组共享状态里。	线程只在进入syscall 或返回用户态时读取这些命名空间，不拥有独立副本。
生命周期	exit、exec、vfork、wait 和job-control 事件跨线程协同，因此放在ProcessData。	线程保存退出标志、状态提示、用户内存访问标记和本线程CPU 用时。
Linux 可见效果	fork/clone 决定哪些共享对象被继承或复制；exec 更新进程级映像并协调全组线程。	signal delivery、restart、set_tid_address 和robust futex 清理按照线程粒度生效。

图 4.1 ProcessData 与 Thread 的分离设计：进程共享vs 线程局部状态

功能域	共享状态与作用
身份与映像	proc、exe_path、cmdline 记录线程组身份、当前可执行文件和用户态参数快照。
地址空间与资源	aspace_handle、scope、heap_top、rlim 决定VMA、fd/fs scope、堆顶和资源限制。
凭证与安全	creds、caps、supplementary_groups、personality、no_new_privs、seccomp_mode 组成进程级安全控制面。
信号与同步	signal 和futex_table 分别承载进程pending signal、信号处置协同和进程私有futex key。
生命周期事件	child_exit_event、exit_event、exec_event、stop_event、vfork_event 连接wait、exec、job control 和vfork。
控制面	job_ctl、ptrace_ctl、exec_ctl、vfork_ctl 保存跨syscall 的控制状态。
命名空间	cgroup_ns、pid_ns、user_ns、uts_ns、time_ns、net_ns 给clone/exec/procfs 提供共享边界。
统计	posix_timers、exited_threads_usage、waited_children_usage、maxrss_kb 支撑timer、wait4 和资源统计。

所有字段通过Arc<ProcessData> 在线程组内共享；每个Thread 只保存线程局部状态并反向引用该共享对象。

图 4.2 ProcessData 字段分类全景：按功能域分组展示全部字段

起点	触发	终点	内核含义
Created	clone/fork 完成注册	Runnable	任务已经拥有pid、线程组关系和初始上下文，可以进入run queue。
Runnable	scheduler 选中	Running	线程取得CPU，用户态上下文从保存的trap frame 恢复。
Running	yield 或抢占	Runnable	当前线程仍可运行，只是主动让出或被时钟中断切回队列。
Running	futex、poll、sleep 等阻塞点	Sleeping	线程状态切到interruptible 等待，并把唤醒条件交给对应等待队列。
Sleeping	wake、timeout 或 signal	Runnable	等待队列删除waiter，恢复可运行状态；signal 会影响后续restart 或返回值。
Runnable	SIGSTOP 或 ptrace stop	Stopped	job control 暂停线程，调度器不会继续选择该任务运行。
Stopped	SIGCONT	Runnable	job control 解除暂停，线程重新回到可调度集合。
Running/Sleeping	exit 或 fatal signal	Zombie	线程退出并发布wait 可见的退出状态，资源回收等待父进程或线程组处理。
Zombie	wait/reap	Dead	父进程消费退出状态后，pid 和剩余引用可以完成释放。

图 4.3 任务生命周期状态机

第五章 地址空间与内存管理：虚拟内存的后端抽象

5.1 问题：不同映射需要不同填页方式

Linux用户态的mmap不只是“分配一段虚拟地址”。同一个系统调用会创建多类语义完全不同的VMA：

1. **私有匿名映射**，即MAP_PRIVATE|MAP_ANONYMOUS。它服务于堆、栈和malloc，首次访问分配清零页，fork后通过COW共享，写入时再私有化。
2. **私有文件映射**，即MAP_PRIVATE加文件fd。它常用于ELF段和动态库，首次读从文件填页，写入只影响当前进程的私有副本。
3. **共享文件映射**，即MAP_SHARED加文件fd。多个进程共享页缓存中的同一文件页，脏页由msync或页缓存回写路径同步到文件。
4. **共享匿名映射**，即MAP_SHARED|MAP_ANONYMOUS。它用共享页池表达跨进程共享内存，使参与映射的进程取得同一组物理页。
5. **设备或物理地址映射**。它把用户虚拟地址线性映射到设备或物理内存窗口，页表在map阶段即可建立。

这些映射的共同点是都有VMA与页表；差异集中在四个问题：页从哪里来、写入后是否私有化、fork时复制还是共享、回写或驱逐时如何与外部对象同步。若把这些判断散落在mmap、fork、page fault、msync等路径中，每新增一种映射都会扩大多个系统调用的分支面。

TheKernel的解法是把映射差异收敛到Backend。每个VMA仍由MemorySet管理区间和权限，但具体填页、克隆、回写、权限校验由backend决定。地址空间机制由AddrSpace统合，并沿page fault、fork、四类backend和相关系统调用展开。

5.2 AddrSpace：地址空间的状态边界

代码5.1 AddrSpace 核心状态

```
26     Handled,  
27     SigBus,  
28     Unhandled,  
29 }  
30
```

```

31  /// The virtual memory address space.
32  pub struct AddrSpace {
33      va_range: VirtAddrRange,
34      areas: MemorySet<Backend>,
35      growdown_starts: BTreeSet<VirtAddr>,
36      wipe_on_fork_ranges: BTreeMap<VirtAddr, VirtAddr>,
37      dontfork_ranges: BTreeMap<VirtAddr, VirtAddr>,
38      locked_ranges: BTreeMap<VirtAddr, VirtAddr>,
39      lock_future_mappings: bool,
40      lock_future_on_fault: bool,
41      pt: PageTable,
42  }

```

5.2.1 AddrSpace 的状态分层

AddrSpace 是进程虚拟内存语义的聚合点。结构体中的字段分为三层：

1. **地址边界与页表。**va_range 定义该地址空间允许出现的虚拟地址范围，pt 持有真实页表。所有map、unmap、protect 和 fault 处理最终都会落到这两者上。
2. **VMA 主集合。**areas:MemorySet<Backend> 保存连续虚拟区间、权限与backend 的绑定关系。查找VMA、寻找空洞、拆分区间、修改权限都从这里开始；backend 则回答“这个VMA 的页应如何产生”。
3. **Linux VMA 附加语义。**growdown_starts、wipe_on_fork_ranges、dontfork_ranges、locked_ranges 与两个future-lock 标志是mmap、madvise、fork、mlock 共同可见的地址区间属性，可以作用在任意backend 上。

这种拆分避免把所有语义塞进VMA flags。比如MADV_DONTFORK 和MADV_WIPEONFORK 都会影响fork,但它们可以作用在任意backend 上;mlockall(MCL_FUTURE) 影响未来的mmap 和brk, 也不能只绑定到某个现有VMA。

5.2.2 区间集合的复用逻辑

wipe_on_fork_ranges、dontfork_ranges 与locked_ranges 都保存半开地址区间,起点作为键, 终点作为值。它们共享同一套区间算法：

1. insert_interval() 合并所有与新区间重叠或相邻的旧区间，再插入合并后的边界。
2. clear_interval() 从已有区间中剪掉目标范围；被截断的前后两段会重新插回集合。
3. interval_end_covering() 与next_interval_start() 让fork 主循环能在dontfork、wipeonfork 与普通COW 段之间找到下一段边界。
4. interval_overlaps() 用于判断一段范围是否被某类属性覆盖。

这些区间工具由try_clone()、madvise 和mlock 路径共同使用；这也是三个区间集合没有各自复制一套算法的原因。合并、裁剪和覆盖查询的实现位于kernel/src/mm/aspace/mod.rs:113-185。

5.2.3 栈向下增长状态

`growdown_starts` 只记录带 `MAP_GROWSDOWN` 语义的 VMA 起点。栈向下扩展时，VMA 起点会变化，因此 `refresh_growdown_starts()` 会在 `unmap`、`protect` 等操作后清洗失效起点，`move_growdown_start()` 则在 `fault` 扩栈成功后更新该集合。

StarryOS baseline 的地址空间结构只有 `va_range`、`areas` 与 `pt`，没有 `growdown` 状态；TheKernel 增加 `STACK_GUARD_GAP_PAGES=256` 作为 `guard` 区上限，并在 `page fault fallback` 中检查用户栈指针，使栈只能在符合 Linux 语义的访问路径上按页向下增长。

5.3 Page Fault 与fork 的主路径

5.3.1 Page Fault 处理

`handle_page_fault_result()` 把硬件页错误转换成三个内核可处理的结果：`Handled` 表示 `fault` 已被填页解决，`SigBus` 表示文件映射越界应向用户态发送 `SIGBUS`，`Unhandled` 表示应继续走普通缺页异常路径并最终 `SIGSEGV`。

这段代码的控制流按顺序推进：

1. 先拒绝地址空间范围外的 `fault`。
2. 在 `areas` 中查找覆盖 `fault` 地址的 VMA；没有 VMA 时转入 `growdown fallback`。
3. 检查 VMA 权限是否覆盖本次访问权限。
4. 交给 `backend` 判断是否应 `SIGBUS`，典型场景是文件映射访问到 EOF 之后。
5. 计算 `fault-around` 范围。匿名 COW 写 `fault` 会一次填 4 页，其余路径按 `backend` 页大小填一页。
6. 调用 `backend.populate()`，并在返回后执行可选的 `PopulateCallback`。FileBackend 用这个回调延后处理页缓存驱逐，避免在页缓存锁内重新拿地址空间锁。
7. 若没有填入任何页或 `populate` 出错，则返回 `Unhandled`；否则返回 `Handled`。

页错误主处理函数位于 `kernel/src/mm/aspace/mod.rs:883-931`。内核访问用户指针时的 `trap` 入口在 `kernel/src/mm/access.rs`，它只在 `access_user_memory()` 保护的上下文中尝试补页。这个边界保证普通内核地址错误不会被误当成用户缺页。

5.3.2 fork 时的地址空间克隆

`try_clone()` 创建一个空的子地址空间，再逐段复制父进程的 VMA。这里没有把 `MADV_DONTFORK` 和 `MADV_WIPEONFORK` 做成 `backend` 行为，因为它们是 VMA 级元属性，和 `Cow`、`File`、`Shared`、`Linear` 后端正交。

每个 VMA 内部按页大小推进，决策顺序为：

1. **dontfork 覆盖段。**当前地址落在`dontfork_ranges`时，直接跳到该属性区间的结束地址，不在子进程建立映射。
2. **wipeonfork 覆盖段。**当前地址落在`wipe_on_fork_ranges`时，子进程得到新的匿名COW后端，内容清零，同时继承`wipeonfork`标记以便再次fork。
3. **普通段。**先找到下一个`dontfork`或`wipeonfork`边界，再调用`area.backend().clone_map()`。COW后端会降级父页权限并共享物理帧；Shared和File后端保持共享对象；Linear后端继承固定物理映射。

遍历完成后，子地址空间刷新`growdown_starts`。这样，fork之后子进程既继承动态栈增长能力，也不会保留已被`dontfork`过滤掉的临时映射。主循环位于`kernel/src/mm/aspace/mod.rs:947-1025`；各后端通过`clone_map()`执行自己的共享、复制或线性继承规则。

5.3.3 栈自动增长

`try_handle_growdown_fault()`只在普通VMA查找失败后触发。它通过一组条件防止随机地址访问伪装成栈增长：

1. 必须能取得用户栈指针`user_sp`。
2. `fault`页必须正好位于当前`growdown` VMA起点的下一页下方。
3. 用户栈指针必须位于`fault`页和当前VMA起点之间。
4. VMA权限必须覆盖本次`fault`权限。
5. 只有COW后端允许动态扩栈；文件、共享匿名和线性物理映射都不会被`growdown`规则扩展。
6. 扩展前检查256页`guard`区内没有其他VMA，避免栈越过已有映射。

扩展成功后，内核在`fault`页创建新的匿名COW映射，移动`growdown`起点，再递归调用`handle_page_fault_result()`给新VMA填页。递归的原因是扩栈只创建了VMA；真正物理页仍由backend的`populate`路径负责。完整判断位于`kernel/src/mm/aspace/mod.rs:772-841`。

5.4 Backend 抽象：把填页方式编码为trait

5.4.1 BackendOps 统一接口

代码5.2 BackendOps 统一接口

73 `#[enum_dispatch]`

```

74 pub trait BackendOps {
75     /// Returns the page size of the backend.
76     fn page_size(&self) -> PageSize;
77
78     /// Map a memory region.
79     fn map(&self, range: VirtAddrRange, flags: MappingFlags, pt: &mut PageTableCursor) -> AxResult;
80
81     /// Unmap a memory region.
82     fn unmap(&self, range: VirtAddrRange, pt: &mut PageTableCursor) -> AxResult;
83
84     /// Called before a memory region is protected.
85     fn on_protect(
86         &self,
87         _range: VirtAddrRange,
88         _new_flags: MappingFlags,
89         _pt: &mut PageTableCursor,
90     ) -> AxResult {
91         Ok(())
92     }
93
94     /// Populate a memory region and return how many pages now satisfy
95     /// `access_flags`.
96     ///
97     /// If another thread has already mapped the page with sufficient permissions,
98     /// treat it as populated.
99     fn populate(
100         &self,
101         _range: VirtAddrRange,
102         _flags: MappingFlags,
103         _access_flags: MappingFlags,
104         _pt: &mut PageTableCursor,
105     ) -> AxResult<(usize, Option<PopulateCallback>)> {
106         Ok((0, None))
107     }
108
109     /// Duplicates this mapping for use in a different page table.
110     ///
111     /// This differs from `clone`, which is designed for splitting a mapping
112     /// within the same table.
113     ///
114     /// [`BackendOps::map`] will be latter called to the returned backend.
115     fn clone_map(
116         &self,
117         range: VirtAddrRange,
118         flags: MappingFlags,
119         old_pt: &mut PageTableCursor,
120         new_pt: &mut PageTableCursor,
121         new_aspace: &Arc<Mutex<AddrSpace>>,
122     ) -> AxResult<Backend>;
123 }

```

BackendOps 通过 `enum_dispatch` 做静态分发。调用方只持有 Backend enum，编译器把方法解析到具体实现，避免在 page fault、fork、mprotect 等路径反复写 backend 类型分支。这组接口分成六个入口：

1. `page_size()` 返回 backend 的页粒度。COW 和 File 当前以 4K 为主，类型系统仍保留 2M 与 1G 的大页表达。
2. `map()` 在 VMA 建立时执行后端校验。COW 后端保持惰性，Linear 后端立即建页表，

File 和 Shared 后端主要检查权限与区间。

- 3. `unmap()` 释放页表映射与后端资源。COW 后端需要按全局引用计数决定是否释放物理帧。
- 4. `on_protect()` 是 `mprotect` 的前置钩子。FileBackend 用文件打开模式校验写权限，SharedBackend 用 `may_protect` 限制最大权限。
- 5. `populate()` 执行真正填页，并返回已满足访问权限的页数和可选事后回调。
- 6. `clone_map()` 为 fork 准备子地址空间中的 backend，并在需要时更新父子页表和引用计数。

Backend trait 定义在 `kernel/src/mm/aspace/backend/mod.rs`。四个实现分别位于 `cow.rs`、`file.rs`、`shared.rs` 与 `linear.rs`。

5.4.2 四类Backend 的语义边界

Backend	Linux语义	填页来源	fork 行为
CowBackend	私有匿名与私有文件映射	匿名映射分配清零帧；文件映射从 <code>CachedFile</code> 读取	降低父页写权限，父子只读共享，写 <code>fault</code> 时分裂
FileBackend	共享文件映射	从文件页缓存取得物理页	克隆页缓存句柄与驱逐监听器
SharedBackend	共享匿名映射	从 <code>SharedPages</code> 页池取得物理页	克隆同一个页池的 <code>Arc</code>
LinearBackend	设备或物理地址直映射	按固定 VA 到 PA 偏移建立页表	克隆同一物理窗口映射

5.5 COW：私有映射的核心后端

5.5.1 CowBackend 状态

代码5.3 CowBackend 状态

```
96  #[derive(Clone)]
97  pub struct CowBackend {
98      start: VirtAddr,
99      size: PageSize,
100     file: Option<(CachedFile, u64, Option<u64>, bool)>,
101     map_id: Arc<()>,
102     materialized: Arc<AtomicBool>,
103 }
```

这五个字段围绕COW 生命周期分工：

1. `start` 与 `size` 定义虚拟地址到文件偏移或页大小的计算基准。
2. `file` 决定该后端是匿名页还是文件私有映射。文件映射还保存 `file start`、`file end` 与 `EOF` 后是否 `SIGBUS` 的策略。
3. `map_id` 与 `materialized` 服务于 VMA 合并、拆分和释放。相同 `map_id` 说明两个后端来自同一次映射序列；`materialized` 让 `unmap` 能跳过从未分配过物理页的惰性区间。

5.5.2 全局物理页引用计数

COW 正确性的基础是 `FRAME_TABLE`。它把物理地址映射到受 `SpinNoIrq` 保护的引用计数，并用 `Arc` 让多个地址空间共享同一计数对象。

释放顺序是这段代码的关键：当计数降到 0 时，必须先从 `FRAME_TABLE` 删除条目，再释放物理帧。否则同一个物理地址可能被分配给新帧，而表中仍残留旧条目，后续 `COW fault` 会把新帧误认为旧共享帧。引用计数表与释放规则位于 `kernel/src/mm/aspace/backend/cow.rs:23-79`。

5.5.3 fork 时的 COW 克隆

`CowBackend::clone_map()` 负责把父进程已物化页面变成父子只读共享页。它的流程分三步：

1. 对可写的文件私有映射，先在父进程中补齐未物化页。这样子进程看到 `fork` 时父进程的私有数据快照，避免 `fork` 后再从文件重新读取。
2. 扫描父页表中已 `present` 的页，并在 `FRAME_TABLE` 中取得或创建引用计数对象。
3. 对每个 `present` 页递增引用计数；若父页有写权限，则先降为只读，再把同一物理帧以只读权限映射到子页表。若子映射失败，恢复父页权限并回滚计数。

因此，`fork` 本身不复制页面内容，只复制“未来写入时需要分裂”的条件。`COW` 克隆路径位于 `kernel/src/mm/aspace/backend/cow.rs:428-489`。

5.5.4 写 fault 的分裂决策

`handle_cow_fault()` 在写入只读 `COW` 页时运行。它按引用计数分三种情况：

1. 物理帧不在 `FRAME_TABLE` 中，说明该页不属于 `COW` 追踪的共享帧；直接恢复写权限。
2. 引用计数为 1，说明当前地址空间已经独占该帧；无需复制，只恢复写权限。
3. 引用计数大于 1，说明仍有其他地址空间共享该帧；分配新帧、复制旧内容、`remap` 当前虚拟页，并递减旧帧引用计数。

所有路径最后都会标记 `materialized`，使后续 `unmap` 知道该后端可能持有实际物理页。写 `fault` 分裂逻辑位于 `kernel/src/mm/aspace/backend/cow.rs:174-213`。

5.5.5 文件私有映射的首次填页

`alloc_new_at()` 负责私有映射的首次缺页。匿名映射分配清零帧；文件映射跳过全页预清零，因为文件读取会覆盖有效内容，只需要清理页内前缀和文件尾部之后的空洞。

这也是ELF段能正确表达BSS的原因：文件中存在的部分从CachedFile读入，`file_size < mem_size`的尾部由后端清零，不会被判为SIGBUS。首次填页路径位于`kernel/src/mm/aspace/backend/cow.rs:132-172`。

5.5.6 EOF、驻留与fault-around

`faults_with_sigbus()`、`cached_page_resident()` 和 `fault_around_size()` 是COW后端暴露给上层的三个查询能力。前者让page fault handler在文件EOF之后返回SIGBUS；第二个供`mincore()`和进程统计判断文件页是否驻留；第三个给匿名写fault做4页fault-around，降低连续堆栈写入的缺页频率。

COW后端的EOF、驻留与fault-around查询位于`kernel/src/mm/aspace/backend/cow.rs`；`mremap`迁移已物化页的辅助函数是`clone_materialized_pages()`。

5.6 FileBackend：共享文件映射与页缓存生命周期

5.6.1 文件映射状态

代码5.4 文件映射状态

```

54  #[doc(hidden)]
55  pub struct FileBackendInner {
56      start: VirtAddr,
57      cache: CachedFile,
58      flags: FileFlags,
59      offset_page: u32,
60      file_end: Option<u64>,
61      handle: AtomicUsize,
62      map_id: Arc<()>,
63      futex_handle: Arc<()>,
64      writable_mapping: Option<Arc<memfd::WritableMappingRegistration>>,
65  }
66  impl Drop for FileBackendInner {
67      fn drop(&mut self) {
68          let handle = self.handle.load(Ordering::Acquire);
69          if handle != 0 {
70              unsafe {
71                  self.cache.remove_evict_listener(handle);
72              }
73          }
74      }
75  }

```

FileBackend 把共享文件映射接入页缓存、futex 与 memfd 写密封。字段分为四组：

1. `start`、`offset_page` 和 `file_end` 用于虚拟地址到文件页号的转换，以及文件截断后的EOF 检查。
2. `cache` 与 `flags` 连接文件页缓存和打开权限，`mmap` 与 `mprotect` 都通过它检查读写权限。
3. `handle` 保存页缓存驱逐监听器注册句柄，Drop 时注销。
4. `map_id`、`futex_handle` 与 `writable_mapping` 分别服务于VMA 合并、文件futex key 复用和memfd write-seal 检查。

5.6.2 页缓存驱逐回调

共享文件映射必须处理页缓存驱逐。页缓存释放某个文件页前，进程页表中不能继续保留指向该物理页的PTE。

驱逐监听器执行三步：

1. 通过弱引用确认地址空间和FileBackend 仍然存在。
2. 尝试获取地址空间锁；若锁正在被populate 路径持有，则跳过，由populate 返回的回调统一处理。
3. 计算被驱逐文件页对应的虚拟地址，确认该VMA 仍属于当前FileBackend，再取消页表映射。

StarryOS baseline 已有基础驱逐监听器；TheKernel 在此基础上增加动态EOF、direct I/O 排除、memfd 可写共享映射跟踪和更完整的文件futex key 复用。驱逐回调位于`kernel/src/mm/aspace/backend/file.rs:77-123`。

5.6.3 populate: 页缓存与页表协调

FileBackend的`populate()` 在页缓存和页表之间保持一致性。对范围内每页，它按页表现状分三种情况：

1. 页表已有映射但本次需要写权限时，标记页缓存页为dirty，并remap 升级页表权限。
2. 页表已有映射且权限足够时，直接计入已填页数量。
3. 页表未映射时，通过`with_page_or_insert()` 从页缓存取得或插入页面，再建立页表映射。若插入导致其他页被驱逐，记录页号并构造`PopulateCallback`。

回调延迟到populate 返回后执行，是为了避免页缓存锁与地址空间锁形成嵌套死锁。填页主体位于`kernel/src/mm/aspace/backend/file.rs:317-379`。

5.6.4 文件futex与动态EOF

`file_futex_handle()` 以设备号和inode为键缓存futex句柄，使多个进程映射同一个文件时能在同一个futex key上等待和唤醒。`faults_with_sigbus()` 则查询当前文件长度，覆盖mmap时记录的初始长度，因此文件被截断后，越界访问会正确变成SIGBUS。

FileBackend的futex key、EOF检查和populate主体位于`kernel/src/mm/aspace/backend/file.rs`。

5.7 SharedBackend：共享匿名页池

5.7.1 SharedPages 页池

SharedPages是共享匿名映射的物理页池。它用`Mutex<Vec<PhysAddr>>`保存物理帧，多个进程或线程fault到同一shared mapping时会取得同一组帧。

`ensure_len()`采用双检查：先在锁外准备新增帧，再加锁确认是否仍需要扩展。如果其他线程已经完成扩展，新分配的帧会立即释放。这样既避免长时间持锁分配，也避免重复扩容。页池实现位于`kernel/src/mm/aspace/backend/shared.rs:13-67`。

5.7.2 SharedBackend 状态与权限

SharedBackend只把地址空间中的一段VMA连接到共享页池。它保存映射起点、`Arc<SharedPages>`、允许被`mprotect`提升到的最大权限`may_protect`，以及用于判断映射同源性的`map_id`。

fork时SharedBackend直接clone自身。关键点在`pages`字段：它是`Arc<SharedPages>`，clone只增加引用计数，不复制页池。这与COW的“先共享、写时分裂”不同；共享匿名映射的语义就是父子进程继续看到同一组物理页。状态和克隆路径分别位于`kernel/src/mm/aspace/backend/shared.rs:144-200`与`kernel/src/mm/aspace/backend/shared.rs:276-285`。

5.7.3 SharedBackend 填页

SharedBackend的populate先把虚拟地址转换为页池索引，再取出对应物理帧。之后的页表处理与FileBackend类似：已有映射但权限不足则remap，已有且权限足够则跳过，未映射则map到共享物理帧。填页路径位于`kernel/src/mm/aspace/backend/shared.rs:239-274`。

5.8 LinearBackend：物理地址直映射

LinearBackend表示固定VA到PA偏移的映射。它适用于设备寄存器、物理内存窗口和其他不需要按fault分配物理页的区域。

`map()` 是四类 backend 中唯一在映射建立阶段直接写页表的实现。原因是线性映射的物理地址在创建时已经确定，运行期不会因为缺页而改变。`fork` 时 `clone_map()` 直接 clone 后端，子进程继承同一物理窗口。状态与映射逻辑位于 `kernel/src/mm/aspace/backend/linear.rs:13-128`。

5.9 mmap、munmap 与 mprotect

5.9.1 mmap 的参数校验与地址选择

`sys_mmap()` 的前半段先把用户参数转换成内核可处理的映射请求：

1. 解析 `prot` 和 `flags`，拒绝无效的映射类型；对 `MAP_SHARED_VALIDATE` 保留 Linux 的未知 flag 报错语义。
2. 校验匿名映射的 `offset` 必须为 0，非匿名映射必须带有效 `fd`，长度不能为 0，文件 `offset` 必须按 4K 对齐。
3. 根据 `huge-page flag` 选择页大小：1G、2M 或默认 4K。
4. 将地址和长度按页大小对齐；匿名可写映射先做 `overcommit` 检查。
5. `MAP_FIXED` 先清理目标区间；非固定映射调用 `find_kernel_area()`，优先尝试追加到现有映射之后，再回退到 `first-fit`。

参数校验和地址选择位于 `kernel/src/syscall/mm/mmap.rs:352-439`。这一阶段还没有把 VMA 插入地址空间，因此大多数错误都可以在没有副作用的情况下返回。

5.9.2 mmap 的 Backend 选择

后端选择遵循映射类型和文件来源：

1. `MAP_SHARED` 加普通文件使用 `FileBackend`；加 `io_uring` 或设备匿名映射使用 `SharedBackend`；加设备物理映射使用 `LinearBackend`。
2. `MAP_SHARED|MAP_ANONYMOUS` 创建新的 `SharedPages` 页池。
3. `MAP_PRIVATE` 加普通文件使用 `CowBackend`，设备物理映射使用 `LinearBackend`。
4. `MAP_PRIVATE|MAP_ANONYMOUS` 使用匿名 `CowBackend`。
5. `MAP_LOCKED` 或 `MCL_FUTURE` 会触发 `RLIMIT_MEMLOCK` 检查；`MAP_POPULATE`、锁定映射或 `future-lock` 非 `on-fault` 模式会立即 `populate`。
6. 私有匿名 `MAP_GROWSDOWN` 会调用 `mark_growdown()`，使后续 `fault fallback` 能识别动态栈扩展。

后端选择与锁定处理位于 `kernel/src/syscall/mm/mmap.rs:456-602`。提交点是 `aspace.map()`：在此之前只是参数、fd、权限和资源限制检查；提交后，VMA、backend 和必要的预填页状态才进入地址空间。

5.9.3 munmap 与 mprotect

`sys_munmap()` 验证地址页对齐和长度，再调用 `aspace.unmap()`。`unmap` 不只清理页表和 VMA，也会清除 `wipeonfork`、`dontfork`、`locked` 区间，并清除对应 `mempolicy`。

`sys_mprotect()` 拒绝 `PROT_GROWSDOWN` 和 `PROT_GROWSUP`，再把 `MmapProt` 转成 `MappingFlags` 调用 `aspace.protect()`。每个 backend 的 `on_protect()` 会在这个路径中检查自己的权限约束。

`sys_munmap()`、`sys_mprotect()` 与权限转换逻辑位于 `kernel/src/syscall/mm/mmap.rs`；VMA 清理与 backend 钩子在 `kernel/src/mm/aspace/mod.rs` 和各 backend 实现中完成。

5.10 mremap：旧地址段迁移

`mremap` 的实现分两类模式。

1. **共享扩展模式**，即 `old_size==0`。内核收集旧地址处的 segments，要求所有 backend 都可共享，再在目标地址建立一组 `relocated segments`。该模式不保留原映射 `identity`。
2. **完整扩缩容模式**，即 `old_size>0`。缩小时 `unmap` 尾部；可原地扩展时补齐尾部 backend；不能原地扩展但允许移动时，选择新地址、迁移保留段、补齐新尾部、清理旧地址。

锁定状态贯穿整个迁移流程。`locked_segments_for_remap()` 记录被保留部分的锁定区间，`set_relocated_locked_segments()` 在新地址恢复；扩容时还要检查新增锁定字节是否超过 `RLIMIT_MEMLOCK`。

`mremap` 主体在 `kernel/src/syscall/mm/mmap.rs`。StarryOS baseline 中该函数仍是 TODO 风格的“新建私有匿名映射、复制字节、`unmap` 旧段”，TheKernel 则保留 backend `identity`、共享属性和锁定状态。

5.11 brk：堆的扩展与收缩

`sys_brk()` 管理进程堆顶，遵循 Linux 的“失败时返回旧 brk”风格；`errno` 只用于参数非法等前置错误。流程分为三段：

1. **查询**。`addr==0` 时直接返回当前堆顶。
2. **扩展**。新堆顶向上对齐，扩展范围从初始堆末尾和当前堆顶中较大者开始。扩展前检查 `overcommit`、与非堆 VMA 的碰撞，以及 `future-lock` 对 `RLIMIT_MEMLOCK` 的要求；随

后创建匿名COW映射，必要时立即populate。

3. **收缩**。只释放初始堆之后的页，避免破坏ELF loader已建立的初始堆区域；收缩后清除mempolicy并更新堆顶。

初始堆由loader以populate=true建立，确保程序启动后的基础堆立即可用；brk后续扩展则保持惰性，首次访问时由page fault填页。实现位于kernel/src/syscall/mm/brk.rs:13-102。

5.12 madvise: 细粒度内存提示

sys_madvise()把advice分成四类处理：

1. **纯提示**。MADV_NORMAL、MADV_RANDOM、MADV_SEQUENTIAL、MADV_WILLNEED等只验证范围有效，不改变内核状态。
2. **fork行为控制**。MADV_DONTFORK、MADV_DOFORK操作dontfork区间；MADV_WIPEONFORK只允许作用于私有匿名映射；MADV_KEEPONFORK清除wipeonfork区间。
3. **主动填充**。MADV_POPULATE_READ和MADV_POPULATE_WRITE直接调用populate_area()。
4. **内存释放**。MADV_DONTNEED丢弃物理页但保留VMA，拒绝shared mapping和locked range；MADV_FREE只允许私有匿名映射。

这些advice最终都落到前文的区间集合和backend populate/unmap机制上，madvise入口只负责解析Linux advice并调用共享机制。分支实现位于kernel/src/syscall/mm/mmap.rs:844-923。

5.13 mlock与mlockall: 内存锁定层级

内存锁定由三层状态共同表达：

1. locked_ranges记录已锁定VMA范围，供mlock、munlock、mremap和统计函数查询。
2. lock_future_mappings表示mlockall(MCL_FUTURE)，后续mmap与brk扩展会自动带锁定属性。
3. lock_future_on_fault表示MCL_ONFAULT，未来映射只记录锁定状态，不在创建时立即populate。

sys_mlock2()先验证范围可访问，再计算新增锁定字节是否超过RLIMIT_MEMLOCK；没有MLOCK_ONFAULT时立即填页，然后登记locked range。sys_mlockall()对MCL_CURRENT

遍历当前VMA，对MCL_FUTURE 设置future 标志；`sys_munlockall()` 则清空所有锁定状态。入口实现位于`kernel/src/syscall/mm/mmap.rs:978-1137`。

锁定字节统计函数位于`kernel/src/mm/aspace/mod.rs`，包括`locked_bytes()`、`locked_bytes_in_range()`、`current_mapping_bytes()` 与 `resident_user_bytes()`。

5.14 ELF Loader：从二进制到进程地址空间

5.14.1 ELF 缓存与段映射

`ElfLoader` 使用 `LRUCache<ElfCacheEntry,32>` 缓存最近解析的ELF 文件。每个 `ElfCacheEntry` 通过 `ouroboros::self_referencing` 把原始字节和解析后的ELF 头保存在同一结构中，避免重复读取和重复解析常见二进制及动态链接器。

`map_elf()` 为每个PT_LOAD segment 创建 `CowBackend`。它保留非页对齐的文件偏移，把起始地址向下对齐到4K，再把 `file_size<mem_size` 的尾部交给COW 文件填页逻辑清零。ELF 映射把 `sigbus_on_eof` 设为 `false`，因此BSS 空洞不会被错误解释成EOF SIGBUS。

段映射位于`kernel/src/mm/loader.rs:87-137`。`loader` 只建立VMA 和 `backend`，实际物理页仍在首次访问或显式 `populate` 时进入页表；这让ELF 装载和普通私有文件映射复用同一套COW 文件填页规则。

5.14.2 动态链接器、栈与堆

`ElfLoader::load()` 先解析主ELF，查找PT_INTERP。若存在动态链接器，则把解释器放到 `USER_INTERP_BASE`，主程序放到 `USER_SPACE_BASE`，入口点选择解释器 `entry`；否则直接使用主程序 `entry`。辅助向量中补入 `AT_UID`、`AT_EUID`、`AT_GID`、`AT_EGID` 与 `AT_SECURE`，其中 `AT_SECURE` 由真实/有效 `uid` 或 `gid` 是否不同决定。

ELF 映射完成后，`loader` 建立初始栈和堆。栈底保留一页 `guard page`，顶部用于写入 `argv`、`envp` 和 `auxv`；写栈前显式 `populate` 对应范围，确保用户程序进入前栈内容已经可访问。堆从 `USER_HEAP_BASE` 开始，以 `populate=true` 创建初始区域，之后交给 `brk()` 扩展。动态链接器和辅助向量构造位于`kernel/src/mm/loader.rs:187-280`，用户程序初始化位于`kernel/src/mm/loader.rs:359-454`。

5.14.3 Shebang 与脚本回退

`load_user_app()` 在ELF 解析失败后检查 `#!`。若存在 `shebang`，它解析解释器路径并重组参数递归加载；若解释器失败且脚本以 `.sh` 结尾，则按 `busybox` 和 `/bin/sh` 等候选解释器回退。无 `shebang` 且不符合脚本回退条件的文件返回 `InvalidExecutable`。

5.15 用户内存访问：安全的跨特权级读写

5.15.1 用户指针范围校验

UserPtr<T> 与 UserConstPtr<T> 在返回引用前都经过check_region()。它依次检查对齐、整段地址是否可访问、并对惰性映射执行populate_area()。因此syscall 层拿到&'staticmutT 或&'staticT 前，页表已经满足本次访问权限。

空终止字符串需要逐页检查。扫描过程中可能触发page fault，而fault handler 也需要地址空间锁；check_null_terminated() 因此先逐页确认可访问性，再在access_user_memory() 范围内用volatile 读取寻找null terminator。范围校验和字符串检查分别位于kernel/src/mm/access.rs:59-78 与kernel/src/mm/access.rs:80-130。

5.15.2 用户拷贝中的page fault

注册到PAGE_FAULT 的处理函数只在三个条件同时满足时补页：线程上下文已经准备、当前任务是用户线程、当前线程处于access_user_memory() 设置的访问用户内存状态。这样，内核自己的错误指针不会被误当作用户缺页处理。

用户拷贝fault 入口位于kernel/src/mm/access.rs:277-298。它把“syscall 正在访问用户内存”做成显式线程状态，避免普通内核fault 进入地址空间补页路径。

5.15.3 用户拷贝分块

copy_user_bytes() 把大块拷贝切成16KB 的chunk，每个chunk 在IrqSave 保护下完成。这样可以限制单次关中断时间；若某个chunk 触发无法处理的page fault，则返回VmError::AccessDenied。分块拷贝实现位于kernel/src/mm/access.rs:312-333。

5.16 与StarryOS baseline 的设计比较

TheKernel 的地址空间子系统继承了StarryOS 的AddrSpace+MemorySet<Backend>+PageTable 骨架，但把Linux VMA 语义显式化。下表只比较源码设计边界，不引入本地运行环境说明。

能力	StarryOS baseline	TheKernel
地址空间状态	仅保存地址范围、VMA 集合和页表	增加growdown、dontfork、wipeonfork、locked ranges 与future-lock 状态
Backend 体系	已有Linear、COW、Shared、File 四类后端，但状态较薄	增加map_id、materialized、动态EOF、权限上限和memfd/futex 集成
MAP_GROWSDOWN	无growdown 集合与fault fallback	256 页guard 区、SP 校验、COW-only 扩展和起点迁移
MADV_DONTFORK / WIPEONFORK	sys_madvise() 基本为空实现	独立区间集合，fork 时跳过或清零，且wipeonfork 限定私有匿名映射

能力	StarryOS baseline	TheKernel
mlock / mlockall	mlock2() 返回成功但不记录锁定状态	locked ranges、MCL_CURRENT、MCL_FUTURE、MCL_ONFAULT 和 RLIMIT_MEMLOCK 检查
mremap	TODO 风格复制：新建私有映射、复制字节、unmap 旧段	segment 收集、共享验证、原地扩展、搬迁扩展、backend identity 与锁定状态迁移
SIGBUS on EOF	文件映射缺少完整 EOF fault 判断	CowBackend 与 FileBackend 均通过 faults_with_sigbus() 区分 EOF fault
页缓存驱逐协调	已有基础监听器与 callback	增加 direct I/O 排除、动态 EOF、memfd write-seal 跟踪与文件 futex key 复用
COW 引用计数	u8 计数，未区分未物化区间	u32 计数、materialized 标志、写文件映射 fork 前 populate 与 ABA 安全释放顺序
fault-around	每次 fault 按一页处理	匿名 COW 写 fault 一次填 4 页
brk	基础扩缩容	overcommit、碰撞检测、future-lock、memlock 与初始堆 populate 集成
msync	返回成功但不执行同步	收集 backend、处理 MS_SYNC / MS_ASYNC / MS_INVALIDATE，并释放锁后 sync

两者的根本差异在职责边界。StarryOS baseline 的地址空间主要维护 VMA 和页表；TheKernel 把 Linux 用户态能观察到的 VMA 附加语义放入 AddrSpace，把每类映射的填页与克隆语义放入 BackendOps。这样，系统调用层只负责解析 Linux ABI 和选择 backend，复杂行为由地址空间和 backend 的统一协议承接。

层次	状态对象	生命周期职责
地址空间所有者	AddrSpace	持有VMA 集合、页表、grow-down 起点、fork 过滤范围、mlock 状态和未来映射锁定策略。所有 backend 都通过它进入页表提交。
统一扩展点	Backend / backend ops	把匿名页、文件页、共享匿名页和线性映射统一成同一组populate、fork、unmap、writeback 与权限更新操作。
私有匿名映射	CowBackend	MAP_PRIVATE 通过只读共享页和frame refcount 延迟复制；写fault 时在复制新帧和原地升级之间选择。
文件共享映射	FileBackend	MAP_SHARED 从文件和page cache 补页，脏页通过msync 或回收路径写回，越过文件末尾时产生Linux 可见错误。
共享匿名映射	SharedBackend	跨进程共享匿名页池；fork 继续共享同一组物理页，适合SysV shm 与匿名共享映射。
线性映射	LinearBackend	把设备或物理内存窗口直接映射到用户地址空间，生命周期由VMA 与页表权限约束。

图 5.1 地址空间的Backend 抽象：四种映射类型统一到同一生命周期框架

阶段	操作	保证
锁定父地址空间	父进程在clone 路径持有AddrSpace 锁，冻结当前VMA 与页表视图。	子页表构造期间父子看到同一组待共享物理页。
构造子页表	子进程创建新的页表结构，并复制VMA 元数据。	VMA 语义继承，页表项可以单独提交或回滚。
共享私有页	present page 的frame refcount 加一，父子PTE 都降级为只读；第一次写入共享页时由硬件触发page fault。	写入之前没有进程能绕过COW 协议直接修改共享页。
处理写 fault	fork 路径已经返回后，父子任一方都可能触发写 fault；fault handler 调用backend populate 并读取frame refcount。	refcount 大于一时复制新帧；等于一时原地恢复写权限。
提交或释放	父子地址空间分别维护自己的PTE；旧帧 refcount 递减，归零后释放物理帧。	fork、munmap 和exit 走同一个引用计数闭环。

图 5.2 COW fork + write fault 的完整时序

页面状态	进入条件	离开条件	不变量
Unmapped	VMA 尚未materialize，或页表项已经被munmap/exit 移除。	匿名映射、文件映射或brk 路径首次提交页表项。	无有效PTE；非法访问按VMA 和权限规则转为fault 或SIGSEGV。
Read-only shared	fork 后的MAP_PRIVATE 页，或private file mapping 被materialize 后共享给父子。	写fault 进入COW 处理；munmap/exit 递减refcount。	PTE 可读不可写；frame refcount 记录共享者数量。
Writable private	新分配匿名页，或COW fault 后得到的独占页。	fork 再次把它降级为只读共享；munmap/exit 释放映射。	当前地址空间独占写权限；用户写入不会影响其它进程。
Freed	最后一个引用被munmap、exit 或COW 拆分释放。	物理页返回全局分配器，后续映射重新分配。	只有refcount 归零的帧可以释放；页表项必须先从地址空间移除或替换。

图 5.3 COW 页面状态机：Unmapped → Mapped-ReadOnly-Shared → Mapped-Writable-Private

第六章 文件系统：一切皆是文件对象

6.1 问题：整数fd 无法承载完整文件模型

Linux用户态把普通文件、目录、pipe、socket、epoll、timerfd、signalfd、pidfd、memfd、inotify、fanotify、userfaultfd 等资源都暴露成整数fd。用户程序通过read(2)、write(2)、poll(2)、close(2) 等统一接口操作它们，但每种对象背后的状态机完全不同。

单一整数槽位无法表达Linux fd 语义。典型场景有四个：

1. dup(2) 创建新fd 整数，但两个fd 共享同一个打开文件描述；其中一个lseek 会改变另一个的读写位置。
2. fork 后父子进程可共享同一份打开文件描述，因此共享offset 与O_APPEND、O_NONBLOCK 等status flags。
3. execve 只关闭带CLOEXEC 的槽位；没有关闭的fd 继续引用原来的打开文件描述。
4. fcntl lock 同时存在进程级POSIX lock 与打开文件描述级OFD lock，owner 的边界不能只靠fd 整数表达。

因此，TheKernel 将fd 模型拆成三层：底层对象实现I/O 行为，中间层保存共享的打开文件状态，最外层fd table 槽位只保存fd 编号相关属性。

6.2 三层模型：对象、打开描述、槽位

6.2.1 FileLike：所有fd 对象的共同接口

代码6.1 FileLike 接口

```
144 #[allow(dead_code)]
145 pub trait FileLike: Pollable + DowncastSync {
146     fn read(&self, _dst: &mut IoDst) -> AxResult<usize> {
147         Err(AxError::InvalidInput)
148     }
149
150     fn write(&self, _src: &mut IoSrc) -> AxResult<usize> {
151         Err(AxError::InvalidInput)
152     }
153
154     fn stat(&self) -> AxResult<Kstat> {
155         Ok(Kstat::default())
156     }
157
158     fn path(&self) -> Cow<'_, str>;
159 }
```

```

160 fn ioctl(&self, _cmd: u32, _arg: usize) -> AxResult<usize> {
161     Err(AxError::NotATty)
162 }
163
164 fn nonblocking(&self) -> bool {
165     false
166 }
167
168 fn set_nonblocking(&self, _nonblocking: bool) -> AxResult {
169     Ok(())
170 }
171
172 fn from_fd(fd: c_int) -> AxResult<FileHandle<Self>>
173 where
174     Self: Sized + 'static,
175 {
176     get_typed_file(fd)
177 }
178
179 fn add_to_fd_table(self, cloexec: bool) -> AxResult<c_int>
180 where
181     Self: Sized + 'static,
182 {
183     add_file_like(Arc::new(self), cloexec)
184 }
185 }

```

FileLike 是“可通过fd访问”的对象协议。它给read、write、stat、ioctl、poll、nonblocking 状态等行为提供统一入口，但默认实现大多返回错误，具体语义由对象自己覆盖。

trait 让special fd可以独立扩展。epoll、eventfd、timerfd、signalfd、pidfd、memfd 等对象都在各自模块实现FileLike，fd table 只保存Arc<dynFileLike>，需要具体能力时再通过downcast 恢复类型。

6.2.2 FileDescription：共享的打开文件状态

代码6.2 打开文件描述的共享状态

```

92 pub struct FileDescription {
93     pub inner: Arc<dyn FileLike>,
94     open_credentials: OpenCredentials,
95     flock_owner: u64,
96     status_flags: AtomicU32,
97     write_open_key: Option<ExecutableKey>,
98     async_io: Mutex<AsyncIoState>,
99 }
100
101 impl FileDescription {
102     pub(in crate::file) fn new(inner: Arc<dyn FileLike>) -> Arc<Self> {
103         Self::new_with_flags(inner, 0)
104     }
105
106     pub(in crate::file) fn new_with_flags(
107         inner: Arc<dyn FileLike>,
108         status_flags: u32,

```

```

109     ) -> Arc<Self> {
110         Self::new_inner(inner, status_flags, None)
111     }
112
113     pub(in crate::file) fn new_with_write_open_key(
114         inner: Arc<dyn FileLike>,
115         status_flags: u32,
116         write_open_key: Option<ExecutableKey>,
117     ) -> Arc<Self> {
118         Self::new_inner(inner, status_flags, write_open_key)

```

FileDescription 对应Linux 的 open file description。它先通过inner 持有真正的FileLike 对象，最后一个description 引用释放时底层对象才进入析构路径。

打开时凭证、可执行写保护和异步通知也属于这一层。open_credentials 保存打开文件时的身份，后续写入路径可以在with_write_credentials() 中恢复这份凭证；write_open_key 防止正在执行的映像文件被写入替换；async_io 保存FASYNC owner 和信号配置，pipe 等对象从这里同步异步通知状态。

flock_owner 与status_flags 则承载Linux 打开文件描述级共享语义。前者给fcntl lock、lease 和OFD lock 提供全局唯一owner，后者保存O_APPEND、O_NONBLOCK、FASYNC 等状态，fcntl(F_SETFL) 修改的正是这一层。

析构路径同样说明FileDescription 是共享语义边界。最后一个引用消失时，内核释放fanotify、POSIX lock、OFD lock、lease 和executable write-open 引用；普通fd 槽位关闭时只减少引用，不会提前清理这些共享资源。清理入口位于kernel/src/file/desc.rs:163-174。

6.2.3 FileDescriptor: fd table 槽位

FileDescriptor 只有两个字段：指向共享FileDescription 的引用，以及槽位自己的cloexec 标志。这个结构故意很薄，因为槽位层只回答“这个fd 编号当前指向哪里、exec 时是否关闭”；读写位置、状态flags、lock owner 和对象生命周期都属于FileDescription。槽位定义位于kernel/src/file/desc.rs:267-270。

6.2.4 fd table 的分配、dup 与fcntl

fd table 由进程级RwLock 保护，读路径获取共享锁，分配或关闭路径获取写锁。新增fd 时先检查RLIMIT_NOFILE，再把FileDescription 包装成新的FileDescriptor 槽位。

dup 不复制底层对象，也不复制打开文件状态。它只新建一个槽位，并让该槽位的description 指向同一个Arc<FileDescription>。这解释了为什么dup 后的两个fd 共享offset、status flags 和OFD lock owner。

fcntl(F_SETFL) 修改FileDescription 的status_flags，因此会影响所有共享该description 的fd；fcntl(F_SETFD) 只修改当前fd table 槽位的cloexec，因此不会影响dup 出来的其他fd。fd table 分配、dup 和fcntl 分层分别位于kernel/src/file/fd_table.rs:17-68 与kernel/src/syscall/fs/fd_ops.rs:907-1247。

6.2.5 三层模型与StarryOS baseline

StarryOS baseline 的fd table 槽位直接保存Arc<dynFileLike> 与cloexec。这可以支持基本read/write/close，但缺少FileDescription 层，导致“槽位状态”和“打开文件描述状态”难以分离。TheKernel 在这一层增加FileDescription 后，dup、fork、execve、fcntl lock、FASYNC 和executable write-open 才能落在正确的所有权边界上。

StarryOS baseline 的FileDescriptor 位于/tmp/starryos-upstream/kernel/src/file/mod.rs；TheKernel 的三层模型位于kernel/src/file/types.rs、kernel/src/file/desc.rs 与kernel/src/file/fd_table.rs。

6.3 统一接口下的特殊fd 对象

6.3.1 epoll：fd 整数和打开描述共同作为key

epoll 的interest 不能只用fd 整数做key。TheKernel 使用(fd,Arc<FileDescription>) 组合：fd 区分同一个epoll 中的槽位编号，FileDescription 区分dup、close、重开之后对象身份是否仍然相同。

触发模式也存放在interest 上，epoll wait 逻辑只消费已经维护好的触发状态：

1. level-triggered interest 在底层对象持续ready 时持续报告。
2. edge-triggered interest 依赖ready queue 中的状态转换，只在边沿上报告。
3. one-shot interest 在触发后disarm，等待用户态通过修改事件重新armed。

interest key 与触发状态位于kernel/src/file/epoll.rs:107-168。这个设计让epoll 同时保留Linux 对fd 数字的可见行为和对打开文件描述身份的稳定判断。

6.3.2 eventfd：计数器就是文件状态

eventfd将一个64-bit 计数器包装成fd 对象。read 在计数非零时消费计数，semaphore 模式每次只消费1；write 累加计数，遇到溢出时按阻塞或非阻塞语义等待或返回EAGAIN。

poll 语义直接由计数器决定：非零可读，未接近u64::MAX 可写，达到错误状态时报告ERR。FileLike 统一接口保留对象差异，只提供同一组入口让每种对象表达自己的状态机。eventfd 的计数器、读写入口和poll 语义位于kernel/src/file/event.rs:13-120。

6.3.3 timerfd 与signalfd：内核事件转为可读字节流

timerfd把内核alarm 转换为fd 可读事件。TimerFdInner 保存到期次数、重复间隔和下一次到期时间；读路径按当前时间惰性更新到期次数，再向用户态返回64-bit expiration count。

signalfd 把pending signal 转成固定128 字节的signalfd_siginfo。对象内部保存mask；read 时只取mask 覆盖的pending signal，并在读出POSIX timer signal 时同步确认timer 通知。

timerfd 和 signalfd 的 fd 对象分别位于 kernel/src/file/timerfd.rs:63-122 与 kernel/src/file/signalfd.rs:17-86。

6.3.4 pidfd 与 memfd：对象身份和匿名文件

pidfd 通过弱引用指向进程或线程对象，并注册退出事件用于 poll。这样用户态拿到的是一个稳定对象句柄，不会因为 PID 数字复用而误操作新进程。

memfd 的关键是 seal 状态。TheKernel 将 seal 和可写共享映射计数挂在文件 location 的 user data 上；添加 F_SEAL_WRITE 前会检查当前是否存在 active writable shared mapping，写入和 resize 也会查询 seal 限制。pidfd 和 memfd 的对象状态分别位于 kernel/src/file/pidfd.rs:19-91 与 kernel/src/file/memfd.rs:10-116。

6.3.5 inotify、fanotify、userfaultfd 与异步环接口

这些对象遵守同一条边界：对象模块实现自己的事件队列、权限检查或 ring 状态，fd table 只负责把它们作为 FileLike 暴露给用户态。

1. inotify 监控 inode/path 事件，事件进入队列后由 read 消费。
2. fanotify 覆盖更宽的文件系统事件，并可生成 permission event。
3. userfaultfd 把用户态缺页处理协议封装为 fd。
4. io_uring 当前覆盖 ring 注册与基础提交路径，完整异步执行模型属于实现边界。

这些对象分别位于 kernel/src/file/inotify.rs、kernel/src/file/fanotify.rs、kernel/src/file/userfaultfd.rs、kernel/src/file/io_uring.rs 与 kernel/src/syscall/fs/io_uring.rs。

6.4 普通文件I/O：统一fd入口后的对象分派

sys_read() 和 sys_write() 首先通过 fd table 取得 FileHandle<dynFileLike>。普通文件会额外走 direct I/O 对齐、mount write 权限、append offset、可执行文件写保护、同步写回和 inotify/fanotify 通知；非普通对象则直接进入自己的 FileLike 实现。

这条路径体现了三层模型的分工：fd table 负责把整数 fd 翻译成 FileDescription，FileDescription 提供 status flags 与打开凭证，FileLike 对象执行真正的 I/O。统一入口位于 kernel/src/syscall/fs/io.rs:232-285。

6.5 pseudofs：内核状态到格式化文件

6.5.1 SimpleFs 与 SimpleFsNode

pseudofs 用 SimpleFs 建立 procfs、sysfs、tmpfs-like 控制文件等虚拟文件树。每个 SimpleFsNode 有独立 inode 和 metadata，但内容不一定来自持久存储。文件系统和节点元数据定义位于 `kernel/src/pseudofs/fs.rs:24-86`。

6.5.2 SimpleFile 的生成式读写

SimpleFile 的核心是 `read_all()`。动态文件在每次 read 时重新生成内容，例如 `/proc/[pid]/fdinfo/[fd]`、`/proc/mountinfo`、`/proc/meminfo`、`/proc/sysvipc/*`；控制文件也可以通过 `write_all()` 把写入文本转换成内核状态更新。

生成式文件要处理四个读写语义：短读、seek 后读取、并发状态变化、多次 read 之间的大小更新。TheKernel 在 `read_at()` 中先生成完整内容，再按 offset 和缓冲区长度切片，同时回写 metadata size。函数包装和生成式读取位于 `kernel/src/pseudofs/file.rs:13-170`。

6.6 mount ledger：挂载状态的真相源

mount ledger 用 MountRecord 记录 source、target、文件系统类型、挂载参数、设备号、flags 和 lazy unmount 过期标记。传统 `mount(2)` 和新 mount API 都写入这份 ledger，使 `/proc/mountinfo`、权限检查和路径解析看到同一组挂载事实。

ledger 还处理 propagation、move mount 和 umount expiry。比如 `move_tree()` 会把挂载点及其子挂载整体改写到新路径；`change_propagation()` 会递归更新 shared、private、slave、unbindable 等传播属性。记录结构、插入、传播和移动逻辑位于 `kernel/src/mounts.rs:10-181`。

6.7 工程边界

当前文件系统实现的工程边界集中在四类责任上：

1. VFS 与 fd 模型已经把槽位、打开文件描述和对对象行为分开，但真实 ext4/fat 的全部语义仍取决于底层 fs backend 覆盖范围。
2. io_uring 具备基础 fd 和 ring 接口，完整异步执行、取消、链接请求和高级 CQ/SQ 行为仍是独立扩展面。
3. fanotify permission event、userfaultfd write-protect fault、复杂 rename/unlink 并发语义需要靠专门测试继续收敛。
4. pseudofs 的生成式文件适合 proc/sys 状态导出，但它不替代真实持久文件系统的一致性模型。

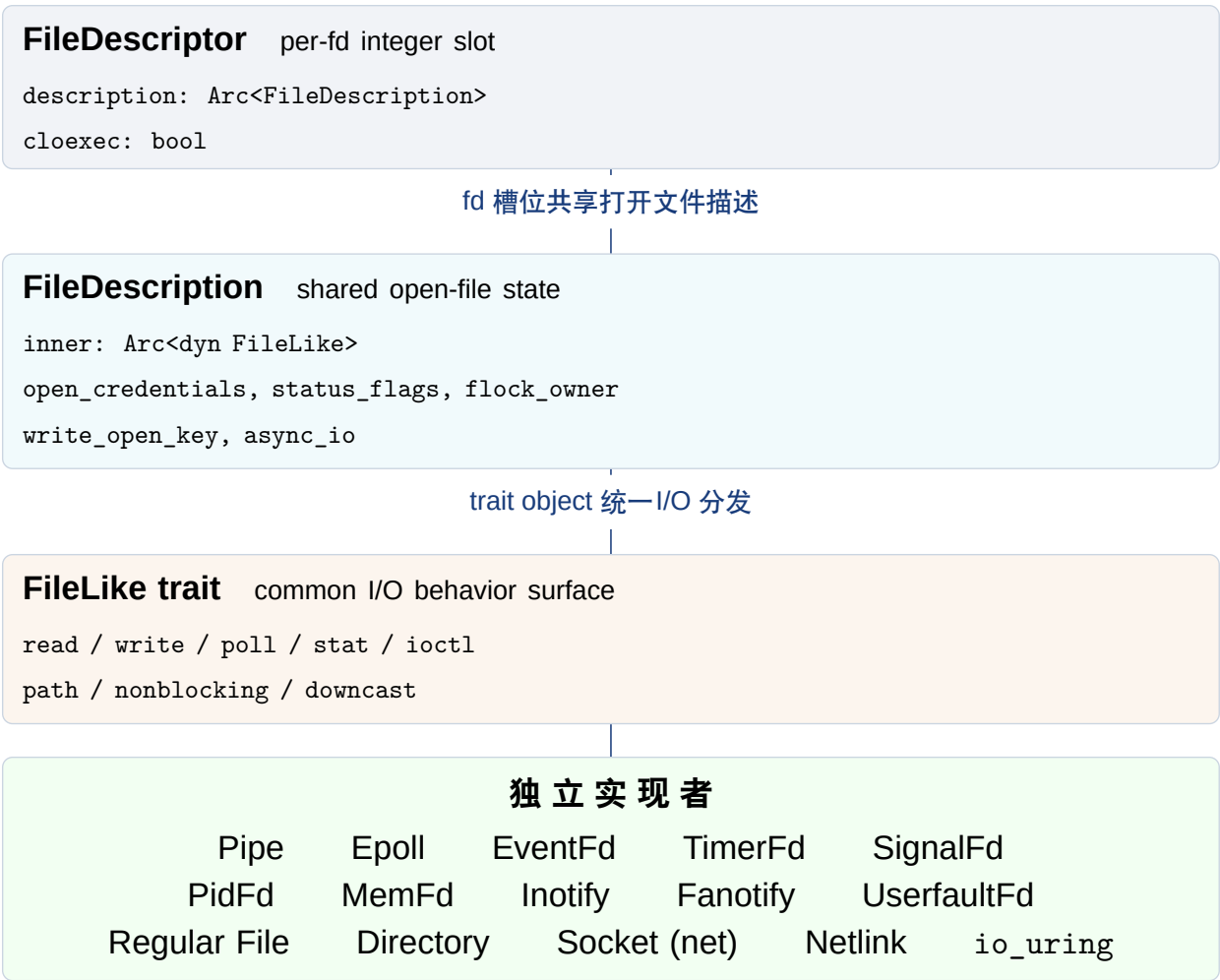


图 6.1 FileLike / FileDescription / FileDescriptor 三层模型

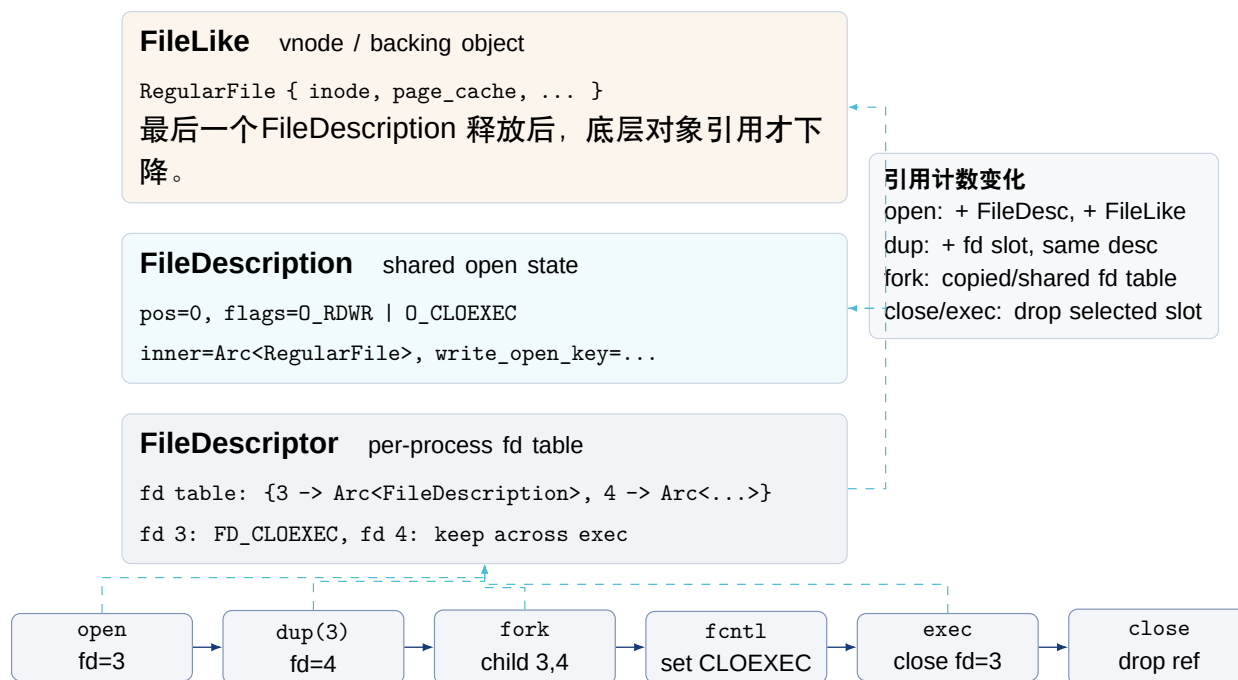


图 6.2 文件描述符生命周期：open → dup → fork → exec → close 三层引用模型

第七章 网络栈：axnet-ng 的协议无关 Socket 模型

7.1 多协议共享的 socket ABI

Linux 用户态通过 `socket(2)`、`bind(2)`、`connect(2)`、`listen(2)`、`accept(2)`、`sendmsg(2)`、`recvmsg(2)` 等系统调用操作网络对象。整数 `fd` 背后可能是 TCP 连接、UDP 数据报、Unix domain socket、vsock，也可能是 packet、netlink、AF_ALG 等兼容对象。TheKernel 把这个 ABI 拆成五个相对稳定的责任层次。

1. syscall 层解析 Linux 地址结构、socket type、协议号和 flags，并负责把用户指针、非阻塞标志和 close-on-exec 标志转换成内核对象状态。
2. kernel fd wrapper 把 axnet-ng socket 接入 FileLike 与 Pollable，使 socket 可以进入普通 fd、poll、epoll、ioctl 和 close 路径。
3. axnet-ng 用 SocketOps 与 Socket enum 统一 TCP、UDP、Unix 与 vsock。调用方只依赖共同操作，具体传输负责自己的状态机。
4. TCP、UDP 和 Unix transport 各自维护连接状态、地址状态、缓冲区、waker 和错误码映射。它们共享通用的 socket option 与阻塞等待协议。
5. NetStack、Service、Router 和 ListenTable 承载网络 namespace、路由、socket set、SYN 队列和设备轮询。

StarryOS baseline 已经通过 workspace 依赖使用发布版 axnet-ng，并有 kernel 侧 socket wrapper 和 socket 系列 syscall。TheKernel 的增量集中在本地 vendored 的 axnet-ng 实现、网络 namespace 状态、更多兼容对象、socket option、BPF filter、RDS local datagram，以及 packet、netlink、ALG 等入口。

7.2 统一 socket 抽象

7.2.1 共同操作边界

代码 7.1 SocketOps 与 Socket 枚举边界

```
157 /// Operations that can be performed on a socket.
158 #[enum_dispatch]
159 pub trait SocketOps: Configurable {
160     /// Binds an unbound socket to the given address and port.
```

```

161 fn bind(&self, local_addr: SocketAddrEx) -> AxResult;
162 /// Connects the socket to a remote address.
163 fn connect(&self, remote_addr: SocketAddrEx) -> AxResult;
164
165 /// Starts listening on the bound address and port.
166 fn listen(&self) -> AxResult {
167     Err(AxError::OperationNotSupported)
168 }
169 /// Accepts a connection on a listening socket, returning a new socket.
170 fn accept(&self) -> AxResult<Socket> {
171     Err(AxError::OperationNotSupported)
172 }
173
174 /// Send data to the socket, optionally to a specific address.
175 fn send(&self, src: impl Read + IoBuf, options: SendOptions) -> AxResult<usize>;
176 /// Receive data from the socket.
177 fn recv(&self, dst: impl Write + IoBufMut, options: RecvOptions<'_>) -> AxResult<usize>;
178
179 /// Get the local endpoint of the socket.
180 fn local_addr(&self) -> AxResult<SocketAddrEx>;
181 /// Get the remote endpoint of the socket.
182 fn peer_addr(&self) -> AxResult<SocketAddrEx>;
183
184 /// Shutdown the socket, closing the connection.
185 fn shutdown(&self, how: Shutdown) -> AxResult;
186 }
187
188 /// Network socket abstraction.
189 #[enum_dispatch(Configurable, SocketOps)]
190 pub enum Socket {
191     /// UDP socket.
192     Udp(UdpSocket),
193     /// TCP socket.
194     Tcp(TcpSocket),
195     /// Unix domain socket.
196     Unix(UnixSocket),
197     /// Virtio socket.
198     #[cfg(feature = "vsock")]
199     Vsock(VsockSocket),
200 }

```

SocketOps 是网络对象的主协议。连接生命周期由bind、connect、listen、accept、shutdown 表达，数据路径由send 与recv 表达，地址查询由local_addr 与peer_addr 表达。无连接协议沿用默认的unsupported 返回，因此fd 层不需要知道某个transport 是否真的支持被动打开。

Socket enum 用enum_dispatch 固定分发边界。新增传输类型时，改动集中在enum variant 和该transport 的trait 实现；syscall 层、fd wrapper 和通用poll 路径仍面向SocketOps。这个边界也限定了错误码的归属：地址族不匹配在地址转换处返回EAFNOSUPPORT，状态不合规则由具体transport 返回EINVAL、ENOTCONN、EINPROGRESS 或EOPNOTSUPP 等Linux 可见错误。

SocketAddrEx、SendOptions 和RecvOptions 是这层抽象的参数面。地址enum 覆盖IP、Unix 和vsock；send options 保存显式目标地址、flags 和ancillary control message；recv options 保存发送方地址回写位置、PEEK、TRUNCATE 和控制消息输出。这样TCP 可以忽略sendto 目标地址，UDP 可以在显式目标和connected peer 之间切换，Unix datagram 可以把fd

传递和发送方地址一起交给本地队列。

7.2.2 接入fd 层与BPF filter

kernel 侧的Socket wrapper 持有axnet-ng 的Socket enum, 并通过Deref 暴露底层方法。它实现FileLike 时做了四件事: read 转成默认recv, write 转成默认send, stat 返回S_IFSOCK, nonblocking 与set_nonblocking 映射到socket option。对用户态来说, socket 因此可以像普通fd 一样被read/write、poll/epoll、fcntl、close 和ioctl 处理。

BPF socket filter 也挂在wrapper 层。内核中的BPF 程序被包成axnet-ng 的socket filter, filter 执行结果决定数据报保留的字节数; 返回0 表示丢弃, 返回小于原始长度的值表示截短。这个位置让BPF 属于kernel policy, TCP、UDP 和Unix datagram 只需要调用统一的filter trait。对应代码集中在kernel 侧的socket wrapper 文件中。

7.3 NetStack: 网络namespace 的状态容器

代码7.2 NetStack namespace 状态

```

29 pub struct NetStack {
30     pub(crate) listen_table: Arc<ListenTable>,
31     pub(crate) socket_set: Arc<SocketSetWrapper<'static>>,
32     pub(crate) service: Mutex<Service>,
33     pub(crate) tcp_ephemeral_port: Mutex<u16>,
34     pub(crate) udp_ephemeral_port: Mutex<u16>,
35     ipv4_conf_default_tag: AtomicI32,
36     ipv4_conf_lo_tag: AtomicI32,
37 }
38
39 const PORT_START: u16 = 0xc000;
40 const PORT_END: u16 = 0xffff;

```

NetStack 挂在ProcessData.net_ns 上, 是网络namespace 的真实状态对象。普通clone 共享父进程的Arc<NetStack>; 带CLONE_NEWNET 的clone 创建新的stack。新建namespace 使用loopback-only 配置: socket set 和listen table 从空状态开始, router 只有lo 设备, 地址包含127.0.0.1/8 与::1/128, 对应路由的源地址也来自loopback。这样本机通信和namespace 隔离先成立, 外部NIC 路径再由设备驱动和路由表补充。

这个对象同时维护三类共享状态。

- 1. 对象注册状态。**socket set保存smoltcp socket handle, listen table 保存TCP 端口到SYN 队列的映射。listen table 在结构体中排在socket set 前面, 析构时能先清理弱引用持有的预连接socket handle, 避免监听端和socket set 互相保活。
- 2. 协议推进状态。**service包含smoltcp interface 和router。send、recv、connect、accept、poll 都会需要在需要时调用poll_interfaces(), 先收设备包, 再推进smoltcp 状态, 最后分发待发送包。

3. **Linux 兼容状态。** TCP 和UDP 临时端口分别从0xC000 到0xFFFF 循环分配，并通过 socket set 检查同类型端口是否已占用；ipv4_conf_* tag 保存proc/sysctl 可见的接口兼容状态。

锁顺序由with_service_and_socket_mut() 固定为先锁service、再锁socket set。connect、bind、路由解析和socket 状态更新经常同时触碰这两个对象，统一顺序可以避免service 路由锁与socket set 互斥锁之间形成AB-BA 死锁。失败路径通常在具体transport 内返回错误，已经分配的smoltcp handle 则由Drop 或显式remove 释放。

7.4 TCP：状态事务、握手和被动打开

7.4.1 状态提交规则

代码7.3 TCP 状态事务提交

```

49 pub fn lock(&self, expect: State) -> Result<StateGuard<'_, State>, State> {
50     match self.0.compare_exchange(
51         expect as u8,
52         State::Busy as u8,
53         Ordering::Acquire,
54         Ordering::Acquire,
55     ) {
56         Ok(_) => Ok(StateGuard(self, expect as u8)),
57         Err(old) => Err(old.try_into().expect("invalid state")),
58     }
59 }
60 }
61
62 #[must_use]
63 pub struct StateGuard<'a>(&'a StateLock, u8);
64 impl StateGuard<'_> {
65     pub fn transit<R>(self, new: State, f: impl FnOnce() -> AxResult<R>) -> AxResult<R> {
66         match f() {
67             Ok(result) => {
68                 self.0.0.store(new as u8, Ordering::Release);
69                 Ok(result)
70             }
71             Err(err) => {
72                 self.0.0.store(self.1, Ordering::Release);
73                 Err(err)
74             }
75         }
76     }
77 }

```

TCP socket 的状态由Idle、Connecting、Connected、Listening、Closed 和短临界区Busy 组成。状态切换先用CAS 把预期状态改成Busy，闭包成功后提交新状态，闭包失败则恢复旧状态。这一点很关键：bind、connect、listen、shutdown 都可能在路由解析、端口分配或smoltcp 调用处失败，失败后用户态再次调用时会看到原来的socket 状态，半提交的中间态不会泄漏出去。

底层 smoltcp socket 的并发访问由 socket set 的互斥锁串行化。状态锁只保护 Linux 生命周期状态，不承载收发缓冲本身。poll 这种高频路径只需要读取状态并检查 smoltcp readiness；bind/connect/listen/shutdown 才进入 CAS 事务。

7.4.2 主动连接

TcpSocket::connect() 的事务边界在 crates/axnet-ng/src/tcp.rs。它先把 0.0.0.0 或:: 形式的远端地址规范化为 localhost，再从 Idle 进入 Connecting。在事务闭包内，service 按路由表解析出站路径；未绑定 socket 选择源地址和临时端口，已绑定 socket 检查本地端口不能与远端端口相同。随后实现按固定锁顺序拿到 service 与 smoltcp socket，写入 bound endpoint，调用 smoltcp connect() 发起握手，并把设备唤醒 mask 写入 GeneralOptions。

状态提交为 Connecting 后，connect 进入发送侧 poller。poller 每轮先推进网络接口，再检查 smoltcp 状态。只要 smoltcp 已经越过三次握手，Established、FinWait、CloseWait、TimeWait 等握手后状态都视为 connect 成功；这匹配 Linux 在 peer accept 后立即 close 的竞态语义。若 socket 回到其他失败状态，则状态被写成 Closed，用户态得到连接失败错误。非阻塞 fd 上的 WouldBlock 在 syscall 层映射为 EINPROGRESS。

7.4.3 listen、accept 与 SYN 队列

被动打开由 ListenTable 和 router 的 RxToken 共同完成。listen 时，TCP socket 必须已有非零本地端口；listen table 在端口槽位保存监听 endpoint、SYN 队列和 socket set 的弱引用。弱引用避免监听表和 socket set 循环持有，监听项析构时会移除队列中尚未 accept 的预连接 handle。

接收 TCP 首包时，router 在 smoltcp 消费 packet 之前检查 TCP SYN 且 ACK 未置位的包。若目标端口存在监听条目且队列未满，listen table 创建一个新的 smoltcp TCP socket，令其 listen 在相同 endpoint 上，并把 handle 推入 SYN 队列。后续 SYN-ACK/ACK 仍由 smoltcp 推进。accept 使用接收侧 poller 等待队列中出现已经离开 Listen/SynReceived 的 handle，然后把该 handle 包装成新的 TcpSocket::new_connected() 并分配 fd；如果 handle 已经关闭，accept 返回连接重置错误。

7.4.4 收发、poll 与关闭

TCP send 和 recv 都复用 GeneralOptions 的 poller。send 先推进接口，再检查 socket 是否 active 与可发送；可发送时把用户缓冲拷入 smoltcp tx buffer。recv 在状态上拒绝 Idle、Connecting 和 Listening；连接可读时处理 EOF、PEEK 和普通消费。poll 根据状态分派：Connecting 检查 OUT，Connected/Idle/Closed 检查流式收发，Listening 检查 SYN 队列，Busy 返回空事件。读半关闭由 rx_closed 和单独的 poll set 表达，便于 RDHUP 唤醒。

shutdown 的提交点同样受状态锁保护。写半关闭调用 smoltcp close() 并推进接口；listener 关闭会从 listen table 注销端口；Drop 会先尝试双向 shutdown，再给 loopback peer 一个短窗口观察优雅关闭，最后从 socket set 移除 handle。这个顺序保证 fd 已经关闭时资源一定

释放，同时尽量让本机peer 看见FIN/EOF，降低关闭阶段直接丢失socket 的概率。

7.5 UDP：无连接数据报与双地址视图

UDP的核心状态由三组地址组成。实际绑定地址保存在`local_addr`，可能是通配地址；Linux `getsockname()` 应报告的地址保存在`reported_local_addr`；`connect()` 设置的默认远端和路由解析出的源地址保存在`peer_addr`。这个分离覆盖了常见兼容场景：程序以`INADDR_ANY:0` 绑定再`connect` 到远端时，内核仍保持通配符接收语义，但用户态查询本地地址时要得到可告知peer 的具体源地址。

UDP `bind`、`connect`、`send`、`recv` 的控制流分别承担不同的Linux 语义。

1. **bind** 分配端口、校验本地地址是否可作为源地址、检查同类型端口冲突，并把smoltcp socket 绑定到`IpListenEndpoint`。端口为0 时从UDP 临时端口计数器分配。
2. **connect** 不发送数据包。它在未绑定时自动绑定通配地址和临时端口，解析到远端的出站路由，记录device mask，并把路由选中的源地址保存为报告地址。
3. **send** 在显式目标和connected peer 两条路径之间选择。发送前检查端口、未指定地址和最大数据报长度；未绑定时自动绑定；写入smoltcp tx buffer 后主动推进接口，使loopback 接收端尽快看到readiness。
4. **recv** 在recvfrom 场景接受任意来源并写回发送方地址，在connected 场景只接受匹配peer 的数据报。若用户缓冲小于数据报，普通recv 返回实际拷贝长度，带TRUNCATE 时返回原始数据报长度。

错误边界也由这些步骤决定。未绑定recv 返回`ENOTCONN`；目标地址缺失、端口为0 或地址未指定返回无效参数；数据报超过UDP 长度上限返回`EMSGSIZE`；发送缓冲满时返回`WouldBlock`，再由通用poller 决定阻塞、超时、信号中断或立即返回。

7.6 Unix domain socket：地址服务发现与本地传输

Unix domain socket把地址服务发现与传输分开。地址有无名、抽象命名空间和文件系统路径三种形式。抽象地址进入全局`ABSTRACT_BINDS`；路径地址通过VFS 创建socket inode，并把BindSlot 挂到inode user data 上。BindSlot 内部分开保存stream 与datagram 绑定，因此同一路径可以分别承载两类传输。

stream transport 用一对ring buffer 通道表达双向字节流。`connect` 时，客户端通过监听端Bind 创建client/server 两个Channel，server 端经`async_channel` 送入accept 队列。`send` 使用ringbuf 的空闲连续切片写入字节，写入后唤醒peer 的poll set；`recv` 从已占用切片读取并推进读指针。读写半关闭分别由原子布尔值表达，双向关闭后取走channel 并唤醒peer。

datagram transport 用`async_channel` 传递完整消息，并用`queued_bytes` 加CAS 进行按字节计费的背压。容量取发送缓冲与接收缓冲的较小值，保证发送端不会超过接收端配

置。发送路径支持显式目标地址和connected channel 两种模式；送入队列前执行BPF filter, filter 可以丢弃或截短消息。接收路径取出完整packet, 扣减queued bytes, 写回发送方地址和控制消息；带TRUNCATE 时返回消息原始长度。

Unix socket 的accept() 由异步接收转换成阻塞syscall 语义：transport 的accept future 被interruptible 包装后block-on, 信号到达时可以中断等待。peer credentials 由创建socket 的PID 构造，stream 连接后返回peer PID, datagram 因为无稳定peer, 返回当前socket 创建者的兼容凭证。

7.7 设备轮询、路由与通用阻塞语义

Service::poll() 是网络栈推进点。它先让router 从设备收包，再让smoltcp interface 处理协议状态和socket set, 最后把待发包交给router 分发。出站路由按最长前缀规则查找目标地址；若socket 绑定了具体源地址，源地址必须与路由规则一致，否则返回EADDRNOTAVAIL。IPv4/IPv6 分发时也会检查packet 源地址是否等于路由规则的源地址，避免错误源地址离开设备。

通用阻塞语义由GeneralOptions 统一。每个socket 保存nonblocking、reuse address、dont route、收发超时、缓冲区大小、IPv6-only 和device mask。send/recv 调用闭包时，如果闭包返回WouldBlock 且fd 非阻塞，错误直接返回；阻塞fd 会注册waker, 等待readiness、timeout 或signal interrupt。device mask 让service 只把waker 注册到相关设备，降低无关设备唤醒带来的空转。

socket option 处理采用两级分派。具体transport 先处理TCP no-delay、keepalive、UDP TTL、Unix credential 等自己认识的option；若返回不支持，通用层再处理GeneralOptions 覆盖的socket-level option；最终仍不支持的option 统一返回ENOPROTOOPT。这样各传输不用重复实现nonblocking、timeout 和buffer size, 但仍能覆盖自己的协议级选项。

7.8 syscall 到数据包的路径

sys_socket() 先解析socket type 中的O_NONBLOCK 与O_CLOEXEC, 拒绝未知flags, 再按domain/type/proto 选择对象。AF_ALG、AF_PACKET 和AF_NETLINK 走内核兼容对象；INET stream 创建TCP, INET datagram 创建UDP, INET raw 返回EPROTONOSUPPORT, RDS seqpacket 复用UDP local datagram 路径, Unix stream/datagram 创建对应本地transport, vsock 在feature 打开时创建vsock transport。创建完成后，nonblocking 写入socket option, cloexec 写入fd table 槽位。

bind、connect、accept4 和socketpair 是syscall 层最重要的提交边界。

1. **bind** 先识别AF_ALG、netlink、packet 的专用路径，普通socket 则从用户态读取SocketAddrEx, 检查低端口绑定权限，再交给transport。权限检查在调用transport 前完成，因此失败不会改变socket 状态。

2. **connect** 先确认fd 存在，再识别AF_UNSPEC 作为断开连接语义。普通地址交给transport 的connect 方法；底层WouldBlock 被转换成非阻塞connect 需要的EINPROGRESS。
3. **accept4** 解析nonblocking 与cloexec flags，调用底层accept 得到新的axnet-ng socket，设置nonblocking，写回peer 地址，最后把新socket 安装到fd table。安装完成之前，新socket 对用户态不可见。
4. **socketpair** 只支持Unix domain。stream 创建一对StreamTransport，datagram 和seqpacket 创建一对DgramTransport；第二个fd 分配失败时关闭第一个fd，避免半成功状态暴露给用户态。

一次loopback TCP 通信会依次经过这些边界：server 和client 创建TCP socket；server bind/listen 把端口写入smoltcp endpoint 与listen table；client connect 解析loopback 路由并发出SYN；router 预处理SYN 并把预连接handle 推入SYN 队列；smoltcp 完成握手；server accept 将handle 包装为新fd；client send 后，router dispatch 到loopback device，下一次poll 把数据包送入server 接收队列，server recv 再复制给用户态。

7.9 与StarryOS baseline 的设计差异

StarryOS baseline使用发布版axnet-ng 依赖，kernel 侧已有基础Socket wrapper、socket 系列syscall、TCP/UDP/Unix/vsock 类型分发。TheKernel 的变化集中在四个方面。

1. **本地化依赖**。仓库内vendored crates/axnet-ng，并通过[patch.crates-io] 让直接和间接依赖都使用同一份本地实现，便于把OSComp/LTP 兼容补丁落在协议栈内部。
2. **namespace 状态**。NetStack 成为进程namespace 对象，CLONE_NEWNET 可以得到独立loopback-only 栈，普通clone 共享父进程栈。
3. **Linux 兼容面**。RDS local datagram、AF_PACKET、AF_NETLINK、AF_ALG、BPF socket filter、socket option、IPv6-mapped IPv4 和TCP connect 竞态语义都在TheKernel 侧补强。
4. **阻塞和poll 语义**。各传输复用GeneralOptions 的poller，使nonblocking、timeout、signal interrupt 与epoll readiness 走同一套等待协议。

7.10 当前边界

网络栈当前的责任边界有四类。

1. 外部NIC 路径依赖实际设备驱动和virtio-net 适配；loopback-only 栈保证新namespace 和本机通信可工作。
2. INET SOCK_RAW 仍返回EPROTONOSUPPORT，原始IP 构造不在当前实现范围内。
3. TCP_INFO 兼容入口存在，但还没有填充完整的Linux tcp_info 数据。

4. TCP shutdown 已覆盖基本读写半关闭和listener 注销, FIN 细节、异常关闭和长连接压力仍需要更多专门测试。

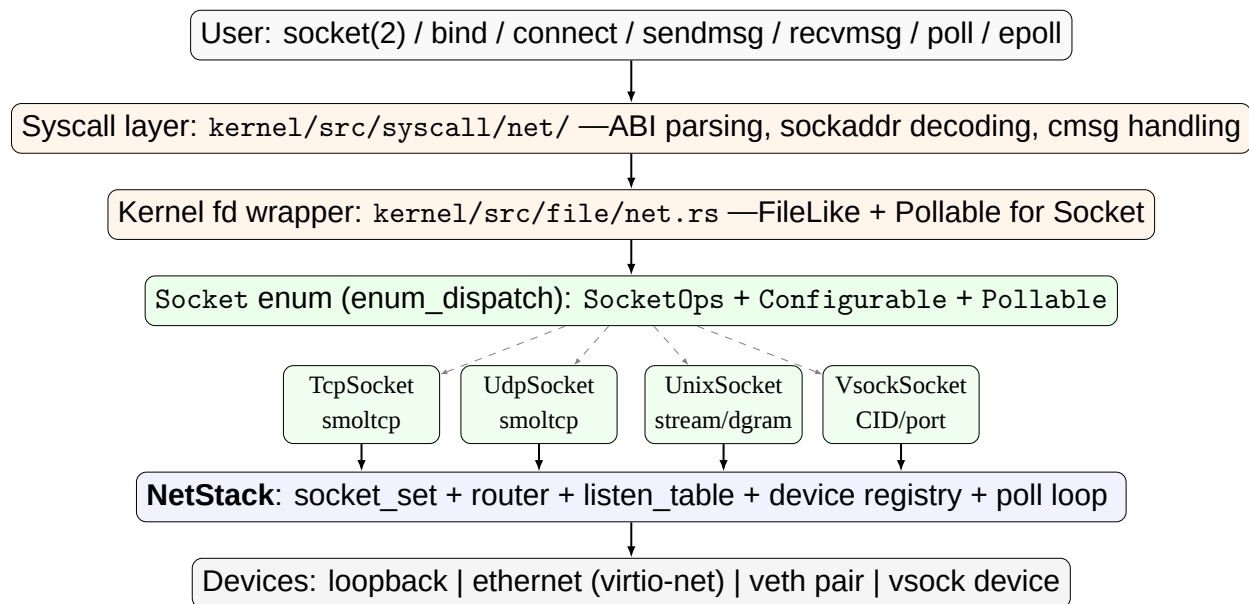


图 7.1 axnet-ng 网络栈分层架构

状态	进入条件	离开条件	Linux 可见语义
Idle	创建socket 对象, 尚未绑定本地端点。	bind 或隐式绑定。	fd 已存在, 但还不能收发连接数据。
Bound	已分配本地地址或端口。	listen 进入监听, 或 connect 发起主动连接。	可以承接TCP server/client 两条路径。
Listening	listen(backlog) 注册监听队列。	accept 取出完成握手的连接。	poll 暴露readable 语义, 表示backlog 中有可接受连接。
Connecting	connect 发起三次握手。	握手完成进入 Connected; RST、timeout 或路由错误进入Closed。	非阻塞连接通过poll writable 或错误码向用户态报告结果。
Connected	accept 或 connect 成功。	shutdown 半关闭, 或 close 直接释放 fd。	readable/writable 由 rx/tx buffer、FIN 和 socket error 共同决定。
Shutdown	一侧读写方向已经关闭。	close 或对端结束剩余方向。	继续报告FIN、HUP 或可读EOF, 直到fd 生命周期结束。
Closed	本地close、远端RST 或错误终止。	无新的收发路径。	poll 通过HUP/ERR 或后续syscall 错误把关闭状态返回用户态。

图 7.2 TCP Socket 连接状态机 (State enum in axnet-ng)

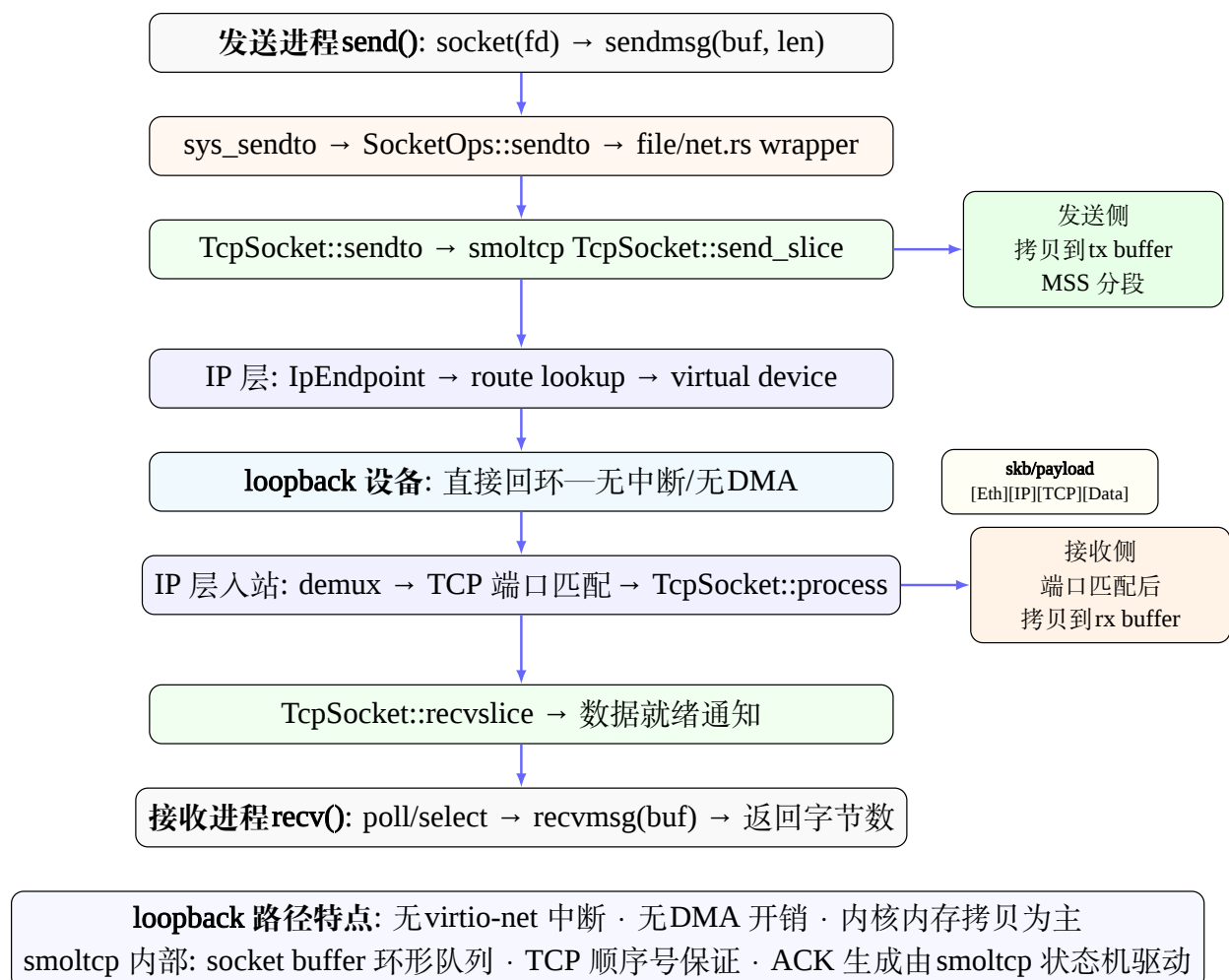


图 7.3 TCP Loopback 数据路径: send → TCP → IP → loopback → IP → TCP → recv 全层穿透

第八章 信号、同步与时间：阻塞路径上的共同控制面

8.1 阻塞线程的三个入口

信号、futex和时间子系统在Linux ABI 中经常同时出现。一个线程可能在futex 队列中睡眠，另一个线程向它发送信号；一个带超时的等待可能被handler 打断并经restart_syscall 回到原等待；一个POSIX timer 到期后可能投递信号，也可能唤醒timerfd 的poll waiter。TheKernel 将这些路径收束到三个共享控制点。

1. 返回用户态前的信号投递。线程每次回到用户态前检查pending signal，必要时构造signal frame、驱动stop/continue/terminate，并把本次投递结果交给restart tracker。
2. futex 等待队列。用户态锁只在竞争时进入内核；内核需要把用户地址映射成稳定key，并在signal、timeout、wake、requeue 之间维护waiter 生命周期。
3. 统一deadline。nanosleep、futex timeout、itimer、POSIX timer 和timerfd 都落到realtime 或monotonic 时间域，差异体现在通知动作、clockid 解码和namespace 偏移上。

8.2 信号：两级pending 队列和投递点

8.2.1 队列所有权

代码8.1 pending signal 的排队状态

```
7 pub struct PendingSignals {
8     /// The pending signals.
9     ///
10    /// Note that does not correspond to `pending signals` as described in
11    /// Linux. `Pending signals` in Linux refers to the signals that are
12    /// delivered but blocked from delivery, while `pending` here refers to any
13    /// signal that is delivered and not yet handled.
14    pub set: SignalSet,
15
16    /// Signal info of standard signals (1-31).
17    info_std: [Option<Box<SignalInfo>>; 32],
18    /// Signal info queue for real-time signals.
19    info_rt: [VecDeque<SignalInfo>; 33],
20 }
```

信号状态分成进程级和线程级两层。进程级manager 保存共享pending queue、信号处置表、默认restorer 和线程manager 弱引用；线程级manager 保存本线程pending queue、blocked

mask、sigsuspend/sigtimedwait 临时mask 和 sigaltstack 配置。进程级信号先进入共享队列，再由未阻塞该信号的线程处理；线程级信号直接进入指定线程队列。

pending 队列结构的三个字段表达Linux 排队语义。标准信号用bitset 加单个siginfo 槽，同号标准信号已经pending 时会被合并；实时信号按信号号维护队列，每个实例都保留下来；set 记录哪些信号号当前可被取出。实时信号队列长度还用于发送上限检查，超限时用户态发送路径返回EAGAIN。

8.2.2 发送与权限

发送路径先解析目标，再决定写入进程队列或线程队列。tgkill 和tkill 解析可见TID，确认目标仍是线程，并在tgid 参数存在时校验线程属于同一线程组。普通kill 按pid 值区分单进程、当前进程组、指定进程组和广播发送。权限检查使用调用方ruid/euid 与目标uid/euid/suid 的关系；root 直接放行；目标已经成为zombie 时，内核读取退出快照中的uid，让kill 在退出竞态中仍保持Linux 兼容行为。

真正入队前还有两类控制动作。SIGCONT 会先恢复stopped 进程，ptrace stop 会把状态交给tracer 或父进程并中断同组线程；普通信号入队后，只中断当前未阻塞该信号的线程。这个选择很重要：被阻塞信号虽然已经pending，但唤醒阻塞该信号的线程不会产生投递，只会制造空转。

8.2.3 返回用户态前的投递

用户态返回前，线程级manager 先走快速路径：线程级和进程级possibly_has_signal 都为false 时直接跳过。慢路径构造当前可投递mask，先从线程pending queue 取信号，再从进程pending queue 取信号，然后读取进程级action 表决定默认动作、忽略、handler 或core dump。

handler 分支的提交点在signal frame 写入用户栈之后。内核先选择当前栈或altstack，写入ucontext、siginfo 和原用户上下文，再把用户态入口寄存器改为handler，参数指向signo、siginfo 和ucontext。随后根据action mask 更新blocked set；没有SA_NODEFER 时，当前信号也加入blocked set；带SA_RESETHAND 时，action 表恢复默认处置。若signal frame 写入失败，投递结果转成core dump 动作。

kernel 侧的投递收尾处理三件事：确认POSIX timer signal、把OS action 和handler 是否可重启交给restart tracker、执行terminate/core dump/stop/continue 等内核动作。handler 本身已经由signal frame 指向用户态函数，内核在该分支只完成上下文切换准备。

8.2.4 等待类系统调用

rt_sigprocmask 修改当前线程blocked mask，并保证SIGKILL 与SIGSTOP 不能被阻塞。rt_sigaction 修改进程级action 表，也拒绝改写这两个不可捕获信号。sigsuspend 和sigtimedwait 都会临时替换blocked mask：前者等待任意可投递信号并固定以EINTR 返回，后者等待指定集合并支持超时。等待结束或被信号打断后，旧mask 通过线程状态或

signal frame 恢复。

syscall restart 与信号投递共享同一个栈式状态机。被中断的syscall 先保存syscall 编号、参数和返回地址；信号投递结束时，restart tracker 根据OS action 和handler 的SA_RESTART 标志做出Pending、Restart 或Interrupt 决策；handler 通过sigreturn 返回时，tracker 校验正在恢复的用户上下文与保存的syscall 上下文匹配。匹配成功且决策为Restart 时，dispatcher 使用恢复后的上下文继续执行原syscall。

8.3 futex：地址key、等待队列与重启

8.3.1 地址到等待队列的映射

代码8.2 futex key 的两类地址

```

692 pub enum FutexKey {
693     /// A futex that is private to the current process.
694     Private {
695         /// The memory address of the futex.
696         address: usize,
697     },
698
699     /// A futex in a shared memory region.
700     Shared {
701         /// The offset of the futex within the shared memory region.
702         offset: usize,
703         /// The shared memory region.
704         region: Result<Weak<SharedPages>, Weak<()>>,
705     },
706 }

```

futex key 的任务是把用户地址映射成内核等待队列。private futex 使用当前进程虚拟地址，适合同一地址空间内的线程同步。shared futex 需要跨进程对齐同一共享位置：匿名共享映射使用SharedPages 弱引用，文件映射使用文件后端给出的句柄和偏移。VMA 不存在时先退回private key，后续用户内存读取会给出page fault 或EFAULT，因此key 构造本身不会提前吞掉用户内存错误。

FutexTable 将key 映射到FutexEntry，每个entry 持有一个等待队列。普通wake 等不跨阻塞点的操作使用借用型guard；多futex 等跨阻塞路径使用拥有型handle，保证entry 在等待future 存活期间不会被清理。shared futex 通过全局分表按共享区域指针归类，每100 次操作清理无外部引用且为空的分表，避免长时间运行后积累空表。

8.3.2 等待者生命周期

代码8.3 futex 等待队列和等待者状态

```

29 /// Wait queue used by futex.

```

```

30  #[derive(Default)]
31  pub struct WaitQueue {
32      gate: Mutex<()>,
33      queue: SpinNoIrq<VecDeque<Arc<SpinNoIrq<WaiterEntry>>>>,
34  }
35
36  struct WaiterEntry {
37      bitset: u32,
38      awakened: bool,
39      cancelled: bool,
40      owner: Weak<FutexEntry>,
41      task: WeakAxTaskRef,
42      waker: Option<Waker>,
43  }

```

等待队列有两层同步。外层gate保护涉及多个waiter的插入、wake和requeue操作；队列里的每个WaiterEntry再用自己的spinlock保存bitset、awakened、cancelled、owner、task和waker。这样的拆分让全队列操作保持原子性，同时允许单个waiter在取消或唤醒时独立更新状态。

等待路径的顺序决定了用户态锁不会丢唤醒。

1. syscall 入口先读取用户地址，若值已经不等于期望值，直接返回EAGAIN。
2. key和entry解析完成后，wait future在持有gate时再次读取用户地址。这个二次检查覆盖参数解析和真正入队之间的竞态。
3. 条件仍满足时，内核创建waiter，写入bitset、owner、当前task弱引用和waker，并把线程状态提示设为interruptible。
4. 普通timeout通过sleep future竞争；剩余时间不超过2ms的等待走短futex路径，在同一个future上忙等、检查signal interrupt和deadline，减少极短超时反复进入调度器的开销。
5. timeout、signal interrupt或future drop会触发取消。取消先标记waiter为cancelled，再通过owner弱引用找到当前所属entry并从队列移除；循环中重新读取owner，用来处理取消和requeue同时发生时waiter已经移动到另一队列的竞态。

8.3.3 wake、requeue和系统调用入口

wake/requeue的核心是对源队列中的waiter分类。已取消者直接丢弃；bitset命中且未超过唤醒上限者标记awakened、清除线程状态提示并收集waker；可移动且未超过requeue上限者改写owner后进入目标队列；其余waiter保留在源队列。真正唤醒waker的动作发生在释放gate之后，避免持锁唤醒任务。

requeue涉及源和目标两个等待队列。TheKernel按等待队列对象地址固定加锁顺序，地址较小者先锁，地址较大者后锁；源和目标相同时退化为单队列操作。这条规则把任意双队列requeue归约成一致的锁顺序，避免AB-BA死锁。

`sys_futex` 在入口处解析 `private flag`、`clock flag` 和主命令。`wait` 与 `wait-bitset` 会计算 `deadline`，带 `timeout` 时安装 `futex restart block`，保存地址、期望值、`bitset`、`deadline` 和时钟域；`wake` 按 `bitset` 唤醒指定数量 `waiter`；`requeue` 与 `cmp-requeue` 先做必要的值检查，再把 `wake` 数和 `requeue` 数交给等待队列。多 `futex wait` 为每个目标创建拥有型 `handle`，在各 `entry` 的 `gate` 下验证初始条件，聚合 `future` 返回第一个被唤醒的 `waiter` 索引。

`robust list` 处理线程持有用户态锁后退出的场景。退出路径读取用户注册的链表头，遍历每个节点，根据 `futex_offset` 找到锁字，只有 `owner TID` 匹配当前线程时才用 `CAS` 写入 `FUTEX_OWNER_DIED`。成功标记后唤醒一个等待者，用户态 `pthread` 库随后通过一致性修复接口决定如何恢复 `mutex` 状态。

8.4 时间：clockid、alarm 和文件描述符

8.4.1 clockid 和 time namespace

`clockid` 解码先处理 Linux 动态 CPU clock，再处理标准正 `clockid`。动态 `clockid` 用低位区分 `PROF`、`VIRT`、`SCHED` 和进程/线程目标，返回 `process CPU` 或 `thread CPU` 时间域；标准 `clockid` 覆盖 `realtime`、`coarse realtime`、`monotonic`、`coarse monotonic`、`boottime`、`process/thread CPU` 和 `TAI`。`coarse clock` 按 `tick` 分辨率量化，`fine clock` 对外报告 `1ns` 分辨率。

`time namespace` 只保存 `monotonic` 与 `boottime` 两个偏移量。用户态读取 `monotonic` 或 `boottime` 时，内核将 `host` 时间加上偏移；`timerfd` 或 `absolute clock nanosleep` 使用 `namespace` 内 `deadline` 时，需要反向换算成 `host deadline`，再交给 `alarm` 系统。这样，进程看到的是命名空间时间，内核队列维护的是统一 `host clock`。

8.4.2 CPU 时间、itimer 和 alarm 队列

`TimeManager` 挂在每个线程上，统计用户态和内核态 CPU 时间，并驱动三类 BSD `interval timer`。进入内核、返回用户态和上下文切换都会更新计时状态；`poll` 时按当前状态累加 CPU 增量，按 `wall delta` 推进 `real itimer`，并在 `RLIMIT_CPU sticky` 标志启用后检查 CPU 资源限制。软限制触发 `SIGXCPU`，硬限制触发 `SIGKILL`；提高软限制后状态会允许再次发送 `SIGXCPU`。

`itimer` 使用 `interval`、`remaining` 和 `sequence` 三个量。`sequence` 在设置或续期时递增，`alarm` 到期后通过 `sequence` 校验当前 `itimer` 是否仍是当初注册的那一代，过期 `alarm` 会被丢弃。通用 `alarm` 系统把到期动作和睡眠 `future` 分开：`realtime` 与 `monotonic` 各有一个按最早 `deadline` 排序的 `heap`，`entry` 的 `action` 可以是 `itimer` 轮询、`timerfd poll set` 唤醒或 `POSIX timer` 投递。新 `alarm` 早于当前堆顶时，通过 `event` 唤醒常驻 `alarm` 任务重新计算等待时间。

8.4.3 POSIX timer

代码8.4 POSIX timer 的提交状态

```

85  #[derive(Debug, Clone)]
86  pub(crate) struct PosixTimer {
87      pub clock: PosixTimerClock,
88      pub notify: PosixTimerNotify,
89      pub interval: Duration,
90      pub deadline: Option<Duration>,
91      pub sequence: u64,
92      pub overrun: i32,
93      pub signal_pending: bool,
94  }

```

POSIX timer 的状态记录时钟域、通知方式、重复间隔、下一次 deadline、sequence、overrun 和 pending 标志。创建时在进程的 timer slot 表里分配位置；设置时更新 interval、清零 overrun、清除 pending、递增 sequence，并按绝对或相对模式换算 deadline。只要 deadline 非零，就把 timerid、clock 和 sequence 注册进 alarm 系统。

到期处理先重新取得进程 timer slot，并校验 timer 仍存在、sequence 仍匹配、当前时间已经越过 deadline。随后根据 interval 计算本次跨过的 expiration 数。一次性 timer 清空 deadline 并递增 sequence；重复 timer 计算下一次 deadline、递增 sequence，并重新注册 alarm。通知方式决定是否真正投递信号：SIGEV_NONE 只维护计数；signal 通知在没有 pending signal 时写入 overrun 并设置 pending，已有 pending signal 时把额外到期次数累加到 overrun。信号确认路径会清除 pending 标志，使下一次到期可以继续投递。

8.4.4 timerfd

代码8.5 timerfd 的可读状态

```

63  struct TimerFdInner {
64      /// Number of expirations since last read.
65      expirations: u64,
66      /// Interval for repeating timers (Duration::ZERO = one-shot).
67      interval: Duration,
68      /// Absolute time of next expiration (None = disarmed).
69      next_expiration: Option<Duration>,
70  }

```

timerfd 将定时器暴露成 fd。内部状态保存累计 expirations、重复 interval 和下一次 absolute expiration。读、poll 和 gettime 都先按当前 host 时间惰性刷新 expirations；这样即使 alarm 唤醒被调度延迟，用户态读到的也是从上次读取以来真实跨过的 expiration 数。成功读取会写回 64-bit counter 并清零；重复 timer 在读后注册下一次 pollset alarm。

系统调用入口负责把 Linux flags 和 clockid 转成这个对象状态。timerfd_create 只接受 realtime、monotonic 和 boottime，按 flags 设置 nonblocking 与 cloexec。timerfd_settime 校验 flags，读取用户 itimerspec，把绝对 deadline 通过 timerfd clock 转成 host deadline，其中 monotonic 和 boottime 需要通过 time namespace 反向换算。disarm 时清空 interval 和 next expiration；rearm 时清零 counter 并注册 pollset alarm。

8.5 三类交叉场景

8.5.1 futex wait 被信号打断

1. 线程进入 `sys_futex`，带 `timeout` 的 `wait` 安装 `futex restart block`。
2. `futex wait` 再次验证用户地址值，构造 `key`，取得 `entry`，并把 `waiter` 放入等待队列。
3. 另一个线程发送信号，目标线程的 `interrupt poll` 让等待 `future` 返回 `Interrupted`。
4. `wait future drop` 时调用取消逻辑，从当前 `owner` 队列移除 `waiter`。
5. `dispatcher` 返回用户态投递 `signal`；`handler` 返回后，`restart tracker` 根据 `SA_RESTART` 和 `restart class` 决定是否恢复等待。
6. 重启时使用保存的绝对 `deadline`，继续按原来的超时边界等待。

8.5.2 POSIX timer 唤醒 `sigsuspend`

1. 进程创建 `signal` 通知的 `POSIX timer`，并把目标信号放入 `sigsuspend` 临时允许的集合。
2. `alarm` 到期后进入 `POSIX timer` 处理，写入 `overrun`，设置 `pending`，构造 `timer SignalInfo`。
3. 信号发送到进程或指定线程，目标线程在返回用户态前执行 `signal` 检查。
4. `signal frame` 建好后，`sigsuspend` 的返回值保持为 `EINTR`，原 `blocked mask` 通过 `signal frame` 恢复。
5. `handler` 或默认动作完成后，`POSIX timer` 的 `pending` 标志在信号确认路径中清除，下一次到期可以再次投递。

8.5.3 `timerfd` 和 `epoll`

1. `timerfd_settime` 将用户传入的 `value` 转成 `host deadline`，并注册 `pollset alarm`。
2. `epoll` 注册 `timerfd` 时，`timerfd` 把 `waker` 放入 `poll set`，并确保下一次 `deadline` 已注册到 `alarm` 系统。
3. `alarm` 到期后唤醒 `poll set`；随后 `timerfd poll` 惰性刷新 `expirations` 并报告可读。
4. 用户读 `timerfd`，内核写回 64-bit `expirations counter` 并清零；重复 `timer` 注册下一次 `alarm`。

8.6 与 StarryOS baseline 的设计差异

StarryOS baseline 已经具备信号等待、`signalfd`、`futex wait/wake/bitset/requeue`、`robust list`、基础 `clock` 和 `itimer`。TheKernel 的变化集中在控制面补强，主要体现在五个方面。

1. 信号与 `restart` 绑定。TheKernel 把 `signal handler`、非 `handler` 动作和 `restart_syscall` 决策收束到 `restart tracker`，带 `timeout` 的 `futex wait` 可以在 `handler` 返回后按原 `absolute`

deadline 继续。

2. **futex waiter 生命周期**。waiter 记录 owner、cancelled 和 task hint，requeue 与 cancel 通过 owner 弱引用协调；多 futex wait 使用拥有型 handle 保持 entry 生命周期。
3. **clockid 语义**。TheKernel 解析 Linux 动态 CPU clock，区分 coarse/fine 分辨率，并引入 time namespace offset；StarryOS baseline 的 clock 路径主要按固定 clockid 返回 wall 或 monotonic 时间。
4. **POSIX timer 和 timerfd**。StarryOS baseline 的 POSIX timer syscall 在 dispatcher 中仍是空返回，timerfd 属于 dummy fd；TheKernel 将 POSIX timer、timerfd 和 alarm 队列接到真实状态机和 poll 唤醒路径上。
5. **CPU 时间和资源限制**。per-thread TimeManager 区分用户态与内核态 CPU 时间，支持 itimer、process/thread CPU clock 和 RLIMIT_CPU 信号投递。

8.7 当前边界

1. 实时信号在单个 pending queue 内保持排队；不同线程队列之间的全局顺序没有额外串行化。
2. FUTEX_REQUEUE_PI 和优先级继承 futex 仍未实现，实时调度场景的优先级反转保护不完整。
3. CLOCK_MONOTONIC_RAW 当前沿用 monotonic 读数路径，没有单独隔离 NTP 频率调整。
4. adjtimex 保存 offset、frequency、status 等状态，但墙钟调整仍以直接设置为主，尚未实现渐进 slewing。
5. timer 精度受平台硬件定时器与调度延迟限制，纳秒级接口不代表实际唤醒一定达到纳秒级。

阶段	内核状态与分支	用户态结果
进入syscall	SavedSyscall 记录编号、参数和原始返回地址；只有可能阻塞或可能被信号影响的syscall 需要restart 跟踪。	正常完成时直接返回实现结果。
被信号打断	当前syscall 尚未提交最终结果时收到信号，内核把可恢复现场压入pending restart 状态。	先转入signal delivery，而不是立即丢弃syscall 上下文。
handler 执行	线程处在用户态handler/trampoline 路径；handler 返回后通过sigreturn 回到内核恢复现场。	内核重新判断该syscall 应重放还是返回错误。
重放路径	restart class 允许重启，或handler 设置与SA_RESTART 组合允许重启；内核恢复syscall 参数和架构PC。	用户态看到syscall 像没有被信号永久打断一样重新执行。
-EINTR 路径	restart class 禁止重启，或信号语义要求向用户暴露中断；内核写回 -EINTR，不恢复syscall 入口PC。	libc 或应用按Linux 语义处理中断返回。

图 8.1 RestartTracker：信号中断后的syscall 返回路径

栈区间	保存内容	恢复作用
高地址保护区	red zone 与ABI 对齐空间	给用户态handler 留出调用约定要求的栈空间，保证构造signal frame 后的栈指针仍满足架构对齐。
siginfo_t	信号编号、errno、code、fault address 等Linux siginfo 字段。	作为handler 第二个参数暴露信号原因，用户态无需再次查询内核。
ucontext_t	通用寄存器、PC、SP、浮点寄存器、sigmask 和用户栈描述。	sigreturn 读取这份上下文，把线程恢复到被信号打断前的执行点或restart 入口。
返回桩	用户态返回桩或SA_RESTORER 指定的restorer 地址。	handler 返回时跳入restorer，由它发起sigreturn syscall。
低地址填充	frame 尾部padding	维持16 字节栈对齐，避免handler 内部调用破坏ABI。
恢复路径	handler 执行结束后的内核入口	内核从ucontext_t 恢复寄存器和信号屏蔽字，再按restart 状态决定继续原PC 或重放syscall。

图 8.2 信号栈帧内存布局：siginfo → ucontext → trampoline → 对齐填充

Thread A: 等待者	FutexTable / 内核边界	Thread B: 唤醒者
原子读取futex_word, 用户态确认当前值仍等于val。	此时尚未入队, 唤醒端看不到等待者。	用户态修改同一futex_word, 准备执行wake。
进入FUTEX_WAIT 后, 内核在queue gate 下再次检查*addr= val。	二次检查与入队受同一队列边界保护, 避免检查后到入队前丢失wake。	进入FUTEX_WAKE, 按key 定位同一个wait queue。
条件仍满足时创建WaiterEntry, 记录bitset、task weak ref 与waker。	key=uaddr+PRIVATE/SHARED; 队列保存等待者并支持取消、requeue 与bitset 匹配。	扫描队列, 选择最多n 个匹配bitset 的等待者。
线程状态提示切到interruptible, 阻塞等待signal、timeout 或wake。	被唤醒或取消时从队列移除waiter, 并清理线程状态提示。	设置awakened, 触发waker, 返回实际唤醒数量。

图 8.3 Futex Wait/Wake 双线程交互序列与FutexTable 等待队列

第九章 跨架构差异处理：RISC-V 与 LoongArch

9.1 同一套Linux ABI 落到两种trap 机器上

TheKernel同时面向RISC-V64 和LoongArch64。Linux 用户态看到的是同一组进程、地址空间、fd、signal、futex 和time 接口；内核进入点却受到架构约束：trap frame 字段不同，syscall 号和time64 alias 不同，页表元数据和TLB flush 指令不同，QEMU 设备总线和构建产物也不同。

源码里的处理方式分为三层。

1. **syscall 与用户态上下文层**。kernel crate 通过 `cfg(target_arch)` 选择编号别名、异常处理和必要的寄存器保存路径。
2. **patch crate 层**。axcpu 和page-table patch crate 把trap frame、用户态上下文和页表操作封装成上层可共享的接口。
3. **构建层**。Makefile生成不同的evaluator kernel 和辅助磁盘镜像，并给RISC-V 指定mmio virtio 配置。

9.2 syscall 编号和重启边界

9.2.1 指令长度与restart

代码9.1 syscall 重启指令长度

```
81  #[cfg(any(target_arch = "riscv64", target_arch = "loongarch64"))]
82  const SYSCALL_INSN_LEN: usize = 4;
83  #[cfg(target_arch = "x86_64")]
84  const SYSCALL_INSN_LEN: usize = 2;
85
86  impl SavedSyscall {
87      fn capture(uctx: &UserContext) -> Self {
88          let return_ip = uctx.ip();
89          Self {
90              sysno: uctx.sysno(),
91              args: [
92                  uctx.arg0(),
93                  uctx.arg1(),
94                  uctx.arg2(),
95                  uctx.arg3(),
96                  uctx.arg4(),
97                  uctx.arg5(),
98              ],
99          }
100      }
101  }
```

```

99         return_ip,
100         restart_ip: return_ip.saturating_sub(SYSCALL_INSN_LEN),
101     }
102 }
103
104 fn restore(self, uctx: &mut UserContext) {
105     uctx.set_sysno(self.sysno);
106     uctx.set_arg0(self.args[0]);
107     uctx.set_arg1(self.args[1]);
108     uctx.set_arg2(self.args[2]);
109     uctx.set_arg3(self.args[3]);
110     uctx.set_arg4(self.args[4]);
111     uctx.set_arg5(self.args[5]);
112     uctx.set_ip(self.restart_ip);

```

syscall 被信号打断后, restart tracker 需要把 PC 回退到 syscall 指令起点。RISC-V 与 LoongArch 的 syscall 指令长度都是 4 字节, x86-64 的 syscall 指令长度是 2 字节; 保存点记录的是异常返回后的 PC, 因此 capture 阶段用架构常量计算 restart PC。后续 restore 会写回 syscall number、六个参数和 restart PC, 使用户态重新执行同一条 syscall 指令。

这个边界放在 restart tracker 内, 而没有散落在各个 syscall 实现中。futex、nanosleep、socket 等阻塞路径只需要描述自己是否可重启以及 restart block 的内容; 真正的 PC 回退、参数恢复和 restart_syscall 编号替换由同一处代码完成。

9.2.2 LoongArch 的 time64 alias

dispatcher 中的主实现只维护一套时间逻辑。LoongArch 额外暴露 clock_gettime64、clock_gettime64、clock_getres_time64、semtimedop_time64、recvmsg_time64 和 timerfd time64 入口时, 分发表把这些编号映射到同一个实现函数。这样, 架构差异停留在 syscall number 层, 时间、socket 和 SysV semaphore 子系统不需要复制逻辑。

这类 alias 的主要风险是遗漏新编号。后续新增架构 syscall 时, 需要先落在 dispatcher 表, 再复用既有子系统入口; 如果在时间或网络子系统内部写架构分支, 后续维护会出现两套语义。

9.3 用户态上下文和trap 返回

9.3.1 RISC-V 与 LoongArch 的上下文差异

RISC-V 的用户态上下文包装 TrapFrame, 创建时写入 a0、用户栈顶和 sepc, 并设置 sstatus 中的中断与用户内存访问位。用户态 ecall 返回后, run() 将 sepc 加 4 并报告 syscall; load/store/instruction page fault 分别转成统一的读、写、执行 page fault; 其他异常保存 scause 和 stval 供上层映射信号。

LoongArch 使用相同的 UserContext 抽象名, 但底层字段是 era、prmd、badv 和 badi。syscall 异常返回时 era 加 4; page fault 来源来自 badv, 触发指令来自 badi。这使上层看到的 ReturnReason 与 RISC-V 一致, 底层仍保留 LoongArch 的异常细节, 后续 misaligned

emulation 和 core dump 可以读取架构特有信息。

9.3.2 kernel 用户态主循环

kernel 层只面向 ReturnReason。syscall 返回进入 dispatcher；page fault 进入地址空间处理，失败时转成 SIGBUS 或 SIGSEGV；interrupt 触发调度；exception 按统一类别转成 SIGTRAP、SIGILL、SIGBUS 或 SIGSEGV。LoongArch 的 misaligned exception 会先尝试软件模拟，成功后直接继续用户态。

LoongArch 在用户态运行前后显式 restore/save 当前任务的 FPU 状态，并在 fork、exec 等路径中提供复制和重置辅助。RISC-V 侧由 axcpu 的 trap frame 与 fp-simd feature 处理基础状态。这里的可见语义是信号、exec 和上下文切换不会把另一个任务的浮点寄存器暴露给当前任务；完整 signal frame 中浮点状态覆盖仍需要专门测试。

9.4 页表 patch 层

RISC-V 和 LoongArch 的页表差异被封装在 page-table patch crate 的 metadata 中。RISC-V 提供 Sv39 和 Sv48 metadata：Sv39 是三层页表、39 位虚拟地址，Sv48 是四层页表、48 位虚拟地址，TLB flush 通过 sfence.vma 完成。TheKernel 当前 RISC-V evaluator 路径落在 Sv39 类型上，上层 AddrSpace 只依赖统一的 cursor、query、map 和 unmap 接口。

LoongArch 使用四级 LA64 页表，metadata 给出 48 位物理地址和 48 位虚拟地址。TLB flush 由 dbar 和 invtlb 组成：带地址时清理对应 VA 的非全局条目，不带地址时清理全部条目。页表结构差异留在 patch crate 中，内存管理章节里的 backend、VMA 和 page fault 逻辑因此可以共享。

这个分层也决定了错误归属。地址空间层负责 VMA、backend、COW、文件映射和 page fault 结果；架构 metadata 负责页表层数、地址位宽、PTE 格式和 flush 指令。跨架构移植时，应优先检查 metadata 与 trap 返回是否正确，再判断上层内存策略是否有缺陷。

9.5 构建产物和运行入口

Makefile 将两个 evaluator 目标拆成显式产物：

```

1 kernel-rv:
2     $(MAKE) -C make ARCH=riscv64 BUS=mmio defconfig
3     $(MAKE) -C make ARCH=riscv64 BUS=mmio build-elf-fast
4     ...
5     cp "$$kernel" "$$(ROOT_DIR)/kernel-rv"
6
7 kernel-la:
8     $(MAKE) -C make ARCH=loongarch64 defconfig
9     $(MAKE) -C make ARCH=loongarch64 build-elf-fast
10    ...
11    cp "$$kernel" "$$(ROOT_DIR)/kernel-la"

```

RISC-V 的build path 显式设置BUS=mmio，对应QEMU virt 平台上的virtio-mmio 设备；LoongArch 走默认平台配置，对应virtio-pci。顶层artifacts 目标同时生成kernel-rv、kernel-la、disk.img、disk-rv.img 和disk-la.img，使评测入口可以按架构选择内核和辅助磁盘。

9.6 当前边界

- 1. RISC-V 与LoongArch 的用户态上下文已经统一为ReturnReason，但signal frame 中浮点状态的架构覆盖仍应通过专门用例验证。
- 2. LoongArch 页表元数据已经接入四级页表和TLB flush，硬件支持的更多页表特性尚未全部反映到上层内存策略。
- 3. time64 alias 当前集中在dispatcher，后续新增架构syscall 编号时仍需要在分发表中显式落点，避免子系统内部出现架构分叉。

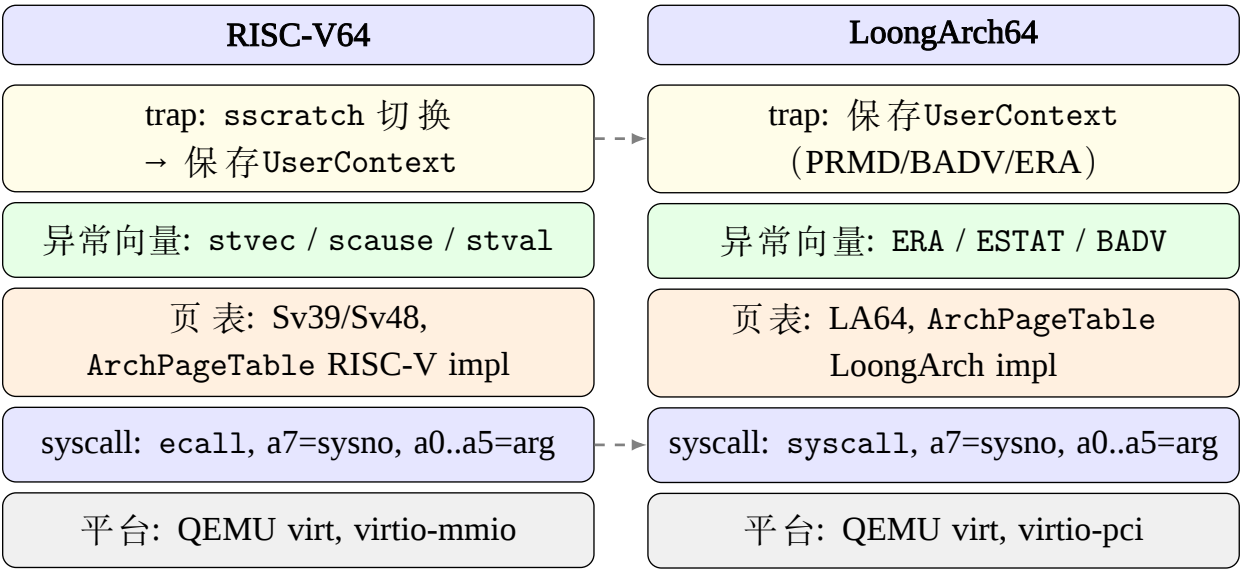


图 9.1 RISC-V64 与LoongArch64 的对应层：trap 上下文、页表、syscall 入口与平台

第十章 AI 工具使用与工程责任边界

TheKernel开发过程明确区分AI工具的使用范围、人机分工、验证方式和错误纠正责任。AI工具参与了代码生成、代码审查、测试脚本编写和日志分析；架构取舍、根因判断、接口约束、验收标准和最终代码责任由开发者承担。

10.1 使用的工具

开发过程中主要使用三类工具：

1. **Claude Code**。用于在开发者给出的模块边界、语义约束和验收条件下生成Rust代码，运行局部验证，并根据失败日志迭代修复。
2. **Codex CLI**。用于独立代码阅读、交叉审查、文档整理和备选实现检查，避免单一路径中的遗漏长期保留。
3. **Superpowers** 和 **OpenSpec**。用于记录变更目标、任务拆分、设计约束和验收项，使开发过程从临时对话转成可复查的工程记录。

这些工具的输出不会直接成为最终依据。每次进入核心路径的改动都需要经过源码审查、构建、测试或针对性回归检查。

10.2 任务范围

AI工具主要参与以下工作：

1. 查阅Linux ABI、RISC-V与LoongArch调用约定、ArceOS和StarryOS接口和LTP用例行为，形成候选实现路径。
2. 根据既定架构生成结构体、trait实现、系统调用分支、测试脚本和小型辅助工具。
3. 阅读失败日志，定位触发路径，提出最小修复方案。
4. 对关键路径做第二视角审查，重点检查并发、生命周期、用户指针访问、权限检查和跨模块状态一致性。
5. 将阶段性方案整理为OpenSpec和Superpowers记录，便于后续回看设计约束和验收标准。

上述任务都处于开发者定义的范围之内。AI可以给出候选实现，不能自行改变子系统边界、绕过验收条件或以临时workaround掩盖失败。

10.3 人机分工

TheKernel的核心架构由开发者确定，AI 工具在明确约束下完成具体实现和验证辅助。
表 10.1 按子系统概括分工。

表 10.1 AI 工具参与范围与人工责任边界

子系统	AI 参与内容	人工责任	验证方式
task 模型	结构体与方法实现	ProcessData 与 Thread 拆分、clone 语义映射	clone、fork 回归
地址空间	backend 实现、page fault 分支	backend 抽象边界、COW 与 VMA 语义	mmap、mprotect、COW 用例
文件与 fd	FileLike、fd table、特殊 fd	三层 fd 模型、打开描述所有权	fs、epoll、fcntl 用例
网络栈	socket transport、option 分支	axnet-ng 分层、监听表和 namespace 边界	loopback 与网络用例
信号与同步	signal、futex 路径实现	restart 状态机、TOCTOU 防护	signal、futex 回归
跨架构支持	条件编译分支与适配代码	HAL 抽象边界、双架构对齐策略	双架构构建与运行

几个核心决策体现了这种分工。

1. task state 拆成进程共享对象和线程局部对象，是为了表达Linux clone flags 对共享资源和线程私有资源的不同要求。
2. 地址空间后端使用trait 边界，是为了让COW、file、shared anonymous 和linear mapping 各自维护缺页填充、fork 和权限校验逻辑。
3. 在StarryOS 基础上演进，是为了复用ArceOS 和StarryOS 已有runtime、调度、页表、信号和文件基础，把主要精力放到Linux 兼容语义补齐上。

这些决策由开发者给出，AI 工具负责在决策边界内展开代码和测试。

10.4 错误发现与纠正

AI生成的实现需要经过失败日志和源码审查校正。开发过程中出现过多类典型错误，修复流程均以根因定位为先。

1. fork 路径的早期实现没有在文件映射present page 共享前处理页内容一致性，导致子进程可能观察到错误内容。修复时先定位到clone_map 的映射时机，再约束AI 修改COW 与file backend 交互。

2. `futex` 等待路径早期实现只在 `syscall` 入口检查用户值，忽略了检查到入队之间的竞态。修复要求在持有等待队列 `gate` 后再次读取用户地址，保持 `check-and-sleep` 原子边界。
3. 某些兼容 `syscall` 初版只补了编号或空返回，无法满足用户态可观察语义。修复时先读 `LTP` 触发条件，再把实现接到真实内核对象和错误码路径上。

这个流程的重点是先判断失败属于设计边界、状态生命周期还是局部代码错误。AI 工具在明确修复方向后执行代码修改和回归验证。

10.5 验证和追溯

项目使用以下方式约束 AI 产出：

1. 关键改动进入源码前先明确模块边界、锁顺序、错误码语义和用户态可观察行为。
2. 每个实现切片至少经过构建、局部测试、`LTP` 子集或针对性回归之一；跨模块改动需要扩大验证范围。
3. `OpenSpec` 和 `Superpowers` 记录用于保存目标、约束、验收和证据；`git` 历史记录最终代码变化。
4. `AGENTS.md` 约束工程操作，例如优先使用 `dev-shell` 或 `container` 验证、不提交生成 `artifact`、不用临时修复隐藏问题。

10.6 团队掌握程度

团队能够独立解释并修改最终提交的核心设计：

1. 能说明进程、线程、地址空间、`fd`、`socket`、`signal`、`futex`、`timer` 等对象的所有权边界。
2. 能解释 `fork`、`exec`、`clone`、`page fault`、`signal delivery`、`futex wait`、`timerfd poll` 等跨子系统路径。
3. 能根据失败日志定位到对应内核对象和锁边界，并判断修复应落在 `syscall` 入口、对象模型还是 `backend` 实现。
4. 能在 `RISC-V` 与 `LoongArch` 两个架构上检查 `syscall` 编号、`trap` 上下文和构建产物差异。

因此，AI 工具在本项目中承担执行和辅助审查角色；系统设计、验收标准和最终责任由开发者掌握。

环节	人类责任	AI / 工具链责任	进入下一步的条件
定义边界	给出验收标准、禁止项、工作目录和质量基准。	按边界读取仓库、spec、skill 与现有实现。	任务范围可验证，且不影响主工作树。
生成变更	判断目标行为和根因，选择保留、修复或重写。	生成补丁、排版调整或验证脚本。	变更能解释对应问题，不引入临时遮蔽。
构建验证	判定构建、日志、截图和源码证据是否满足验收。	运行构建、扫描日志、导出 PDF 页面并暴露失败现象。	没有新的页面溢出、遮挡、缺字或错误引用。
失败回路	根据现象重新收窄根因或追加约束。	在同一边界内修复候选补丁并重新验证。	失败原因已经被关闭，或明确记录为后续工作。
合并责任	最终决定是否接受结果并进入下一任务。	保留可复用经验到 skill，避免同类问题反复出现。	产物可被读者直接使用，且经验可复查。

图 10.1 人在环验证闭环：人类定义验收标准与根因，AI 在边界内生成与修复

第十一章 边界、局限与未来工作

11.1 系统调用覆盖边界

TheKernel的系统调用覆盖优先服务用户态自举、基础进程模型、内存管理、文件描述符和最小网络闭环。部分接口已经有入口、状态对象或兼容返回值，但没有把Linux 的完整执行链路接到真实资源控制或内核异步执行上。表 11.1 列出这些兼容边界。

表 11.1 系统调用兼容边界

接口面	当前状态	影响
io_uring	已建立ring mmap、buffer register、READ_FIXED 和SENDMSG 的同步提交路径；内核异步worker、丰富opcode 与completion 等待语义仍未打通	依赖io_uring 高并发I/O 的程序会退回同步或有限兼容路径
quota 管理	quotactl 和quotactl_fd 维护内存态VFS 与XFS quota 记录；文件系统分配路径尚未按quota 记录强制扣账	配额工具可读写部分状态，实际磁盘空间限制仍不生效
seccomp	prctl(PR_SET_SECCOMP) 可记录strict 和filter 模式并检查权限；独立seccomp syscall 仍返回dummy 成功，filter 执行路径未接入syscall dispatcher	seccomp filter 对syscall 放行或拒绝没有实际约束
cgroup	pseudofs 支持cgroup.procs、pids.max、cpu.cfs_quota_us 等文本控制文件；CPU、内存和I/O controller 尚未驱动调度或分配决策	cgroup 可见状态存在，资源限制对运行时行为影响有限
BPF	verifier、map、prog load、test run 和对对象查询已有实现；程序attach 到网络、tracepoint 或LSM 等实际hook 的链路仍不完整	BPF 可作为对象模型和解释器验证，数据路径能力不足

11.2 任务与调度边界

任务与调度子系统已经具备ProcessData 与Thread 拆分、clone 和vfork、基本调度属性和ptrace 控制对象。边界集中在flag 组合、调度策略深度和事件链完整性上。

1. **clone flag 组合覆盖不足。** `do_clone` 已经处理 `pidfd`、`cgroup charge`、`vfork` 等核心分支，但 Linux clone flags 的组合空间很大。罕见组合，例如 `CLONE_NEWPID` 与 `CLONE_THREAD` 并用、`CLONE_PARENT` 与 `CLONE_THREAD` 并用，还需要专门用例验证父子关系、PID namespace 和 wait 语义。
2. **调度策略停留在属性兼容层。** `SchedState` 能保存 nice、RT priority、reset-on-fork 等属性，系统调用层也能解析多类 Linux policy。deadline policy 的 EDF 行为没有实现，CFS 的公平性主要来自 patched CFS queue，完整 Linux CFS 的红黑树加权、最小粒度和带宽控制仍缺失。
3. **vfork 依赖用户态约束。** `begin_vfork` 会阻塞父线程，`execve` 或进程退出时通过 `release_vfork` 唤醒父线程。子进程在 `exec` 前仍共享父进程地址空间，内核不能隔离它对用户栈和数据页的写入；这一路径依赖用户态遵守 `vfork` 后尽快 `exec` 或 `exit` 的约束。
4. **ptrace 事件链不完整。** `ptrace` 已支持 attach、seize、peek、poke、continue、interrupt 和 options 记录；`PTRACE_GETREGSET`、`PTRACE_SETREGSET`、`siginfo` 等接口仍返回错误。syscall entry 与 exit stop、exec trap、clone、fork、vfork event、`PTRACE_O_TRACESYSGOOD` 和 restart tracker 的组合还需要补齐。

11.3 内存管理边界

内存管理子系统能支撑基本的进程地址空间、按需分页与 COW，但在并发正确性、竞争语义与若干高级特性上仍有未闭合之处。

1. **COW 并发 fault 覆盖不足。** COW backend 通过 frame table 和 per-frame lock 保护引用计数，页表修改也经过地址空间路径串行化。仍需专门压力用例覆盖同一物理页上两个线程先后进入写 fault、复制私有页、提交 remap 和释放旧帧的并发窗口，图 11.1 展示了需要验证的顺序关系。
2. **文件映射的 truncate race。** 文件在映射后被 truncate 缩短时，已映射但尚未 fault 的页在后续访问应触发 SIGBUS (EOF)；该行为尚未完整验证，truncate 与 fault 之间的时序竞争未构造专门用例。
3. **mremap 迁移失败组合。** `sys_mremap` 已经在多处失败分支清理目标区间，例如 relocated segments 映射失败后会 unmap 目标范围。fixed remap、locked range、扩张尾段和旧区间 unmap 失败之间的组合仍需要事务化测试覆盖。
4. **userfaultfd 功能面较窄。** `userfaultfd` 已支持创建 fd、UFFDIO_API、missing range register、page fault event 读取和 UFFDIO_COPY。写保护模式、range unregister、fork 继承与复杂 poll/退出交互尚未实现完整 Linux 行为。

11.4 文件与网络边界

文件系统与网络栈在常规路径上可工作，但在边界组合与压力场景下的行为仍需进一步验证。

1. **路径解析的组合竞争**。路径解析在symlink 循环、rename 与unlink 竞争、mount propagation 的组合场景下可能出现未定义或不一致的行为。单一路径已测试，但多源竞争与传播级联的组合未穷举。
2. **fanotify 的双向交互**。fanotify 已经实现init、mark、事件队列、permission event 和 response 写回。需要继续验证监控者阻塞、事件队列溢出、permission fd 生命周期，以及exec 与open 同时推进时的超时与回退语义。
3. **部分socket 与协议不完整**。IPv4 与IPv6 RAW socket 目前返回protocol-not-supported 错误。socketpair 覆盖Unix stream、datagram 和seqpacket。RDS 复用UDP 数据路径处理OSComp 和LTP 局部用法，协议细节仍不完整。这些缺口主要影响诊断类和底层网络工具。
4. **网络栈压测有限**。网络栈依赖loopback 进行最小闭环测试；外部NIC 路径（通过virtio-net）在QEMU 下可工作，但未经大量网络负载的压测，吞吐与并发稳定性尚无量化结论。

11.5 并发与可靠性

TheKernel在内核路径中使用SpinNoIrq、Mutex、RwLock 等同步原语，但以下并发场景尚未充分验证，存在可靠性风险：

1. **多线程exec 的协调**。exec_ctl 通过终止其他线程来完成多线程exec，但若某线程正持有内核锁（例如fd table 写锁），强制终止可能导致死锁。终止路径未与锁持有状态做联动检查。
2. **fd table 的并发修改**。多线程对同一fd table 的并发修改：fork 与close 同时发生时，fd table 复制的一致性依赖复制点的原子选取，极端并发顺序下可能出现句柄泄漏或重复。
3. **signal 与阻塞syscall 的race**。信号在syscall 进入等待之后、但在waker 注册之前到达时，可能导致永久睡眠。该窗口即经典的「信号丢失」问题，本实现未在所有阻塞路径上统一处理。
4. **futex 组合路径验证不足**。单futex wait 在进入队列前会通过wait_if 在gate 下重读用户值，覆盖基础wake-before-sleep 窗口。waitv、多futex requeue、robust-list 退出清理和信号重启组合仍需要系统性覆盖。

11.6 未来工作

后续工作应优先闭合已经有对象模型但行为链路不足的路径：

- 1. 验证并收紧COW per-frame 锁的覆盖范围；如果后续改为CAS 或更短临界区，也必须把“检查—分配—remap—减引用”的提交关系作为一个可验证的事务边界。
- 2. 实现futex 的优先级继承，解决实时线程的优先级反转问题。
- 3. 将pseudofs cgroup 兼容层连接到实际的内存、CPU、I/O 资源限制代码，使配额真正影响调度与分配。
- 4. 实现submission queue 和completion queue 的内核异步执行路径，打通SQE 到CQE 的完整提交—完成链路。
- 5. 将syscall entry 与exit stop、signal delivery、restart tracker 的交互完整实现，补齐LTP ptrace 用例覆盖。
- 6. 当前实现面向QEMU 单核环境（smp=1）；多核扩展需重新审查所有全局可变状态的并发访问，排查数据竞争。

窗口	并发情形	需要验证的规则
共享起点	两个线程映射同一只读物理页，并先后对该页写入，触发COW fault。	FRAME_TABLE 中存在同一个物理页的引用计数对象，初始计数反映共享映射数量。
进入复制路径	Thread 1 取得该帧的FrameRefCnt 锁并看到refcount > 1；Thread 2 即使已经fault，也必须在同一个帧锁之后读取refcount。	不能让两个线程同时依据同一个旧refcount 做复制和减引用决策。
提交私有页	Thread 1 分配新帧、复制旧帧内容、remap 自己的PTE，然后递减旧帧引用计数；Thread 2 等待帧锁，或在Thread 1 提交后重新检查。	copy、PTE remap 和旧帧refcount 递减必须在同一个帧锁覆盖范围内完成。
第二次检查	Thread 2 取得帧锁后若看到refcount = 1，只升级自己的PTE；若仍 > 1，才继续复制。	第二个线程必须基于更新后的refcount 决策，避免重复复制、过早释放或refcount 下溢。
释放边界	旧帧refcount 降到0 时触发drop-frame，后续访问不应再通过旧表项找到已释放物理页。	删除FRAME_TABLE 表项必须早于物理帧释放，防止同一物理地址被重新分配后仍命中旧引用计数。

图 11.1 COW 并发写 fault 的验证窗口：两线程对同一共享页的复制与提交顺序

第十二章 QEMU 单核环境下的热路径开销

12.1 评测压力

TheKernel的性能工作主要面向OSComp/LTP 运行环境：内核在QEMU TCG 模拟器中运行，配置以单核为主，测试结果受wall-clock 超时约束影响很大。这个环境对常数项非常敏感。一次匿名缺页中的清零循环、一次页表修改后的TLB 维护、一次空syscall 的通用记账、一次文件顺序读取的设备请求，都可能在真实硬件上只是局部开销，却在TCG 翻译和单核运行下被放大成测试是否超时的差异。

当前源码中仍然可见的热路径机制包括匿名页清零、文件页缺页裁剪、地址查找、syscall fast path 和按功能触发的轻量化分支。历史上出现过但当前树已经回退或不再成立的做法，例如把COW FRAME_TABLE 分成64 个shard、用FileFast 绕过vtable、把pipe 写成lock-free atomic ring，只作为背景处理。

这些机制分为两类问题。第一类是缺页、地址查找和syscall 入口这类所有程序都会经过的默认路径，优化重点是减少每次必付的工作量。第二类是RLIMIT_CPU、userfaultfd、socket burst 等按功能触发的路径，优化重点是让未启用功能的常见情况保持轻量，同时保证功能真正启用后回到完整语义。

12.2 缺页路径

匿名页 fault 的第一项固定成本是分配后清零。这个清零不能取消，因为新页可能被用户态立即读取；内核必须保证页内没有旧物理页残留。当前实现把这项工作留在frame 分配点完成，并用u64 store loop 清空整页。源码注释记录了这个选择的测量背景：在QEMU TCG 中，编译器生成的memset 路径会产生更多store，匿名页 fault 的每页成本约从170 微秒降到43 微秒。

代码12.1 匿名页清零循环

```
28 fn alloc_frame(zeroed: bool, size: PageSize) -> AxResult<PhysAddr> {
29     let page_size = size as usize;
30     let num_pages = page_size / PAGE_SIZE_4K;
31     let vaddr =
32         VirtAddr::from(global_allocator().alloc_pages(num_pages, page_size, UsageKind::VirtMem)?);
33     if zeroed {
34         // Zero with a u64 store loop instead of core::ptr::write_bytes. Under
35         // QEMU TCG the compiler-emitted memset for write_bytes runs ~4x slower
36         // here (it issues ~4x more stores), which made anonymous page faults
37         // (stack/heap/malloc) dominate at ~170us/page. The u64 loop cuts that
```

```

38 // to ~43us. page_size is always a multiple of 8 (4K/2M).
39 let p = vaddr.as_mut_ptr() as *mut u64;
40 for i in 0..(page_size / 8) {
41     unsafe { *p.add(i) = 0 };
42 }

```

这个代码块给出的是模拟器环境下的常数项控制。真实硬件上的memset可能由向量化或专用指令加速；在TCG中，减少被翻译和执行的store数量更直接影响wall-clock时间。这里的安全边界也很窄：只有zeroed为真时才写零，页大小保证按8字节对齐，最后仍然把同一个物理页交给页表映射。优化改变的是清零方式，没有改变“匿名页公开前必须为零”的Linux可见语义。

文件映射的COW fault做了第二层裁剪。文件内容会覆盖页内主体区域，整页预清零会变成先写零、再写文件数据的重复工作。当前alloc_new_at先根据backend是否带文件对象决定frame是否预清零：匿名页仍走预清零，文件页则延迟到填充阶段处理。随后它计算VMA起点和fault地址之间的head gap、文件可读范围以及页尾tail gap，只对gap写零，文件覆盖区间直接由read_at填充。这样保留了mmap的安全边界：页内不属于文件内容的字节仍然为零，文件内容本身只写一次，page table commit发生在填充完成之后。

缺页路径的风险主要在提交顺序。frame分配和内容填充失败时，页表还没有把这个frame暴露给用户态；只有pt.map成功后，backend才标记为materialized。这个事务边界使性能裁剪不会把半初始化页面提交出去。相关实现位于kernel/src/mm/aspace/backend/mod.rs的frame分配函数和kernel/src/mm/aspace/backend/cow.rs的alloc_new_at。

12.3 地址空间查找与TLB 维护

mmap的非固定地址选择会频繁查找空闲VMA。当前find_kernel_area先尝试caller hint；hint不可用时，优先在地址空间高水位附近追加放置；最后才回到从地址空间基址开始的完整first-fit扫描。这个顺序适合测试程序常见的增长形态：动态库、堆扩展和匿名映射往往连续占用高地址附近区域，append placement可以避免每次都从低地址VMA重扫。语义上，hint仍然被尊重，fallback仍然执行完整空洞搜索，因此它只改变查找顺序，不改变MAP_FIXED之外的地址选择约束。

TLB维护的成本由page table cursor控制。普通cursor在drop时执行必要flush，适合修改当前可能正在运行的页表；cursor_no_flush只用于构造或修改尚未激活的页表。这个接口把性能收益绑定到调用方责任上：loader和地址空间clone可以在目标页表切换前批量写PTE，跳过对当前CPU没有意义的flush；一旦页表可能已经被CPU使用，就必须回到会flush的cursor。

代码12.2 非活跃页表游标

```

139 /// Gets a cursor to modify an inactive page table.
140 ///

```

```

141  /// Callers must ensure the page table is not active on any CPU while the
142  /// cursor exists; otherwise required TLB invalidations may be skipped.
143  pub fn cursor_no_flush(&mut self) -> PageTable64Cursor<'_, M, PTE, H> {
144      PageTable64Cursor::new_no_flush(self)

```

这段源码只保留了最关键的契约：调用者必须保证页表不在任何CPU上活跃。优化点不在于减少一次函数调用，而在于避免把未激活页表的构造成本错误地算到当前地址空间的TLB同步上。地址选择实现位于`kernel/src/mm/aspace/mod.rs`，页表cursor实现位于`third_party/rust-patches/page_table_multiarch/src/bits64.rs`。

12.4 syscall 与时间读取

syscall dispatcher的热路径有两层提前返回。第一层处理`getpid`、`gettid`、`getuid`等只读当前线程状态的getter。这些调用不访问用户缓冲区、不阻塞、不进入文件系统和调度器，也没有可重启语义；把它们放在dispatcher开头，可以跳过`enter_syscall`、restart tracker、timer state 切换等通用bookkeeping。

代码12.3 只读getter的syscall 快路径

```

114  /// Fast path for trivial getter syscalls that only read from the current
115  /// thread and return a scalar: no user copy, no restart, no blocking, no
116  /// timer accounting. Handling them here skips the per-syscall
117  /// enter_syscall/set_timer_state/restart bookkeeping, cutting the null-syscall
118  /// latency that lmbench measures. Signal delivery and preemption run in the
119  /// user-mode loop after handle_syscall returns, so they remain correct.
120  fn fast_path_getter(sysno: Sysno) -> Option<AxResult<isize>> {
121      Some(match sysno {
122          Sysno::getpid => sys_getpid(),
123          Sysno::getppid => sys_getppid(),
124          Sysno::gettid => sys_gettid(),
125          Sysno::getuid => sys_getuid(),
126          Sysno::geteuid => sys_geteuid(),
127          Sysno::getgid => sys_getgid(),
128          Sysno::getegid => sys_getegid(),
129          _ => return None,
130      })

```

第二层处理`gettimeofday` 和非CPU-time 的`clock_gettime`。这类调用只读取wall-clock 或monotonic clock，不需要阻塞等待，也不会因为信号重新启动。进程CPU-time 和线程CPU-time clock 继续走完整路径，因为它们依赖精确的CPU 时间统计，必须和timer accounting 保持一致。

快路径没有绕过用户态返回前的全局语义。信号递送、抢占检查和返回值写回仍然发生在`handle_syscall` 返回后的user-mode loop 中；快路径跳过的是当前syscall 内部不需要的记账，而不是跳过内核到用户态的收尾协议。相关实现位于`kernel/src/syscall/mod.rs`。

12.5 条件关闭的慢路径

有些Linux 兼容功能只在程序显式启用后才需要付出成本。当前源码用sticky flag 或全局存在位把这类慢路径从默认fault/syscall 路径上移开。这里的设计目标是让“从未使用该功能”的常见情况只付出一次原子读或一次分支；功能一旦被启用，后续路径回到完整检查，避免用静默跳过换取速度。

RLIMIT_CPU 是一个例子。默认情况下没有进程设置有限CPU 时间上限，update_rlimit_cpu 可以直接返回，不需要获取当前线程、克隆进程数据引用、读取rlimit 表并计算累计CPU 时间。一旦某个进程设置了有限上限，ANY_RLIMIT_CPU_SET 会保持为真，后续路径重新执行完整检查。这个flag 是sticky 的，所以不会出现“某个进程启用了限制，其他路径仍按未启用处理”的窗口。

userfaultfd 使用同样的思路。page fault 路径本来可能需要扫描全局userfaultfd 列表，把缺页事件投递给注册者并等待用户态解决；大多数测试程序从未创建userfaultfd。当前实现用USERFAULTFD_EVER_CREATED 记录是否出现过userfaultfd 对象。该位为false 时，缺页路径直接返回普通缺页处理，跳过全局mutex 和弱引用列表扫描；第一次创建userfaultfd 后，存在位变为true，后续fault 才进入匹配注册区间、排队、等待resolved page 的完整流程。

这类开关适合用于单调状态：从未启用已启用后不再回退。它们不适合表达频繁开关的per-process 状态，否则会把全局fast path 和局部语义混在一起。相关实现位于kernel/src/task/timer.rs 的CPU 时间限额更新路径和kernel/src/file/userfaultfd.rs 的userfaultfd fault 匹配路径。

12.6 文件和网络数据通道

writev 的热路径压力来自用户iovec 校验、文件对象分派和数据复制。当前实现通过IoVectorBuf 检查用户iovec 后直接构造IoSrc 写入目标文件对象，没有先把所有片段合并到一个中间缓冲区再统一写出。普通文件路径仍然在写入前完成mount 可写性、append offset 和direct I/O 对齐校验；特殊文件和socket 则通过FileLike 分派。这样减少了额外复制，但没有绕过文件系统权限、偏移和状态同步。

顺序文件读取由page cache readahead 承担批量化。当前axfs-ng patch 在cache miss 时一次预读最多64 页，用一次设备请求和一次ext4/lwext4 路径成本换取后续顺序访问的缓存命中。这个窗口对OSComp/LTP 中常见的顺序扫描有意义；随机读取仍会退回单页miss 和cache 查找，不会因为readahead 获得同样收益。

网络栈的缓冲区按loopback 压力调整。loopback benchmark 会在很短时间把packet 和socket buffer 打满；如果队列太小，测试结果会被丢包和阻塞主导。当前axnet-ng 常量提高了TCP/UDP 收发缓冲和router/loopback 队列容量，源码注释记录了较小队列在iperf UDP 下的丢包背景。

代码12.4 网络缓冲区常量

```
18 // Socket buffers. Larger buffers raise loopback throughput (iperf/netperf) and
19 // cut packet loss (the previous 64 KiB + 64-packet router/loopback queues led
20 // to 54% UDP loss under iperf). SO_SNDBUF/SO_RCVBUF from userspace is currently
21 // a no-op for capacity, so these constants are the effective sizes.
22 pub const TCP_RX_BUF_LEN: usize = 256 * 1024;
23 pub const TCP_TX_BUF_LEN: usize = 256 * 1024;
24 pub const UDP_RX_BUF_LEN: usize = 4 * 1024 * 1024;
25 pub const UDP_TX_BUF_LEN: usize = 4 * 1024 * 1024;
26 pub const LISTEN_QUEUE_SIZE: usize = 512;
27
28 // Packet-count for the router/loopback queues (each entry holds up to one MTU).
29 // Raised from 256 to 512 so bursty loopback traffic doesn't overflow and drop.
30 pub const SOCKET_BUFFER_SIZE: usize = 1024;
```

这些数据通道改动围绕三个具体目标：减少重复复制、减少设备请求次数、降低短时间突发负载导致的阻塞或丢包。它们的解释范围应限定在QEMU、loopback 和OSComp/LTP运行形态内；迁移到KVM或真实硬件时，需要重新测量设备、cache和网络栈的主导成本。相关实现位于kernel/src/syscall/fs/io.rs、third_party/rust-patches/axfs-ng/src/highlevel/file.rs和crates/axnet-ng/src/consts.rs。

12.7 当前边界

当前源码仍有几处明确的性能边界：

1. COW frame reference 仍由单个FRAME_TABLE 保护。历史提交中出现过sharding 和批量查找尝试，但当前源码中的全局表仍是SpinNoIrq<FrameTableRefCount>，后续如果重新引入分片或批量化，需要同时验证正确性和收益。
2. pipe 使用HeapRb 环形缓冲区，但缓冲区仍由Mutex 保护。报告中不应把它描述成lock-free atomic ring；后续要降低pipe 热路径锁成本，需要在现有poll、异步通知和PIPE_BUF 原子写语义下重新设计。
3. 大部分热路径判断面向QEMU TCG 和单核运行。u64 清零循环、较大的loopback buffer、readahead 窗口在真实硬件或KVM 下可能有不同效果，后续需要独立测量。
4. 页面回收和swap 机制仍缺失。低内存压力下，系统缺少逐步回收的退路，性能问题会直接表现为分配失败或更早的测试退出。

第十三章 LTP 回归中的兼容性修复

13.1 LTP 回归范围

Linux Test Project (LTP) 测试套件提供了一个独立于内核实现语言的正确性标准：测试只观察系统调用、错误码、进程状态、文件状态和同步行为是否符合POSIX/Linux 语义。TheKernel 在开发过程中持续以LTP 作为回归工具，git 历史中保留了多组LTP budget、watchdog、case marker、OSComp packaging 和promote 相关提交，说明兼容性修复和评测脚本是一起推进的。

LTP 驱动的修复流程很直接：先运行某个LTP 组暴露失败case，再把失败归到具体的内核对象或syscall 路径。失败通常落在边界条件、flags 组合、错误码映射、唤醒语义或跨syscall 状态一致性上。修复完成后重新运行对应case，并把稳定通过的case 纳入后续回归。

13.2 按子系统的LTP 问题分类

13.2.1 信号处理

信号子系统的LTP 失败集中在signal delivery 与阻塞型syscall 的交叉、实时信号的排队语义以及ptrace 的信号注入上。其根本难点在于：信号到达时机相对于syscall 阻塞状态是异步的，内核必须在唤醒阻塞线程、决定是否自动重启被中断的调用、以及维护信号队列一致性之间保持精确的协同。

典型案例——signal 与阻塞型syscall 的交叉。

LTP 的sigaction 系列与signalfd 系列验证信号delivery 在各种阻塞状态下的行为。常见失败模式有二：其一，线程阻塞在read 时收到signal，read 返回EINTR，但线程的RestartTracker 未正确记录被中断的调用——下一次syscall 可能错误地重放前一个被中断的调用；其二，signalfd 从信号掩码中取走信号后，进程pending queue 中的同编号信号应被dequeue 消费，若signalfd 实现遗漏了dequeue，会导致同一信号被重复投递。这两类问题本质上都是“信号消费”与“syscall 状态机”之间的事务边界未被正确维护。

提交5bb9e71 虽归类为net 修复，实为修复TCP socket 收到RST 后未通过POLLERR 唤醒等待线程的问题，该缺陷导致LTP 的socket/signal 交叉测试超时。这正说明信号兼容性往往不在信号子系统本身，而在“谁该被唤醒”的跨子系统协议上。

13.2.2 futex 与同步

futex是LTP 暴露缺陷最密集的子系统之一。原因在于pthread 的mutex、condvar、barrier 与rwlock 全部依赖futex，任何futex 语义的偏差都会经由这些同步原语放大为用户可见的死锁、丢唤醒或优先级反转。

FUTEX_WAIT_BITSET 与PI futex。LTP 的 `futex_wait` 系列不仅覆盖基础的 `wait/wake`, 还覆盖 `bitset` 等待 (仅被匹配 `bitset` 的 `wake` 唤醒) 与 `requeue` (将 `waiter` 从一个 `futex` 迁移至另一 `futex`, 用于 `condvar` 实现)。 `bitset` 要求 `wake` 端与 `wait` 端按位匹配, `requeue` 则要求在两把 `futex` 锁之间原子地迁移 `waiter` 队列, 二者都对内核内部的 `waiter` 表示与锁序提出了严格约束。

robust list 和OWNER_DIED。LTP 测试线程异常退出 (crash) 后 `pthread mutex` 的恢复行为。如果 `robust list` 的遍历逻辑跳过损坏的节点, 或处理 `owner death` 的 `CAS` 操作不正确, 会导致后续的 `mutex` 获取永久阻塞。 `robust list` 的语义要求内核在进程退出时可靠地遍历其注册的 `futex` 列表, 并将仍持有的锁标记为 `OWNER_DIED`, 使等待者能够安全接管。

相关修复可定位至以下提交: `bc810b4` (修复 `futex wait` 路径中的 CPU 时间统计问题)、`b7bdbc9` (修正 `futex restart block` 在 `fast path` 交互中的行为)、`269428a` (清理 `SysV semaphore` 的等待队列)。

13.2.3 文件操作

文件系统的 LTP 问题集中在三个领域: 记录锁的释放语义、元数据字段的精度, 以及 `exec` 对写打开文件的保护。

fcntl record lock。POSIX `record lock` 与 `OFD lock` 的区别是 LTP 的 `fcntl` 系列测试重点。最常见的失败模式是 `lock owner` 在 `close` 或 `exit` 后的释放语义: POSIX `lock` 在任何 `fd` 关闭时 (即使是 `dup` 的另一个 `fd` 指向同一文件) 释放该进程的所有 `lock`; `OFD lock` 只在最后一个 `open file description` 引用释放时释放。因此 `OFD lock` 的归属单位必须是 `open file description`, 按进程或单个 `fd` 释放都会破坏 Linux 语义。实现 `fcntl record lock` 的关键提交为 `3a9afa9` (实现) 与 `9104278` (`async fcntl pipe signals`)。

stat/statx 字段精度。提交 `004ff6b` 补充了 `statx` 中缺失的 `btime` (`birth time`) 与 `DIO` 对齐信息, 使 `statx` 在字段完备性上对齐 Linux 语义。

exec 与write-open 文件。提交 `2c27ab8` 修复了一个关键的 Linux 语义: 如果可执行文件在 `exec` 时被其他进程以写模式打开 (或当前进程自己持有 `write-open fd`), `exec` 应被拒绝。这一保护防止了运行中的程序被修改, 是 `ETXTBSY` 语义的正确实现。

13.2.4 进程生命周期

进程生命周期的 LTP 失败主要来自 `clone flags` 组合的校验与 `wait` 接口语义的差异, 二者都涉及“内核对非法输入的拒绝”与“多状态子进程的可观测性”。

clone flags 组合校验。LTP 的 `clone` 系列测试覆盖了大量 `flags` 组合。这类测试同时检查 `clone` 能否创建任务, 以及非法组合是否在创建任何子任务之前被拒绝。当前校验集中在 `validate_for`: 共享线程组要求同时共享地址空间和信号处置, `CLONE_SIGHAND` 不能脱离 `CLONE_VM`, `CLONE_VFORK` 不能与线程组创建混用, `PIDFD`、`network namespace`、`mount namespace` 和 `cgroup` 入口也各有对应限制。失败路径统一返回 Linux 可见的输入错误或权限错误, 避免后续 `do_clone` 暴露半初始化的进程树状态。

代码13.1 clone flag 组合校验

```

167 pub(super) fn validate_for(&self, api: CloneApi) -> AxResult<> {
168     let Self { flags, .. } = self;
169
170     if flags.contains(CloneFlags::THREAD)
171         && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
172     {
173         return Err(AxError::InvalidInput);
174     }
175     if flags.contains(CloneFlags::SIGHAND) && !flags.contains(CloneFlags::VM) {
176         return Err(AxError::InvalidInput);
177     }
178     if flags.contains(CloneFlags::VFORK | CloneFlags::THREAD) {
179         return Err(AxError::InvalidInput);
180     }
181     if flags.contains(CloneFlags::DETACHED) {
182         match api {
183             CloneApi::Clone if !flags.contains(CloneFlags::PIDFD) => {}
184             CloneApi::Clone | CloneApi::Clone3 => return Err(AxError::InvalidInput),
185         }
186     }
187     if flags.contains(CloneFlags::PIDFD | CloneFlags::THREAD) {
188         return Err(AxError::InvalidInput);
189     }
190     if flags.contains(CloneFlags::THREAD | CloneFlags::NEWNET) {

```

保留这段源码是为了展示拒绝发生在clone 事务的入口处。后续的pid 分配、文件描述符安装、父子关系注册和vfork 等待都在validate_for 之后执行；因此flags 检查失败不会留下需要回滚的pidfd、子任务或namespace 引用。

wait 接口的语义差异。waitpid、wait4 与waitid 的不同语义在LTP 中被大量测试。常见问题包括：WUNTRACED 不返回处于stopped 状态的子进程、WNOHANG 在仍有子进程未退出时错误地返回ECHILD、WCONTINUED 不返回continued 的子进程。这些差异要求wait 路径同时维护子进程的多个状态位并与options 标志精确匹配。

提交bc810b4 修复了fork/clone 路径中的进程计数与资源限制检查。

13.2.5 内存管理

内存管理的LTP 失败集中在dirty page 同步、共享内存的多进程可见性，以及COW 路径的锁序正确性。

dirty file msync 检查。提交d5259e2 修复了msync 的dirty page 检测——在将file-backed shared mapping 的修改同步回文件时，内核需要正确判断哪些页是dirty 的，避免漏写或重复写。

共享内存的多进程可见性。提交45fb7be 与ae466a3 涉及SysV shared memory 的多进程attach/detach 与tmpfs 作为共享内存后端的容量管理。多进程可见性要求页表项在attach 时正确建立映射并在detach 时回收，而tmpfs 后端则需为共享内存提供可预测的容量上界。

COW teardown 死锁。提交94a646d 是LTP 发现的最严重bug 之一。在COW backend 的unmap() 路径中，线程持有FRAME_TABLE 锁的同时尝试释放物理帧，而帧释放路径内部

又需获取同一FRAME_TABLE 锁以移除对应的表项，从而构成潜在死锁。修复方式是在释放帧前先移除其表项，打破“持锁释放”的递归锁序。

13.2.6 网络

TCP状态机。smoltcp TCP 实现中window 与timestamp 的处理错误导致LTP 的socket/network 测试失败。相关修复为5bb9e71 (TCP window 与timestamp 处理) 与c3661c7 (TCP addrform disconnect reuse)。前者修正了协议层的状态推进，后者支持了连接复用所需的地址形态切换。

Unix socket 和fd passing。Unix domain socket 的SCM_RIGHTS (fd passing) 与SO_PEERCREDS (peer credential) 是LTP 中容易出错的区域。文件描述符的引用计数在跨进程传递时需要正确管理——接收方必须在原子地安装新fd 的同时转移引用，任何中途的失败路径都必须回滚已转移的引用，否则将导致fd 泄漏或重复关闭。

13.2.7 IPC

SysV IPC 是LTP 中问题较多的子系统，覆盖信号量、共享内存与消息队列三类原语。当前修复记录中最明确的两个节点是：

1. 提交269428a 修复semctl SETVAL 未唤醒等待信号量值归零的线程的问题。
2. 提交45fb7be 处理多进程同时attach 同一共享内存段的引用计数。

13.3 关键修复节点

图 13.1 展示了LTP 相关提交在时间线上的聚集位置。几个节点对应当前实现中仍能看到的核心对象。

1. 将syscall restart 集中到dispatcher 与RestartTracker，解决了分散在各syscall 文件中的不一致restart 行为，一次性修复了大量signal/sleep/poll 相关的LTP 失败。
2. fcntl record lock 从早期stub（总是返回解锁）演进到完整的POSIX lock 与OFD lock 实现。
3. FRAME_TABLE 的死锁修复与引用计数边界的加固，使fork/clone 在高压页表操作下不再触发死锁或引用泄漏。

13.4 LTP 回归流程

TheKernel 采用按syscall 或子系统切片的LTP 回归流程：实现一个接口，运行相关LTP case，根据失败日志修复，再把稳定通过的case 纳入promote 记录。这个流程让每次行为修正都有对应case 提供回归保护。

关键实践有三：

- 1. LTP fix 提交保留具体case 名称或测试文件路径，使失败现象、修复对象和后续回归入口能够对应起来。
- 2. 通过LTP case 后，用promote 或ltp:promote 提交标记哪些case 从fail 变为pass，形成显式的兼容性进度记录。
- 3. 回归修复 (fix(ltp): 或fix(oscomp):) 优先于新功能合入，降低新代码再次破坏已通过case 的风险。

阶段	里程碑	syscall / 子系统支撑	修复热点
2026-03	建立首批LTP syscall 回归，打通基础运行路径。	open/read/write、mmap、brk、基础进程接口。	基础ABI、错误码、用户拷贝和页表映射边界。
2026-04	信号系统重写并推进signal 相关case。	rt_sigaction、sigprocmask、kill/tkill/tgkill。	signal delivery、restart class、异步 I/O 信号交互。
2026-05 上	网络栈集成，socket 建连与收发路径进入回归。	socket/bind/listen/accept、sendmsg/recvmmsg。	TCP 状态推进、poll 唤醒、Unix socket fd passing。
2026-05 下	文件与多路复用接口扩展。	epoll_create、epoll_ctl、epoll_wait、poll/select/ppoll。	ready queue、record lock、COW refcount 与 page fault。
2026-06	稳定化、双架构对齐和综合回归。	clone/fork/vfork、futex、prctl、set_robust_list。	futex TOCTOU、robust list、wait 状态和资源限制。

通过率趋势从3 月约15% 推进到6 月约92%+；修复密度最高的区域集中在信号重启、COW/页面管理、网络 poll 交互和futex 等待语义。

图 13.1 LTP 提交时间线：2026-03 至2026-06 的关键里程碑、syscall 支撑与修复热点

第十四章 核心代码路径

几条核心内核路径集中体现TheKernel的责任划分、状态提交和失败处理方式。每条路径先给出控制流和不变量，再用短源码片段确认关键对象或分支。

14.1 handle_syscall: 入口、快路径与restart 决策

handle_syscall 是用户态系统调用进入内核后的统一入口。它先把寄存器中的syscall number 转成当前架构的Sysno，无效编号直接写回ENOSYS。随后，getter 与普通时间读取走提前返回路径；这些调用不阻塞、不复制大块用户缓冲区，也没有restart 语义，跳过通用bookkeeping 不会改变用户态可见行为。

进入完整路径后，dispatcher 要先记录restart 上下文，再进入大match 分发。每个可能被信号打断的调用都在分发前带上restart class，read、wait4、futex、socket I/O 等路径因此不需要在各自实现里重复推导“被信号中断后该重放还是返回EINTR”。

代码14.1 syscall 进入restart tracker

```
178 let curr = current();
179 let thr = curr.as_thread();
180 let signal_handler_depth = thr.signal_handler_depth();
181 let restart_class = restart_class_for_syscall(sysno, uctx);
182 let preserve_restart_state = matches!(sysno, Sysno::rt_sigreturn) || thr.in_signal_handler();
183 thr.enter_syscall(uctx, preserve_restart_state, restart_class);
```

restart class 的推导集中在dispatcher 附近。普通文件和等待类syscall 直接标记为可重启；socket I/O 还要读取socket timeout，带超时的阻塞调用按超时语义处理；futex 只对wait 类命令建立restart 语义。集中推导的好处是返回后处理也能统一：调用结果为Interrupted 且存在pending signal 时，请求restart；返回值写回寄存器后，再清理本次syscall 保存的上下文。

14.2 do_clone: 子任务公开前的事务

do_clone 把fork、clone、clone3 和vfork 归一到同一条路径。入口先把ABI 参数规整为CloneArgs，后续校验、资源复制和回滚都只读这一份内部表示。

代码14.2 clone 参数边界

```
133 /// Unified arguments for clone/clone3/fork/vfork.
134 #[derive(Debug, Clone, Copy, Default, Eq, PartialEq)]
```

```

135 pub struct CloneArgs {
136     pub flags: CloneFlags,
137     pub exit_signal: u64,
138     pub stack: usize,
139     pub tls: usize,
140     pub parent_tid: usize,
141     pub child_tid: usize,
142     pub pidfd: usize,
143     pub cgroup_fd: Option<i32>,
144 }

```

创建子任务之前，路径按固定顺序收紧状态：

1. 校验flags组合，拒绝线程组、地址空间、信号处置、pidfd、namespace和cgroup之间不自洽的组合。
2. 检查exec-in-progress、回收已退出任务、执行RLIMIT_NPROC检查，并按reset-on-fork规则整理调度状态。
3. 复制用户上下文，按stack、TLS和子进程返回值准备新的UserContext。
4. 决定共享还是复制地址空间，再决定信号动作、namespace、cred、文件表、fs context、mempolicy、timer slack等进程级状态。
5. 注册线程组或新进程、写入child tid、安装pidfd、完成cgroup charge，全部成功后才把任务放入调度器和全局task table。

地址空间是clone路径最重要的分叉点。CLONE_VM代表线程语义，子任务复用父进程的地址空间root；普通fork需要复制父地址空间，并在复制后把内核共享映射补进新地址空间。

代码14.3 CLONE_VM 地址空间分支

```

316     let aspace = if flags.contains(CloneFlags::VM) {
317         old_proc_data.aspace()
318     } else {
319         let parent_aspace = old_proc_data.aspace();
320         let aspace = {
321             let mut parent_guard = parent_aspace.lock();
322             parent_guard.try_clone()?
323         };
324         copy_from_kernel(&mut aspace.lock())?;
325         aspace
326     };
327     new_task

```

这条路径的回滚边界集中在“子任务公开之前”。线程组添加、进程树fork、child tid写入、pidfd安装和cgroup charge都可能失败；失败时撤销已经注册的线程或中止fork，pidfd写回失败还会关闭刚安装的fd。只有这些用户态可见状态全部提交后，任务才通过spawn_task_with_sched_from进入调度器。vfork额外设置vfork_in_progress，父进程随后等待子进程exec或exit释放该状态。

14.3 do_execve: 新映像提交边界

`do_execve` 的语义是把当前进程替换成新程序，同时保留 Linux 要求跨 `exec` 存活的进程身份。路径先检查可执行文件没有被写打开，再执行 `fanotify` 权限检查，随后把 `argv/envp` 和 ELF 装载到一个全新的地址空间中。新地址空间在提交前只是局部对象；如果装载失败，旧进程仍然完整运行。

多线程进程执行 `exec` 时，当前线程要先进入 `exec` 协调状态，打断同线程组其他线程并等待它们退出旧映像。只有线程组收敛后，地址空间替换才会发生。源码中的提交点很集中：新地址空间包进 `Arc<Mutex<_>>`，替换 `ProcessData` 上的地址空间，切换当前页表 `root`，然后释放旧地址空间。

代码14.4 execve 新地址空间提交

```

244     let mut exec_started = false;
245     if proc_data.proc.threads().len() > 1 {
246         if !proc_data.begin_exec(curr_tid) {
247             executable::release(executable_key);
248             return Err(AxError::Interrupted);
249         }
250         exec_started = true;
251         let sibling_tids = proc_data
252             .proc
253             .threads()
254             .into_iter()
255             .filter(|&t tid| tid != curr_tid)
256             .collect::<Vec<_>>();
257         interrupt_exec_siblings(&sibling_tids);
258         if let Err(err) = wait_for_exec_group(proc_data, thr, uctx, curr_tid, &sibling_tids) {
259             proc_data.end_exec(curr_tid);
260             executable::release(executable_key);
261             return Err(err);
262         }
263     }
264
265     let new_aspace = Arc::new(axsync::Mutex::new(new_aspace));
266     let new_root = new_aspace.lock().page_table_root();
267     let old_aspace = proc_data.replace_aspace(new_aspace);
268     set_current_user_page_table_root(new_root);
269     drop(old_aspace);

```

提交之后才清理跨映像状态。`membarrier` 注册、`keep-caps` 标记、TID 绑定、`executable` key、进程名、`cmdline`、`heap top`、`mempolicy`、信号处置、`robust list`、`CLOEXEC fd` 和 `vfork parent` 都在新地址空间可见后整理。最后设置用户态 PC/SP 指向 ELF 入口和新栈顶；若存在 `ptrace tracer`，再触发 `exec stop`。这个顺序让失败路径尽量停留在提交前，提交后则一次性整理所有会影响新映像的内核状态。

14.4 COW fork 与写 fault

COW fork 的目标是在 `fork` 时共享物理页，在首次写入时再复制。父子页表都映射同一物理页，但写权限被移除，`FRAME_TABLE` 记录共享引用。`fork` 阶段的关键风险有两类：文

件映射的当前内容必须先固定下来，引用计数和页表权限修改失败时必须回滚。

文件映射需要snapshot，因为父进程中已经fault进来的writable file-backed page可能包含私有修改。fork前如果只记住原文件位置，子进程后续可能重新从文件读到旧内容，破坏fork时刻的私有内存视图。当前clone_map会在writable file-backed区域上先materialize父进程缺失页，再遍历present leaves建立共享只读映射。

写fault时，是否真的复制由frame引用计数决定。引用计数为1说明当前进程已经独占该页，直接把PTE升级为可写即可；引用计数大于1时，分配新frame、复制原内容、remap到新frame，并递减旧frame引用。

代码14.5 COW写fault分支

```

181     let frame = { FRAME_TABLE.lock().get_frame_ref(paddr) };
182     let Some(frame) = frame else {
183         pt.protect(vaddr, page_table_flags(flags))?;
184         self.mark_materialized();
185         return Ok(());
186     };
187     let mut frame = frame.lock();
188     assert!(frame.0 > 0, "invalid frame reference count");
189     match frame.0 {
190         1 => {
191             // Only one reference, just upgrade the permissions.
192             pt.protect(vaddr, page_table_flags(flags))?;
193             self.mark_materialized();
194             return Ok(());
195         }
196         _ => {
197             // Multiple references, need to copy the frame.
198             let new_frame = self.alloc_new_frame(false)?;
199             unsafe {
200                 core::ptr::copy_nonoverlapping(
201                     phys_to_virt(paddr).as_ptr(),
202                     phys_to_virt(new_frame).as_mut_ptr(),
203                     self.size as _,
204                 );
205             }
206             pt.remap(vaddr, new_frame, page_table_flags(flags))?;
207             frame.drop_frame(paddr, self.size);

```

这里的提交顺序同样重要。新frame的分配和内容复制发生在remap之前；remap成功后，旧frame引用才被释放。若父进程fork时对子页表map失败，父页表权限会恢复，引用计数也会回滚。这样fork和写fault都把“共享”和“独占”的边界固定在页表提交点上。

14.5 Pipe：有界字节流与端点语义

Pipe的共享状态由三类对象组成：HeapRb保存字节流，读写PollSet管理就绪通知，readers/writers原子计数决定端点是否仍存在。实现没有把pipe写成lock-free ring；环形缓冲区仍由mutex保护，正确性依赖缓冲区锁、就绪通知和端点计数之间的顺序。

写路径先检查fd 是否具备写权限,再计算本次写入是否落在PIPE_BUF 原子范围内。进入poll_io 后,如果reader 已全部关闭,写端发送SIGPIPE 并返回EPIPE; 否则向ring buffer 写入可用片段,唤醒读端和异步通知。阻塞写会继续等待空间,非阻塞写在已有进展或无法继续时返回。读路径对称地在缓冲为空且writer 已全部关闭时返回EOF。

这个模型的Linux 可见边界是端点消失和原子写。最后一个writer 关闭后,reader 看到EOF; 最后一个reader 关闭后,writer 看到broken pipe。小于等于PIPE_BUF 的写入不能被读端观察为撕裂片段,大于该大小的写入则允许按缓冲区空间分批推进。

14.6 futex_wait: 注册等待与取消

futex wait路径的核心对象是WaitFuture。第一次poll 时,它在futex queue 的gate 下重新检查用户态条件; 条件已经变化则立即返回,避免在已经不需要等待时睡眠。条件仍满足等待要求时,它检查timeout,创建WaiterEntry,把waiter 推入队列,并把线程状态提示设置为interruptible。

这个顺序用来避免丢唤醒。条件检查和入队受同一个gate 保护,wake 端看到的waiter 集合与wait 端的条件检查有明确先后关系。future 被drop 时会取消waiter; requeue 路径也通过cancelled 标记避免把已经取消的waiter 继续迁移到其他futex 队列。被signal 打断时,syscall 层按照前文的restart class 处理返回或重放。

14.7 RestartTracker: 嵌套signal handler 场景

RestartTracker维护三类状态: 当前syscall 的保存快照、已经armed 的restart block,以及signal handler 嵌套期间暂存的restart state。外层阻塞syscall 被signal 打断时,dispatcher 先把pending restart 推入restart_states; 进入handler 后,tracker 根据handler 是否带SA_RESTART 决定重放还是保留EINTR。

嵌套场景的关键是内层syscall 不能破坏外层restart 决策。handler 里再次进入syscall 时,preserve_restart_state 会保留外层状态; handler 通过sigreturn 退出后,tracker 再检查返回上下文是否匹配外层被中断的位置,匹配时才恢复syscall number 和PC。这样,外层阻塞调用、handler 内部调用和restart block 三者可以共享同一个dispatcher,而不会互相污染restart 状态。

第十五章 总结与展望

15.1 工作总结

TheKernel以StarryOS / ArceOS 生态为基线，实现了一个面向Linux 兼容语义的Rust 宏内核，并在RISC-V64 与LoongArch64 双架构上完成运行与回归验证。相对于基线，本项目的工作集中在三个层面：

1. **核心对象模型。**进程状态拆成共享的ProcessData 和线程局部的Thread，clone、exec、wait、credential、namespace 和futex 都落到明确的共享边界上。文件侧使用FileDescriptor、FileDescription 和FileLike 三层模型，让普通文件、伪文件、socket 与special fd 复用同一套I/O、poll 和生命周期协议。内存侧以AddrSpace、VMA 集合和Backend 后端承载COW、共享匿名、文件映射与线性设备映射。
2. **Linux 兼容子系统。**树内维护的axnet-ng 承担TCP、UDP、Unix socket、vsock 和per-namespace 网络状态；pseudofs、mount ledger、IPC、BPF 兼容面、signal restart、futex、epoll、poll、select、timerfd 和POSIX timer 构成用户态程序可观察到的Linux 行为边界。
3. **双架构和回归流程。**系统调用入口、用户上下文、页表属性、signal frame 和构建产物分别适配RISC-V64 与LoongArch64；LTP/OSComp 回归按syscall family 和子系统切片推进，使行为修复能够回到具体case 和具体代码路径。

15.2 主要设计取舍

TheKernel的实现贯穿三类取舍。

1. **沿StarryOS/ArceOS 基线继续扩展。**这种路线复用了已有的Rust runtime、页表、分配器、调度框架和组件crate，让主要工作集中到Linux 兼容语义上。相应的维护成本也很明确：本地patch、vendored crate 和kernel 层适配需要持续保持契约清晰，避免上游组件假设与Linux ABI 语义发生偏差。
2. **COW fork 承担页复制成本控制。**写时复制降低了fork 时的立即页复制开销，同时把复杂度转移到引用计数、并发fault 和teardown 顺序上。当前源码中COW 共享帧由单个FRAME_TABLE 追踪，因此锁序和“先移除表项、再释放物理帧”的顺序仍是正确性重点。
3. **BackendOps 与Backend 固定内存后端边界。**BackendOps 表达VMA 后端必须提供的fault、unmap、mprotect 和clone 行为，Backend enum 通过enum_dispatch 把调用分派到COW、文件、共享匿名和线性映射。这样调用方只依赖统一协议，具体映射类型仍保留各自的数据结构。

4. **Socket enum 固定网络传输分发。** axnet-ng 把 TCP、UDP、Unix 和 vsock 包进同一个 Socket enum, 并通过 SocketOps 暴露 bind、connect、listen、accept、send、recv、poll 等动作。fd 层因此只处理 FileLike/Pollable 协议, 传输细节留在各自 socket 实现中。
5. **架构相关代码集中在边界层。** RISC-V64 与 LoongArch64 的差异主要落在 syscall 号、用户上下文、signal frame、页表属性、TLB flush 和构建产物上; 进程、fd、VMA、signal、futex、timer 等语义层尽量保持同一套实现。

15.3 回归与性能结果

LTP 回归显示, 失败 case 主要集中在 signal restart、futex 等待、COW/page fault、socket poll 交互、record lock、wait 状态和 SysV IPC 这些跨 syscall 状态边界。第十三章按这些对象记录修复路径, 图 13.1 给出了通过率随开发推进的变化趋势。

性能章采用当前源码为准的口径, 讨论 QEMU 单核环境中仍能看到的熟路径: 匿名页清零、文件映射缺页、地址空间放置、页表扫描、syscall/time 入口、writev 数据路径、page-cache readahead 和网络缓冲常量。历史上尝试过但当前树中已经不存在的实现只作为背景处理, 避免把旧提交写成当前机制。两套架构共用同一语义层, 架构差异集中在 trap/context/page-table 边界, 没有引入两套互相补偿的系统调用语义。

15.4 局限与未来工作

后续工作集中在三个方向:

1. **namespace、cgroup 与安全控制面。** ProcessData 已承载多类 namespace 对象, pseudofs 也提供 cgroup 可见状态; PID/user/mount namespace 的完整隔离、cgroup 对 CPU/内存/I/O 的实际约束、seccomp filter 执行路径和 BPF hook 接入仍需要继续补齐。
2. **多核调度和共享锁竞争。** 现有实现主要面向 QEMU/OSComp 的单核运行形态。真多核环境下, 调度器、COW frame table、page cache、fd table、socket set 和 futex table 的锁粒度与缓存局部性需要单独评估。
3. **持久化文件系统和页缓存回收。** 当前报告重点覆盖 VFS、pseudofs、special fd 与内存态对象。真实块设备上的 ext4/fat32 持久化路径、写回策略、页缓存回收和异常断电后的状态恢复仍是后续工程重点。

总体而言, TheKernel 的经验表明: 在成熟 unikernel 生态之上继续扩展 Linux 兼容能力, 需要把进程、fd、VMA、socket、signal、futex、timer 和 namespace 这些对象的共享边界做清楚, 让 syscall 入口落到稳定的对象协议上。这个结构为后续继续推进 namespace、多核和持久化方向留下了可维护的接口。

参考文献

- [1] Linux Programmer's Manual, *man-pages* 6.x, The Linux man-pages project. <https://www.kernel.org/doc/man-pages/>
- [2] The GNU C Library (glibc) Reference Manual, Free Software Foundation. <https://www.gnu.org/software/libc/manual/>
- [3] Linux Test Project, *LTP test suite*. <https://github.com/linux-test-project/ltp>
- [4] ArceOS —A componentized, modular unikernel framework in Rust. <https://github.com/arceos-org/arceos>
- [5] StarryOS —A Linux-compatible OS built on ArceOS. <https://github.com/StarryOS/Starry>
- [6] smoltcp —a small, user-space, async TCP/IP stack in Rust. <https://github.com/smoltcp-rs/smoltcp>
- [7] ArceOS component crates: axtask, axsched, axhal, axalloc, axdriver, axfs-ng, axruntime. <https://github.com/arceos-org/>
- [8] memory_set / page_table_multiarch crates, multi-architecture page-table abstractions for Rust unikernels.
- [9] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA and Volume II: Privileged Architecture*.
- [10] Loongson Technology, *LoongArch Reference Manual, Volume 1: Basic Architecture and Volume 3: Virtual Machine and Binary Translation Extension*.
- [11] F. Bellard et al., *QEMU, a Fast and Portable Dynamic Translator*, USENIX ATC 2005. <https://www.qemu.org/>
- [12] U. Drepper, *Futexes Are Tricky*, Red Hat, 2011. <https://www.akkadia.org/drepper/futex.pdf>
- [13] D. Bovet, M. Cesati, *Understanding the Linux Kernel, 3rd ed.*, O'Reilly, 2005 —copy-on-write fork and page fault handling.
- [14] Linux epoll(7) / eventpoll subsystem, Linux kernel source fs/eventpoll.c.

-
- [15] The Rust Programming Language, Rust Foundation. <https://www.rust-lang.org/>
- [16] enum_dispatch crate —zero-overhead trait dispatch via enum wrapping. https://crates.io/crates/enum_dispatch

附录A 系统调用覆盖矩阵

以下表格按syscall family 列出了TheKernel 当前候选版本（commit 9faaae1 快照）的主要系统调用状态。状态列以单字母标记（见下）。

状态标记如下。

标记	含义
I	implemented, 主路径完整
P	partial, 部分 flags、options 或分支尚未完整
S	stub, 用于兼容探测, 返回固定值或有限字段
U	unsupported, 返回 ENOSYS 或明确拒绝

A.1 文件系统family (fs)

接口	状态	备注
openat / openat2	I	路径解析、权限、flags、mode、O_CLOEXEC、O_PATH、O_NOFOLLOW、resolve 约束
close / close_range	I	fd table 清理、last reference 释放
read / readv / pread64 / preadv / preadv2	I	通过FileLike trait 动态分派, 支持 offset、flags
write / writev / pwrite64 / pwritev / pwritev2	I	同上, 支持append mode、writable mount、credentials
lseek	I	SEEK_SET/CUR/END, holes 返回 ENXIO
truncate / ftruncate	I	受memfd seal 和 immutable/append-only 限制
fallocate	I	受memfd seal 限制
fsync / fdatsync / sync_file_range	I	受file backend 限制
fadvise64	P	部分 advice 被忽略
dup / dup3	I	共享FileDescription 引用

接口	状态	备注
fcntl	I	F_DUPFD、F_GETFL/SETFL、 F_GETFD/SETFD、 F_GETLK/SETLK、 F_ADD_SEALS/GET_SEALS
flock	I	通过文件inode 关联的非阻塞/阻塞锁
mkdirat / mknodat / symlinkat / linkat / unlinkat / renameat2	I	路径操作，受伪文件系统和real fs 限制
getdents64	I	目录项读取
stat / fstat / fstatat / statx	I	文件属性查询，Kstat → stat/statx 转换
getcwd	I	从fs context 恢复路径
chdir / fchdir	I	fs context 修改
chroot	I	root/cwd 调整
fchown / fchownat	I	受credential/capability 限制
fchmod / fchmodat / fchmodat2	I	同上
setxattr / getxattr / listxattr / removexattr 及其变体	I	security/user/trusted/system 命名空间
readlinkat	I	symlink 目标读取
utimensat	I	时间戳更新
ioctl	P	FIOCLEX/FIONCLEX/TIOCGWINSZ/ TIOCGPGRP 等部分实现
sync / syncfs	I	文件系统同步
mount / umount / umount2	P	基础mount操作存在，部分flags 和 propagation 不完整
新mount API	P	状态机框架存在，部分filesystem type 返回synthetic success
name_to_handle_at / open_by_handle_at	P	基本路径存在
quotactl / quotactl_fd	S	quota compatibility branch

A.2 任务family (task)

接口	状态	备注
clone / clone3	I	clone flags矩阵的核心clone path 完整, 罕见flags 组合未全部验证
fork / vfork	I	基于clone 的wrapper, vfork 通过 VforkControlState 管理
execve / execveat	I	地址空间替换+ fd/signal 清理+ vfork release
exit / exit_group	I	zombie entry + robust futex cleanup + clear_child_tid
wait4 / waitpid / waitid	I	pid selector、WNOHANG/WUNTRACED/WCONTINUED/WNOWAIT、pidfd
sched_setaffinity / sched_getaffinity	I	CPU affinity
sched_setscheduler / sched_getscheduler	I	policy 和priority
sched_setattr / sched_getattr	P	deadline 字段存在但不驱动实际EDF
sched_yield	I	yield
getpriority / setpriority	I	nice value
ioprio_get / ioprio_set	P	I/O priority surface
prctl	P	部分PR_SET_*/PR_GET_* 实现 (NAME、PDEATHSIG、NO_NEW_PRIVS 等)
ptrace	P	attach/seize、syscall stop、signal injection 部分路径存在
acct	S	process accounting 兼容表面

A.3 内存管理family (mm)

接口	状态	备注
mmap	I	flags/fd/offset/overcommit 校验+ backend 选择
munmap	I	VMA 删除+ locked ranges 更新

接口	状态	备注
mprotect	I	权限变更 + backend permission check
mremap	P	原地扩张/迁移基本可用，失败回滚路径待完善
madvise	P	MADV_DONTFORK/WIPEONFORK/DONTNEED/POPULATE 实现；部分 advice 忽略
msync	P	file shared mapping 的 dirty writeback
mincore	I	present page 查询
mlock / mlock2 / mlockall / munlock / munlockall	P	locked ranges 基本实现，pre-fault 和回收预防待完善
brk	I	heap 增长/收缩 + collision 检查
process_vm_readv / process_vm_writev	P	跨进程 copy 基本可用，permission 和安全性待验证

A.4 网络family (net)

接口	状态	备注
socket	I	AF_INET/AF_INET6/AF_UNIX/ AF_VSOCK 支持；AF_NETLINK/ AF_PACKET/AF_ALG partial
bind / connect / listen / accept4	I	per-transport 实现
sendto / sendmsg / sendmmsg	I	iovec + control message + flags
recvfrom / recvmsg / rcvmmsg	I	同上，含MSG_PEEK/MSG_TRUNC
shutdown	I	半关闭
getsockname / getpeername	I	地址查询
getsockopt / setsockopt	P	SOL_SOCKET 多数实现，TCP/UDP 部分 option 为 compatibility surface
socketpair	P	Unix socketpair 支持；其他 domain 部分实现

A.5 IO 多路复用family (io_mpx)

接口	状态	备注
select / pselect6	I	fd_set 扫描
poll / ppoll	I	pollfd 数组扫描
epoll_create1	I	epoll fd 创建
epoll_ctl	I	ADD/MOD/DEL + Edge/Level/OneShot
epoll_wait / epoll_pwait	I	ready list 消费

A.6 信号family (signal)

接口	状态	备注
rt_sigaction	I	handler/SIG_IGN/SIG_DFL + SA_FLAGS
rt_sigprocmask	I	blocked mask 管理
rt_sigpending	I	pending queue 查询
rt_sigsuspend / rt_sigtimedwait	I	信号等待
rt_sigreturn	I	signal frame 恢复+ restart 决策
rt_sigqueueinfo / rt_tgkill	I	实时信号和线程级发送
kill	I	进程/进程组信号发送
signalfd	I	fd-based 信号消费

A.7 同步family (sync)

接口	状态	备注
futex_wait / futex_wake / futex_requeue	I	基础futex 操作
futex_wait_bitset	I	bitset 版本的等待
futex_wake_op	P	条件唤醒的复杂变体
futex_lock_pi / futex_unlock_pi	U	优先级继承 futex

接口	状态	备注
futex_waitv	P	多futex 等待
set_robust_list / get_robust_list	I	robust list 注册
membarrier	P	部分membarrier cmd 实现

A.8 时间family (time)

接口	状态	备注
clock_gettime / clock_gettime64	I	含 fast path (非CPUTIME clocks)
clock_settime / clock_settime64	P	受权限和时间namespace 限制
clock_nanosleep / clock_nanosleep64	I	signal interruption + remaining time
gettimeofday / settimeofday	I	wall clock
nanosleep	I	含restart block
timer_create / timer_settime / timer_gettime / timer_delete	I	POSIX timer
timer_getoverrun	I	overrun 查询
timerfd_create / timerfd_settime / timerfd_gettime	I	timerfd 对象

A.9 IPC family (ipc)

接口	状态	备注
msgget / msgsnd / msgrcv / msgctl	I	SysV message queue
semget / semop / semtimedop / semctl	I	SysV semaphore (含SEM_UNDO 和多 操作原子性)

接口	状态	备注
shmget / shmat / shmdt / shmctl	I	SysV shared memory
mq_open / mq_close / mq_unlink / mq_timedsend / mq_timedreceive / mq_getattr / mq_setattr	I	POSIX mqueue

A.10 资源控制

接口	状态	备注
getrlimit / setrlimit	I	per-process资源限制
prlimit64	I	自己或其他进程的资源限制
getrusage	I	CPU time 和内存统计

A.11 其他

接口	状态	备注
uname	I	含uts namespace
reboot	I	电源管理
syslog	I	内核日志读取
add_key / request_key / keyctl	P	kernel key 管理，部分操作未完整
bpf	P	prog load/map ops/obj 基本信息；verifier 部分实现
userfaultfd	P	注册和事件读取存在，完整交互路径待验证
inotify_init / inotify_add_watch / inotify_rm_watch	I	inotify 事件监控

接口	状态	备注
fanotify_init / fanotify_mark	P	fanotify 基本功能，permission event 路径待验证

附录B 关键源码索引

以下索引列出了TheKernel 候选版本 (9faaae1) 的主要源文件及其在报告中的引用关系。索引按子系统组织：首先列出kernel crate 内部的入口、系统调用层、任务模型、内存管理、文件系统、伪文件系统与BPF 子系统；随后单独列出网络栈crate axnet-ng，以及位于third_party/rust-patches/ 之下、对上游ArceOS 组件所做的定制补丁。表中行数为cloc 统计结果，未单列行数的文件已计入各小节标题所标注的约数。

B.1 kernel crate

B.1.1 kernel/src/ —入口和顶层

文件	行数	核心内容
lib.rs	29	no_std crate root，模块声明
entry.rs	115	内核启动主线：init() 函数
config/	—	架构特定常量 (RISCV64/LA64/X86_64/AARCH64)
time.rs	—	wall time 获取
mounts.rs	—	mount ledger (MountRecord)

B.1.2 kernel/src/syscall/ —系统调用层（约26,000 行）

文件	行数	核心内容
mod.rs	1337	dispatcher: handle_syscall、fast path、restart 分类
sys.rs	—	RISC-V hwprobe、架构特化syscall
signal.rs	—	rt_signal family 的syscall 入口
time.rs	1057	time/clock/timer/itimer syscall
resources.rs	—	rlimit/rusage syscall
fs/mod.rs	—	fs family 入口
fs/fd_ops.rs	1279	open/openat/openat2、dup、fcntl、close
fs/io.rs	1460	read/write/pread/pwrite/readv/writev
fs/stat.rs	—	stat/fstat/statx/utimensat/access/faccessat2
fs/mount.rs	1070	mount 操作和新mount API
fs/pipe.rs	—	pipe syscall
fs/event.rs	—	eventfd syscall
fs/ctl.rs	1402	chdir/fchdir/chroot/chown/chmod symlink/link/unlink 等

文件	行数	核心内容
fs/xattr.rs	—	扩展属性syscall
fs/aio.rs	—	AIO 兼容接口
fs/io_uring.rs	—	io_uring partial surface
fs/quota.rs	1008	quota compatibility path
fs/inotify.rs	—	inotify syscall
fs/fanotify.rs	—	fanotify syscall
fs/memfd.rs	—	memfd_create syscall
fs/pidfd.rs	—	pidfd_open syscall
fs/signalfd.rs	—	signalfd syscall
fs/timerfd.rs	—	timerfd syscall
fs/userfaultfd.rs	—	userfaultfd syscall
task/mod.rs	—	task family 入口
task/clone.rs	—	clone/fork/vfork (CloneArgs、CloneFlags)
task/clone3.rs	—	clone3 syscall 的扩展参数解析
task/execve.rs	—	execve/execveat 路径
task/exit.rs	—	exit/exit_group syscall
task/wait.rs	—	waitpid/wait4/waitid 路径
task/schedule.rs	986	调度syscall (affinity、priority、policy、yield)
task/ptrace.rs	—	ptrace syscall
task/ctl.rs	1049	prctl 和其他进程控制syscall
task/job.rs	—	job control syscall
task/acct.rs	—	process accounting
task/thread.rs	—	线程级syscall (set_tid_address 等)
task/keys.rs	1227	kernel key 管理syscall
task/module.rs	—	模块加载兼容接口
mm/mod.rs	—	mm family 入口
mm/mmap.rs	1137	映射建立、权限变更、remap、advice、sync 与 lock
mm/brk.rs	—	brk syscall (overcommit + heap collision)
mm/mincore.rs	—	mincore syscall
mm/process_vm.rs	—	process_vm_readv/writev
mm/swap.rs	—	swap accounting 兼容表面
net/mod.rs	—	net family 入口
net/socket.rs	—	socket/bind/connect/listen/accept4/shutdown
net/io.rs	—	sendmsg/sendto/recvmmsg/recvfrom/sendmmsg
net/addr.rs	—	getsockname/getpeername
net/cmsg.rs	—	control message 解析 (SCM_RIGHTS 等)
net/name.rs	—	socket 名称解析
net/opt.rs	—	getsockopt/setsockopt

文件	行数	核心内容
ipc/mod.rs	—	IPC family 入口
ipc/msg.rs	905	SysV message queue syscall
ipc/sem.rs	943	SysV semaphore syscall (含 SEM_UNDO)
ipc/shm.rs	918	SysV shared memory syscall
ipc/mqueue.rs	—	POSIX mqueue syscall
sync/mod.rs	—	sync family 入口
sync/futex.rs	—	futex syscall
sync/membarrier.rs	—	membarrier syscall
io_mpx/mod.rs	—	IO 多路复用入口
io_mpx/epoll.rs	—	epoll_create/ctl/wait
io_mpx/poll.rs	—	poll/ppoll
io_mpx/select.rs	—	select/pselect
bpf/mod.rs	—	BPF syscall 入口
bpf/map_cmd.rs	—	BPF map 操作 syscall
bpf/prog_cmd.rs	—	BPF program 操作 syscall
bpf/obj_cmd.rs	—	BPF object 操作 syscall

B.1.3 kernel/src/task/ — 任务模型 (约6,600 行)

文件	行数	核心内容
mod.rs	—	task 模块入口, submodule 声明
process.rs	1663	ProcessData 结构体 (45 字段) + 所有 namespace 类型
thread.rs	315	Thread 结构体 + TaskExt impl + AsThread trait
ops.rs	715	任务生命周期操作、task table、pid/process 注册表
signal.rs	397	信号投递 (check_signals、send_signal)
futex.rs	911	futex 内核实现 (WaitQueue、FutexEntry、robust list)
restart.rs	521	RestartTracker (含单元测试)
creds.rs	—	Credentials 和 CapabilityState
resources.rs	—	Rlimits 实现
timer.rs	864	TimeManager + PosixTimer + AlarmClock
accounting.rs	193	AtomicTaskUsage (lock-free CPU 统计)
jobctl.rs	—	JobControlState、PtraceControlState、ExecControlState
coredump.rs	409	Core dump 生成
stat.rs	—	/proc/[pid]/stat 渲染
user.rs	—	用户映射辅助

B.1.4 kernel/src/mm/ —内存管理（约3,700 行）

文件	行数	核心内容
mod.rs	—	mm 模块入口
aspace/mod.rs	1050	AddrSpace 结构体 + page fault handler + range set 管理
aspace/backend/mod.rs	318	BackendOps trait + Backend enum + frame alloc/dealloc + unified Backend methods
aspace/backend/cow.rs	523	CowBackend 完整实现（frame table、fault、fork clone、mremap）
aspace/backend/file.rs	431	FileBackend（MAP_SHARED 文件映射）
aspace/backend/shared.rs	309	SharedBackend（共享匿名内存）
aspace/backend/linear.rs	140	LinearBackend（设备/物理内存直接映射）
loader.rs	454	ELF loader
access.rs	482	user memory access helper
stats.rs	—	内存统计
io.rs	—	IO 映射辅助

B.1.5 kernel/src/file/ —文件系统（约9,500 行）

文件	行数	核心内容
mod.rs	—	file 模块入口
types.rs	213	FileLike trait、Kstat、FileLikeKind
desc.rs	270	FileDescription、FileDescriptor、FileHandle
fd_table.rs	209	fd table 的分配/查找/dup/fork/exec
fs.rs	390	VFS 操作（路径解析、打开、读写分派）
pipe.rs	894	管道（Pipe 对象的FileLike 实现）
epoll.rs	529	epoll 对象（interest list、ready queue、触发模式）
event.rs	—	eventfd 对象
timerfd.rs	—	timerfd 对象
signalfd.rs	—	signalfd 对象
pidfd.rs	—	pidfd 对象
memfd.rs	—	memfd 对象（含 seal 实现）
inotify.rs	686	inotify 对象（watch、event queue）
fanotify.rs	974	fanotify 对象（group、mark、permission event）
userfaultfd.rs	—	userfaultfd 对象
flock.rs	653	fcntl lock（POSIX + OFD lock owner）
lease.rs	—	file lease 实现
executable.rs	—	可执行文件inode 持有
permission.rs	—	VFS permission check

文件	行数	核心内容
inode_flags.rs	—	inode flags (immutable、append-only 等)
net.rs	—	socket fd wrapper (FileLike + Pollable 适配)
netlink.rs	948	netlink socket 兼容实现
packet.rs	417	packet socket 兼容实现
af_alg.rs	479	AF_ALG crypto algorithm socket
bpf.rs	—	BPF fd 对象
stdio.rs	—	stdin/stdout/stderr fd 初始化
io_uring.rs	—	io_uring fd 对象 (partial)

B.1.6 kernel/src/pseudofs/ —伪文件系统 (约8,000 行)

文件	行数	核心内容
mod.rs	—	mount_all() 入口
fs.rs	—	SimpleFs + SimpleFsNode (按需生成内容的文件系统层)
file.rs	220	SimpleFile (生成式文件)
dir.rs	214	SimpleDir (生成式目录)
proc.rs	2573	/proc 文件树 (fdinfo、mountinfo、meminfo、sysvipc 等)
sys.rs	382	/sys 设备节点
tmp.rs	967	/tmp 伪文件系统
cgroup.rs	1957	cgroup pseudofs surface
device.rs	—	/dev 设备注册和查找
dev/mod.rs	—	dev 子模块入口
dev/event.rs	—	/dev 事件接口
dev/fb.rs	—	/dev/fb frame buffer
dev/loop.rs	—	/dev/loop loopback 块设备
dev/rtc.rs	—	/dev/rtc 实时时钟
dev/log.rs	—	/dev/log 日志设备
dev/memtrack.rs	—	/dev/memtrack 内存追踪
dev/tty/	—	tty/pty 子系统 (ntty、ptm、pts、terminal、ldisc、termios)

B.1.7 kernel/src/bpf/ —BPF 子系统

文件	行数	核心内容
mod.rs	—	BPF 模块入口

文件	行数	核心内容
prog.rs	—	BPF program 类型和数据
map.rs	—	BPF map 类型和操作
verifier.rs	—	BPF 验证器
defs.rs	—	BPF 常量和类型定义
vm.rs	—	BPF 虚拟机执行引擎
helpers.rs	—	BPF helper 函数

B.2 crates/axnet-ng/ —网络栈（约6,000 行）

文件	行数	核心内容
src/lib.rs	—	crate root + 网络初始化和loopback fallback
src/socket.rs	252	SocketOps trait + Socket enum + CMsgData + RecvOptions/SendOptions + Pollable impl
src/net_stack.rs	—	NetStack（per-namespace 网络状态容器）
src/listen_table.rs	—	ListenTable（TCP 被动连接管理）
src/router.rs	—	Router（IPv4/IPv6 路由查找）
src/wrapper.rs	—	SocketSetWrapper（互斥锁保护的smoltcp socket set）
src/state.rs	—	TCP 连接状态机 （Idle/Bound/Listening/Connected/Shutdown）
src/options.rs	—	Configurable trait + GeneralOptions + GetSocketOption/SetSocketOption
src/general.rs	290	socket option 的通用实现
src/consts.rs	—	网络栈常量（buffer sizes、MSS 等）
src/tcp.rs	649	TcpSocket（含smoltcp 封装和loopback 优化）
src/udp.rs	439	UdpSocket
src/unix/mod.rs	—	Unix socket 模块入口
src/unix/stream.rs	359	Unix stream socket
src/unix/dgram.rs	418	Unix datagram socket
src/vsock/mod.rs	—	vsock 模块入口
src/vsock/stream.rs	379	vsock stream socket
src/vsock/connection_manager.rs	543	vsock connection manager
src/device/mod.rs	—	设备层入口
src/device/loopback.rs	—	loopback 设备
src/device/ethernet.rs	341	以太网设备
src/device/veth.rs	—	veth pair

文件	行数	核心内容
src/device/vsock.rs	280	vsock 设备

B.3 third_party/rust-patches/ — 第三方补丁

Patch	行数	核心修改
axcpu/	—	per-arch trap handler、context switch、signal frame 布置
axtask/	—	扩展 task extension (TaskExt trait)、wait queue、timers
axsched/	—	CFS + FIFO + RR 调度器实现
memory_set/	—	VMA 管理、append-biased area placement、present page 收集
page_table_multiarch/	—	多架构页表操作、leaf scan、batch unmap
axfs-ng/	—	ext4/fat 的 high-level 文件系统接口
axfs-ng-vfs/	—	VFS 层：挂载、路径、节点操作
starry-process/	—	PID/Process/ProcessGroup/Session 抽象
starry-signal/	—	signal action/pending/types + per-arch 实现
starry-smoltcp/	—	TCP window/timestamp handling fix + 集成到 TheKernel
starry-vm/	—	用户态内存读写 (vm_load/vm_write_slice)
lwext4_rust/	—	ext4 filesystem driver
kernel-elf-parser/	—	ELF header、auxv、user stack 解析
axdriver/	—	virtio-blk/virtio-net/virtio-gpu + PCI/MMIO transport
axdriver_virtio/	—	virtio blk/net/gpu/input/socket device drivers